01418331 Theory of Computation

โจทย์ + เฉลยละเอียด (Final)

บทและหัวข้อตรงกับเล่มสอนตามโพยทุกหัว

โจทย์ 4.5.1 = ฝึกหัวข้อ 4.5 ในเล่มสอน
แต่ละข้อ: โจทย์ → แนวคิด → เฉลยทีละขั้น → เขียนตอบ (English) → จุดพลาด

วิธีใช้เล่มนี้

- ปิดเฉลย ลองทำเองก่อน แล้วค่อยเทียบทีละขั้น ทำไม่ได้ → กลับไปอ่านหัวข้อเลขเดียวกันในเล่มสอน
- กล่อง**ฟ้า** = โจทย์ กล่อง**เหลืองอำพัน** = แนวคิด กล่อง**เขียว** = เฉลยละเอียด กล่อง**ชมพู** = สอนแทรก
- กล่อง**ม่วง** = นิยาม กล่อง**ส้ม** = ทฤษฎีบท กล่องแดง = จุดที่เสียคะแนนบ่อย
- กล่องเหลืองกรอบดำ “เขียนตอบข้อสอบ (English)” = คัดไปเขียนตอบได้ทันที **ไฮไลต์เหลือง** = คำตอบสุดท้าย / จุดสำคัญ

สารบัญ

0 ปูพื้นก่อนอ่าน: วิชานี้ถามอะไร 1

1 CFG และ Chomsky Normal Form 8

2 Pushdown Automata 18

3 CFG $\Leftrightarrow$ PDA 29

4 ภาษาที่ไม่ใช่ CFL: Pumping Lemma 40

5 Turing Machines 53

6 Variants of Turing Machines 67

7 Church–Turing Thesis 77

8 Decidable Languages 82

9 Undecidability 98

10 Reducibility 108

11 Basic Complexity: P, NP, NP-complete 120

12 เห็นโจทย์แล้วใช้สูตรไหน 133

13 ตารางสรุป & จุดพลาดก่อนเข้าห้องสอบ 140

บทที่ 0

ปูพื้นก่อนอ่าน: วิชานี้ถามอะไร

0.1 ภาษา = ปัญหา

โจทย์ 0.1.1 — เขียนปัญหาเป็นภาษา

เขียนปัญหา yes/no ต่อไปนี้เป็นภาษา (เซตของ string)

1. DFA B accept string w หรือไม่
2. directed graph G มี path จาก s ไป t หรือไม่
3. TM M หยุดเมื่อรันบน w หรือไม่

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- รูปเดียวใช้ได้ทุกข้อ: $\{\langle \text{instance} \rangle \mid \text{คำตอบของ instance คือ yes}\}$
- object ทุกตัว (DFA, กราฟ, TM) ต้องอยู่ใน $\langle \cdot \rangle$ เพราะ input ของ TM เป็น string

เฉลยทีละข้อ

1. $A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts } w\}$
2. $PATH = \{\langle G, s, t\rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t\}$
3. $HALT_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ halts on } w\}$

! จุดที่คนเสียคะแนนบ่อย

- เขียน $\{B, w \mid \dots\}$ โดยไม่มี $\langle \cdot \rangle$ — ภาษาต้องเป็นเซตของ string

สอนแทรก

- ข้อสอบชอบให้ “formulate as a language” ก่อนพิสูจน์ — เขียนชนิดของ object ในเงื่อนไขเสมอ (“B is a DFA”)
- ชื่อมาตรฐาน: A = accept, E = empty, EQ = equal, ALL = รับทุก string

0.2 ชั้นของภาษา (หัวหอม)

โจทย์ 0.2.1 — จัดภาษาเข้าชั้นที่เล็กที่สุด

ภาษาต่อไปนี้อยู่ในชั้นใดที่**เล็กที่สุด** (Regular / CFL / Decidable / Recognizable / ไม่ recognizable)

1. 0^*1^*
2. $\{0^n1^n \mid n \geq 0\}$
3. $\{ww^{\mathcal{R}}\}$
4. $\{a^nb^nc^n\}$
5. A_{TM}
6. $\overline{A_{\text{TM}}}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ถามตัวเองตามลำดับ: DFA ทำได้ไหม → stack เดียวพอไหม → TM ที่หยุดเสมอทำได้ไหม → TM ที่ loop ได้ทำได้ไหม

เฉลยทีละข้อ

ภาษา	ชั้นที่เล็กสุด	เหตุผล
0^*1^*	Regular	regex ตรง ๆ
$\{0^n1^n\}$	CFL	$S \to 0S1 \mid \varepsilon$; DFA นับไม่ได้
$\{ww^{\mathcal{R}}\}$	CFL	$S \to 0S0 \mid 1S1 \mid \varepsilon$; ไม่ regular
$\{a^nb^nc^n\}$	Decidable	TM ขีดฆ่าได้; pumping lemma บอกว่าไม่ CFL
A_{TM}	Recognizable	U recognize ได้; Thm 4.11 บอกว่าไม่ decidable
$\overline{A_{\text{TM}}}$	ไม่ recognizable	Cor 4.23

สอบแทรก

- “ชั้นที่เล็กที่สุด” ต้องบอก 2 อย่าง: อยู่ในชั้นนี้ (สร้างเครื่องให้ดู) **และ**ไม่อยู่ชั้นที่เล็กกว่า (อ้างทฤษฎีบท/pumping)
- ตอบแค่ชื่อชั้นได้คะแนนไม่เต็ม — ใส่เหตุผล 1 บรรทัดทุกตัว

0.3 Witness: ตัวอย่างที่แยกแต่ละชั้น

โจทย์ 0.3.1 — Give an example

ยกตัวอย่างภาษาพร้อมเหตุผล 1 บรรทัด

1. context-free แต่ไม่ regular
2. decidable แต่ไม่ context-free
3. Turing-recognizable แต่ไม่ decidable
4. ไม่ Turing-recognizable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละข้อ

1. $\{0^n1^n\}$ — มี CFG $S \to 0S1 \mid \varepsilon$; DFA มี state จำกัด จำจำนวน 0 ไม่ได้
2. $\{a^nb^nc^n\}$ — TM ขีดฆ่า a, b, c รอบละตัว; pumping lemma (Ex 2.36) บอกว่าไม่ CFL
3. A_{TM} — universal TM recognize ได้; diagonalization (Thm 4.11) บอกว่าไม่ decidable
4. $\overline{A_{\text{TM}}}$ — ถ้า recognizable จะทำให้ A_{TM} decidable (Thm 4.22) ขัดแย้ง

เขียนตอบข้อสอบ (English)

1. $\{0^n1^n \mid n \geq 0\}$ is generated by $S \to 0S1 \mid \varepsilon$ but is not regular (pumping lemma for regular languages).
2. $\{a^nb^nc^n \mid n \geq 0\}$ is decided by a TM that crosses off one a, b, c per pass, but it is not context free (Example 2.36).
3. A_{TM} is recognized by the universal TM U, but it is undecidable (Theorem 4.11).
4. $\overline{A_{\text{TM}}}$ is not Turing-recognizable (Corollary 4.23).

สอนแทรก

- 4 witness นี้ใช้ตอบ “give an example” ได้เกือบทุกข้อ — ท่องให้ขึ้นใจพร้อมเหตุผล
- ต้องให้เหตุผล**สองฝั่ง**: อยู่ในชั้นบน (เครื่อง) + ไม่อยู่ชั้นล่าง (ทฤษฎีบท)

0.4 Recognize vs Decide (หัวใจครึ่งหลัง)

โจทย์ 0.4.1 — เครื่องที่ loop

ให้ M = “On input w: ถ้าตัวแรกเป็น 1 ให้ accept ไม่งั้นเดินขวาไปเรื่อย ๆ ไม่หยุด”

1. $L(M)$ คืออะไร
2. M เป็น decider ไหม
3. $L(M)$ decidable ไหม
4. จริงหรือเท็จ: “ถ้า M ไม่ใช่ decider แล้ว $L(M)$ undecidable”

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- “decidable” ถามว่า**มีเครื่องใดสักตัว** decide ได้ไหม ไม่ได้ถามเครื่องที่เห็นอยู่

เฉลยละเอียด

1. $L(M) = \{w \mid w$ **ขึ้นต้นด้วย 1**$\}$
   - $L(M)$ = เซตของ string ที่ M **accept** เท่านั้น
   - ขึ้นต้นด้วย 1 $\rightarrow$ accept $\rightarrow$ อยู่ใน $L(M)$
   - ε หรือขึ้นต้นด้วย 0 $\rightarrow$ M loop $\rightarrow$ **ไม่ใช่ accept** $\rightarrow$ ไม่อยู่ใน $L(M)$
2. M **ไม่ใช่ decider**
   - decider ต้อง**หยุดทุก input**
   - บน $w = 0$ (หรือ ε) M เดินขวาไม่หยุด $\rightarrow$ มี input ที่ไม่หยุด
3. $L(M)$ **decidable**
   - สร้างเครื่องใหม่ M' = “อ่านตัวแรก: เป็น 1 $\rightarrow$ accept; เป็น 0 หรือ $\sqcup$ $\rightarrow$ reject”
   - M' อ่านช่องเดียวแล้วหยุดเสมอ และ accept พอดีกับ $L(M)$ $\rightarrow$ M' decide $L(M)$
4. **เท็จ** — ข้อ 2–3 เป็นตัวอย่างค้าน: M ไม่ใช่ decider แต่ $L(M)$ decidable

สอนแทรก

- “decider” เป็นคุณสมบัติของ**เครื่อง** “decidable” เป็นคุณสมบัติของ**ภาษา**
- ภาษาหนึ่งมีเครื่อง recognize ได้ไม่จำกัดตัว ขอแค่**มี 1 ตัว**ที่หยุดเสมอ ภาษานั้นก็ decidable
- ในข้อสอบถ้าโจทย์ให้เครื่องที่ loop มา อย่ารีบตอบว่า undecidable — ลองหาเครื่องที่ดีกว่า

โจทย์ 0.4.2 — Proof: decidable $\subsetneq$ recognizable

พิสูจน์ว่า (1) ทุกภาษา decidable เป็น Turing-recognizable (2) การรวมนี้เป็น proper subset

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- (1) ตามนิยาม: decider ก็เป็น TM ที่ recognize ภาษาเดียวกัน
- (2) ต้องมี witness ที่ recognizable แต่ไม่ decidable → A_{TM}

เขียนตอบข้อสอบ (English)

1. Let L be decidable and M a decider for L. Then M accepts exactly the strings in L, so M recognizes L, and L is Turing-recognizable.
2. A_{TM} is Turing-recognizable (the universal TM U recognizes it) but undecidable (Theorem 4.11). So the inclusion is proper.

สอบแทรก

- proper subset ต้องพิสูจน์ 2 ส่วนเสมอ: "$\subseteq$" (ทุกตัวอยู่) + "$\neq$" (witness 1 ตัว)

โจทย์ 0.4.3 — Sipser Problem 3.22 — ภาษาที่เราไม่รู้คำตอบ

ให้ A มี string เดียวคือ s โดย $s = 0$ ถ้าจะไม่มีวันพบสิ่งมีชีวิตบนดาวอังคาร และ $s = 1$ ถ้าจะพบในวันหนึ่ง A decidable ไหม

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- decidable ถามว่า**มี** TM ที่ decide หรือไม่ ไม่ได้ถามว่าเรารู้ไหมว่าเป็นตัวไหน

เฉลยทีละขั้น

1. คำตอบมีแน่นอน $\rightarrow A = \{0\}$ หรือ $A = \{1\}$
2. M_0 = "$w = 0 \rightarrow$ accept ไม่งั้น reject" decide $\{0\}$
3. M_1 = "$w = 1 \rightarrow$ accept ไม่งั้น reject" decide $\{1\}$
4. ไม่ว่ากรณีไหน มี decider ตัวหนึ่ง $\rightarrow$ A **decidable**

- "ไม่รู้ว่าเครื่องไหน" $\neq$ "ไม่มีเครื่อง"
- ภาษาจำกัดทุกภาษา decidable undecidability มาจาก input ที่มี**ไม่จำกัด**

เขียนตอบข้อสอบ (English)

A is either $\{0\}$ or $\{1\}$. Both are finite, hence decidable (e.g. M_0 = "accept iff $w = 0$"). In either case some TM decides A, so A is decidable, even though we do not know which TM it is.

สอนแทรก

- decidable เป็นคุณสมบัติของ**เซต**ที่ตายตัว ไม่ขึ้นกับว่าเรารู้หรือไม่ — ภาษาจำกัดทุกภาษา decidable

0.5 เครื่องทั้งหมดในตารางเดียว

โจทย์ 0.5.1 — เติมตาราง

เติมจำนวน tuple, type ของ δ และบอกว่า nondeterministic ไหม สำหรับ DFA, NFA, PDA, TM, NTM

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

เครื่อง	tuple	δ	nondet?
DFA	5: $(Q, \Sigma, \delta, q_0, F)$	$Q \times \Sigma \to Q$	ไม่
NFA	5: $(Q, \Sigma, \delta, q_0, F)$	$Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$	ใช่
PDA	6: $(Q, \Sigma, \Gamma, \delta, q_0, F)$	$Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$	ใช่
TM	7: $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$	$Q \times \Gamma \to Q \times \Gamma \times \{\text{L}, \text{R}\}$	ไม่
NTM	7: เหมือน TM	$Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\text{L}, \text{R}\})$	ใช่

อ่าน δ ทีละตัว

- **ด้าน input ของ δ** = “เครื่องดูอะไรก่อนตัดสินใจ” DFA ดู state + ตัวอักษร PDA ดูเพิ่มยอด stack TM ดูตัวใต้หัว (ใน Γ เพราะอาจเป็นตัวที่เขียนทับไว้)
- **ด้าน output** = “ทำอะไรต่อ” PDA ได้ state + สิ่งที่ push TM ได้ state + ตัวที่เขียน + ทิศ L/R
- Σ_ε, Γ_ε = ยอมให้ “ไม่อ่าน” หรือ “ไม่แตะ stack”

สอนแทรก

- จำ tuple แบบต่อยอด: DFA 5 $\to$ PDA เพิ่ม Γ = 6 $\to$ TM เปลี่ยน F เป็น $q_{\text{accept}}, q_{\text{reject}}$ = 7
- เห็น $\mathcal{P}(\cdot)$ ในผลลัพธ์ = “ได้หลายทางเลือก” = nondeterministic
- PDA ใน Sipser เป็น nondeterministic **โดยนิยาม** (DPDA เป็นอีกโมเดลที่อ่อนกว่า)

บทที่ 1

CFG และ Chomsky Normal Form

1.1 CFG

โจทย์ 1.1.1 — derivation, parse tree, ambiguity

1. เขียน G ของ $S \to aSb \mid \varepsilon$ เป็น 4-tuple
2. เขียน leftmost derivation ของ $aabb$
3. แสดงว่า $E \to E + E \mid E \times E \mid a$ เป็น ambiguous

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ambiguous = มี string ที่มี **leftmost derivation 2 แบบ** (หรือ parse tree 2 ต้น) — ยกมา 1 string พอ

เฉลยทีละขั้น

1. $G = (\{S\}, \{a, b\}, R, S)$ โดย $R = \{S \to aSb,\ S \to \varepsilon\}$
2. $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$
3. string $a + a \times a$ มี leftmost derivation 2 แบบ
   - $E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a \quad (= (a + a) \times a)$
   - $E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E \times E \Rightarrow a + a \times E \Rightarrow a + a \times a \quad (= a + (a \times a))$

! จุดที่คนเสียคะแนนบ่อย

- ยก derivation 2 แบบที่**ไม่ใช่ leftmost ทั้งคู่** — ต่างกันแค่ลำดับการแทนไม่นับว่า ambiguous

สอนแทรก

- leftmost derivation 1 แบบ ⇔ parse tree 1 ต้น — ambiguous ต้อง ยก string + derivation **2 แบบ ที่เป็น leftmost ทั้งคู่**
- inherently ambiguous = ทุก grammar ของภาษานั้น ambiguous (เช่น $\{a^i b^j c^k \mid i = j \text{ หรือ } j = k\}$)

โจทย์ 1.1.2 — เขียน CFG แบบ formal

เขียน CFG ของ palindrome ทั้งหมดบน $\{0,1\}$ เป็น 4-tuple (V, Σ, R, S) โดยแจกแจง R ทีละกฎ

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

$G = (V, \Sigma, R, S)$ โดย

1. $V = \{S\}$ — มีตัวแปรตัวเดียว
2. $\Sigma = \{0,1\}$ — terminal = ตัวอักษรของ string
3. R มี 5 กฎ (แยก | ออกเป็นกฎเดี่ยว):
   - $S \to 0S0, \quad S \to 1S1$ — เติม 0 หรือ 1 ทั้งสองข้างพร้อมกัน (ขอบนอกเข้าหากลาง)
   - $S \to 0, \quad S \to 1$ — ตัวกลางของ palindrome ยาวคี่
   - $S \to \varepsilon$ — ปิดท้ายของ palindrome ยาวคู่ (และ ε เองเป็น palindrome)
4. start variable $= S$

ตรวจ: 0110: $S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 0110$ ✓ $\quad 101$: $S \Rightarrow 1S1 \Rightarrow 101$ ✓

สอนแทรก

- ข้อสอบ “give the formal definition” ต้องเขียน**ครบ 4 ส่วน**และแจกแจง R — เขียนแค่กฎ $S \to \ldots$ ไม่ครบ
- $V \cap \Sigma = \emptyset$ เสมอ (ตัวแปรกับ terminal เป็นคนละเซต)
- Neso เขียน (V, Σ, S, P) — ในข้อสอบใช้ลำดับของ Sipser (V, Σ, R, S)

1.2 ออกแบบ CFG: 6 ท่า

โจทย์ 1.2.1 — Give a CFG

1. $\{a^i b^j c^k \mid i = j + k\}$
2. $\{a^i b^j \mid i \geq 2j\}$
3. $\{w \in \{0,1\}^* \mid w$ มี 0 กับ 1 จำนวนเท่ากัน$\}$
4. palindrome ทั้งหมดบน $\{a, b\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ข้อ 1: แตกเลขชี้กำลัง $a^{j+k}b^j c^k = a^k\,(a^j b^j)\,c^k$ → ซ้อนกัน
- ข้อ 2: ทุก b ต้องมาพร้อม aa แล้วเติม a เกินได้
- ข้อ 3: แยกที่จุดกลับมาสมดุลครั้งแรก
- ข้อ 4: สร้างจากขอบเข้ากลาง และมีตัวกลางได้

เฉลยทีละขั้น

1. $S \to aSc \mid T, \quad T \to aTb \mid \varepsilon$
   - ตรวจ $aabc$ $(i = 2, j = 1, k = 1)$: $S \Rightarrow aSc \Rightarrow aTc \Rightarrow aaTbc \Rightarrow aabc$ ✓
2. $S \to aS \mid aaSb \mid \varepsilon$
   - ทุก b มาพร้อม aa ทางซ้าย → $i \geq 2j$ $\quad b$ ต่อท้ายขวาเสมอ → a อยู่ก่อน b
   - ตรวจ $aaab$: $S \Rightarrow aS \Rightarrow aaaSb \Rightarrow aaab$ ✓
3. $S \to 0S1S \mid 1S0S \mid \varepsilon$
   - ตรวจ 1001: $S \Rightarrow 1S0S \Rightarrow 10S \Rightarrow 100S1S \Rightarrow 1001S \Rightarrow 1001$ ✓
4. $S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$

! จุดที่คนเสียคะแนนบ่อย

- ข้อ 2 เขียน $S \to aaSb \mid \varepsilon$ อย่างเดียว → ได้แค่ $i = 2j$ ขาด a ส่วนเกิน
- ข้อ 3 เขียน $S \to 0S1 \mid 1S0 \mid \varepsilon$ → สร้าง 0110 ไม่ได้ (ต้องมี S ตัวที่สองต่อท้าย)

สอนแทรก

- ข้อสอบมักให้คะแนนเหตุผลด้วย: เขียน 1 บรรทัดว่าแต่ละกฎทำอะไร + derive string ตัวอย่าง 1 ตัว
- ถ้าโจทย์บอก “prove” ต้องพิสูจน์ $L(G) \subseteq A$ และ $A \subseteq L(G)$ (induction)

โจทย์ 1.2.2 — Proof: CFL ปิดภายใต้ union, concatenation, star

ให้ $G_1 = (V_1, \Sigma, R_1, S_1)$ และ $G_2 = (V_2, \Sigma, R_2, S_2)$ โดย $V_1 \cap V_2 = \emptyset$ สร้าง CFG ของ $L_1 \cup L_2$, $L_1 L_2$, L_1^* และอธิบายว่าทำไมถูก

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- เพิ่ม start ใหม่ S แล้วต่อเข้ากับ S_1, S_2 ตาม operation
- ตัวแปรสองชุดต้องไม่ซ้ำกัน ไม่งั้นกฎปนกัน

เขียนตอบข้อสอบ (English)

In each case the new grammar has variables $V_1 \cup V_2 \cup \{S\}$, rules $R_1 \cup R_2$ plus the rules below, and start variable S.

- Union: add $S \to S_1 \mid S_2$. A derivation from S first picks S_1 or S_2, then derives a string of L_1 or L_2; so the language is $L_1 \cup L_2$.
- Concatenation: add $S \to S_1 S_2$. Since $V_1 \cap V_2 = \emptyset$, S_1 derives exactly L_1 and S_2 exactly L_2; so the language is $L_1 L_2$.
- Star: add $S \to S S_1 \mid \varepsilon$. Applying $S \to S S_1$ k times and then $S \to \varepsilon$ gives $S \overset{*}{\Rightarrow} S_1^k$, which derives exactly the strings of L_1^k; so the language is $\bigcup_k L_1^k = L_1^*$.

สอนแทรก

- ต้องบอกว่า**เปลี่ยนชื่อตัวแปรให้ไม่ซ้ำ**ก่อนรวม grammar — ไม่งั้นกฎของ G_1 ปนกับ G_2
- star ใช้ $S \to S S_1 \mid \varepsilon$ (ได้ ε ด้วย) — ห้ามลืม ε

1.3 นิยาม CNF

โจทย์ 1.3.1 — กฎไหนเป็น CNF

ให้ S_0 เป็น start variable กฎใดอยู่ในรูป CNF

1. $S_0 \to \varepsilon$
2. $A \to BC$
3. $A \to a$
4. $A \to aB$
5. $A \to BCD$
6. $A \to \varepsilon$
7. $A \to B$
8. $A \to BS_0$

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

CNF อนุญาตแค่ $A \to BC$ (B, C ไม่ใช่ start), $A \to a$, และ $S_0 \to \varepsilon$

ข้อ	กฎ	ผล / เหตุผล
1	$S_0 \to \varepsilon$	CNF — ข้อยกเว้นเดียว: start ให้ ε ได้
2	$A \to BC$	CNF — ตัวแปร 2 ตัวพอดี
3	$A \to a$	CNF — terminal 1 ตัวพอดี
4	$A \to aB$	ผิด — terminal ปนตัวแปร → แก้เป็น $A \to UB$, $U \to a$
5	$A \to BCD$	ผิด — ยาว 3 → แก้เป็น $A \to BA_1$, $A_1 \to CD$
6	$A \to \varepsilon$	ผิด — ε-rule ของตัวที่ไม่ใช่ start → ต้องลบในขั้น 2
7	$A \to B$	ผิด — unit rule → ต้องลบในขั้น 3
8	$A \to BS_0$	ผิด — start อยู่ด้านขวา (ขั้น 1 ทำให้ไม่เกิดกรณีนี้)

สอนแทรก

- ดูกฎที่ผิดแล้ว**บอกได้ว่าขั้นไหน**ของ algorithm จะแก้: ข้อ 8 → ขั้น 1, ข้อ 6 → ขั้น 2, ข้อ 7 → ขั้น 3, ข้อ 4–5 → ขั้น 4
- ทำไมห้าม start ด้านขวา: ถ้า S_0 มี $S_0 \to \varepsilon$ และอยู่ด้านขวา จะสร้าง ε กลาง string ได้ → derivation ไม่ยาว $2n-1$

1.4 แปลง CFG → CNF: 4 ขั้น (ห้ามสลับลำดับ)

โจทย์ 1.4.1 — Convert to CNF (Sipser Exercise 2.14)

แปลง $A \to BAB \mid B \mid \varepsilon, \quad B \to 00 \mid \varepsilon$ (start $= A$) เป็น CNF

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ทั้ง A และ B เป็น nullable → ระวังลูกโซ่ ε และกฎที่เคยลบแล้ว
- ระวัง: อย่าเพิ่ม $A \to \varepsilon$ กลับหลังลบไปแล้ว (วนไม่จบ) และอย่าลืม $S_0 \to \varepsilon$ (ภาษาเดิมมี ε)

เฉลยทีละขั้น

1. **เพิ่ม start:** $S_0 \to A$
2. **ลบ $A \to \varepsilon$:**
   - $S_0 \to A \to$ เพิ่ม $S_0 \to \varepsilon$
   - $A \to BAB \to$ เพิ่ม $A \to BB$

   $$S_0 \to A \mid \varepsilon \qquad A \to BAB \mid BB \mid B \qquad B \to 00 \mid \varepsilon$$
3. **ลบ $B \to \varepsilon$:**
   - $A \to BAB$ (B 2 ตัว) → เพิ่ม AB, BA, A
   - $A \to BB \to$ เพิ่ม B; ตัดทั้งคู่ได้ $A \to \varepsilon$ แต่**เคยลบแล้ว** → ไม่เพิ่ม
   - $A \to B \to$ ได้ $A \to \varepsilon$ เคยลบแล้ว → ไม่เพิ่ม

   $$S_0 \to A \mid \varepsilon \qquad A \to BAB \mid BB \mid B \mid AB \mid BA \mid A \qquad B \to 00$$
4. **ลบ unit:**
   - $A \to A$ ทิ้ง
   - $A \to B \to A \to 00$
   - $S_0 \to A \to S_0$ ได้กฎทั้งหมดของ A

   $$S_0 \to BAB \mid BB \mid AB \mid BA \mid 00 \mid \varepsilon \qquad A \to BAB \mid BB \mid AB \mid BA \mid 00 \qquad B \to 00$$
5. **แปลงรูป:** $BAB \to BA_1$ กับ $A_1 \to AB$ $\quad 00 \to UU$ กับ $U \to 0$

   $$S_0 \to BA_1 \mid BB \mid AB \mid BA \mid UU \mid \varepsilon \quad A_1 \to AB$$
   $$A \to BA_1 \mid BB \mid AB \mid BA \mid UU \qquad B \to UU$$
   $$U \to 0$$

- ตรวจ: ทุกกฎ = 2 ตัวแปร หรือ 1 terminal S_0 ไม่อยู่ด้านขวา ✓

สอนแทรก

- ข้อนี้มาจากแบบฝึกหัด Sipser — เขียนผลลัพธ์ของ**ทุกขั้น**ไว้ให้เห็น อาจารย์ให้คะแนนรายขั้น
- เช็กปลายทาง: ทุกกฎ 2 ตัวแปร / 1 terminal, S_0 ไม่อยู่ด้านขวา, มี $S_0 \to \varepsilon$ ถ้าภาษามี ε

โจทย์ 1.4.2 — นับขั้น derivation

grammar แบบ CNF สร้าง string ยาว 6 ใช้ derivation กี่ขั้น

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. เริ่มด้วย 1 สัญลักษณ์ (S) จบด้วย 6 terminal
2. **กฎ $A \to BC$:** ทำให้ string ยาวขึ้น 1 ทุกครั้ง → จาก 1 เป็น 6 ต้องใช้ $6 - 1 = 5$ ครั้ง
3. **กฎ $A \to a$:** เปลี่ยนตัวแปรเป็น terminal ทีละตัว → ต้องใช้ 6 ครั้ง
4. รวม $5 + 6 = 11 = 2n - 1$ **ขั้น** — **ทุก** derivation เท่ากันหมด ไม่ว่าเลือกกฎลำดับไหน

ตัวอย่างจริง ($S \to AB \mid a$ แบบย่อ): ไม่ต้องมี grammar จริงก็ตอบได้ เพราะสูตรขึ้นกับ n อย่างเดียว

สอนแทรก

- ใช้ทำอะไร: decider ของ A_{CFG} (Thm 4.7) ลองเฉพาะ derivation ยาว $2n - 1$ ขั้น → **จำนวนจำกัด** → หยุดเสมอ
- ข้อยกเว้น $n = 0$: มีแค่ $S_0 \Rightarrow \varepsilon$ = 1 ขั้น (สูตร $2n - 1$ ใช้กับ $n \geq 1$)

โจทย์ 1.4.3 — Proof: derivation ใน CNF ยาว $2n-1$ (Sipser Problem 2.26)

ให้ G อยู่ในรูป CNF พิสูจน์ว่าทุก derivation ของ $w \in L(G)$ ที่ $|w| = n \geq 1$ ใช้ $2n-1$ ขั้นพอดี

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- นับ 2 ปริมาณ: จำนวนสัญลักษณ์ทั้งหมด และจำนวน terminal
- $A \to BC$ เพิ่มสัญลักษณ์ 1 ตัว $A \to a$ เพิ่ม terminal 1 ตัว

เขียนตอบข้อสอบ (English)

Consider any derivation $S \Rightarrow \cdots \Rightarrow w$ with $|w| = n \geq 1$. Since $n \geq 1$, the rule $S \to \varepsilon$ is not used.

- A rule $A \to BC$ increases the length of the current string by exactly 1 and does not create terminals.
- A rule $A \to a$ keeps the length the same and turns exactly one variable into a terminal.
- The derivation starts with 1 symbol and ends with n symbols, so it uses $A \to BC$ exactly $n-1$ times.
- It starts with 0 terminals and ends with n terminals, so it uses $A \to a$ exactly n times.

Hence every derivation has $(n-1)+n = 2n-1$ steps. ■

สอนแทรก

- บทพิสูจน์แบบนับ: หา “ปริมาณที่เปลี่ยนทีละ 1” แล้วนับจากต้นถึงปลาย — ใช้ซ้ำได้กับโจทย์นับหลายแบบ

โจทย์ 1.4.4 — แปลง CNF: $S \to aSb \mid \varepsilon$ (TOC_Practice 1.1)

แปลง $S \to aSb \mid \varepsilon$ เป็น Chomsky normal form แล้วตรวจกับ $aabb$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ภาษามี $\varepsilon \to$ ต้องเหลือ $S_0 \to \varepsilon$ S อยู่ด้านขวาของตัวเอง $\to$ ต้องมี S_0 ใหม่

เฉลยทีละขั้น

1. **start ใหม่:** $S_0 \to S,\ S \to aSb \mid \varepsilon$
2. **ลบ $S \to \varepsilon$:**
   - $S \to aSb \to$ เพิ่ม $S \to ab$
   - $S_0 \to S \to$ เพิ่ม $S_0 \to \varepsilon$ (อนุญาต)
3. **ลบ unit $S_0 \to S$:** $S_0 \to aSb \mid ab \mid \varepsilon$
4. **แปลงรูป:** $A \to a$, $B \to b$, $aSb \to AT$ กับ $T \to SB$, $ab \to AB$

$$\boxed{S_0 \to AT \mid AB \mid \varepsilon \quad S \to AT \mid AB \quad T \to SB \quad A \to a \quad B \to b}$$

- ตรวจ $aabb$: $S_0 \Rightarrow AT \Rightarrow aT \Rightarrow aSB \Rightarrow aABB \Rightarrow aaBB \Rightarrow aabB \Rightarrow aabb$ — 7 ขั้น $= 2 \cdot 4 - 1$ ✓

สอนแทรก

- กฎ $S \to aSb$ มี S ด้านขวา $\to$ ขั้น 1 จำเป็นจริง ๆ (ถ้าไม่เพิ่ม S_0 จะมี $S \to \varepsilon$ และ S ด้านขวาพร้อมกัน)

โจทย์ 1.4.5 — แปลง CNF: มีทั้ง ε-rule และ unit rule (TOC_Practice 1.2)

แปลง $S \to aXbX, \quad X \to aY \mid bY \mid \varepsilon, \quad Y \to X \mid c$ เป็น CNF

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- X nullable ตรง ๆ Y nullable **ทางอ้อม** (ผ่าน $Y \to X$)
- $aXbX$ มี X 2 ตัว → เพิ่ม $2^2 - 1 = 3$ แบบ

เฉลยทีละขั้น

1. **start ใหม่:** $S_0 \to S$
2. **ลบ $X \to \varepsilon$:**
   - $S \to aXbX$ → เพิ่ม abX, aXb, ab
   - $Y \to X$ → ได้ $Y \to \varepsilon$ ใหม่ (ลูกโซ่)
3. **ลบ $Y \to \varepsilon$:** $X \to aY \mid bY$ → เพิ่ม $X \to a \mid b$

$$S_0 \to S \quad S \to aXbX \mid abX \mid aXb \mid ab \quad X \to aY \mid bY \mid a \mid b \quad Y \to X \mid c$$

4. **ลบ unit:**
   - $Y \to X \to Y \to aY \mid bY \mid a \mid b \mid c$
   - $S_0 \to S \to S_0 \to aXbX \mid abX \mid aXb \mid ab$ (S ไม่ถูกใช้อีก → ตัดทิ้ง)
5. **แปลงรูป:** $A \to a, B \to b$
   - $aXbX = AT_1, T_1 \to XT_2, T_2 \to BX \quad abX = AT_2 \quad aXb = AT_3, T_3 \to XB$
   - $ab = AB, \quad aY = AY, \quad bY = BY$

$$S_0 \to AT_1 \mid AT_2 \mid AT_3 \mid AB \quad T_1 \to XT_2 \quad T_2 \to BX \quad T_3 \to XB$$
$$X \to AY \mid BY \mid a \mid b \qquad Y \to AY \mid BY \mid a \mid b \mid c \qquad A \to a \quad B \to b$$

! จุดที่คนเสียคะแนนบ่อย

- ลืมว่า Y nullable ทางอ้อม → ขาด $X \to a \mid b$
- ลบ unit ก่อน ε → เกิด unit ใหม่ที่ไม่ได้ตัด

สอนแทรก

- หา nullable ให้ครบก่อนเริ่ม: X (ตรง) และ Y (ผ่าน $Y \to X$) — เขียนรายการ nullable ไว้ช่วยไม่ลืม

Pushdown Automata

2.1 ทำไมต้องมี stack

โจทย์ 2.1.1 — อธิบายสั้น ๆ

อธิบายว่าทำไม DFA รู้จำ $\{0^n1^n\}$ ไม่ได้ แต่ PDA ทำได้ และทำไม PDA ยังรู้จำ $\{0^n1^n2^n\}$ ไม่ได้

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ทำไม DFA ทำไม่ได้**
   1. สมมติ DFA มี k state ป้อน $\varepsilon, 0, 00, \ldots, 0^k$ ($k+1$ ตัว)
   2. ช่อง state มีแค่ k → pigeonhole: มี $i \neq j$ ที่ 0^i กับ 0^j จบ state เดียวกัน
   3. ต่อ 1^i ทั้งคู่ → DFA ตอบเหมือนกัน แต่ $0^i1^i \in L$ และ $0^j1^i \notin L$ → ขัดแย้ง
2. **ทำไม PDA ทำได้**
   1. push \$ → push 0 ทุกตัว → เจอ 1 pop ทีละตัว → เห็น \$ ตอน input หมด → accept
   2. stack เก็บจำนวนได้ไม่จำกัด (ต่างจาก state)
3. **ทำไม PDA ยังทำ $0^n1^n2^n$ ไม่ได้**
   1. ใช้ stack เช็ก #0 = #1 → 0 ถูก pop หมด → ไม่เหลือข้อมูลไว้เทียบกับ 2
   2. หรือถ้าเก็บ 0 ไว้เทียบกับ 2 → ไม่มีอะไรเทียบกับ 1 (เห็นได้แค่ยอด stack)
   3. ข้อนี้เป็น**สัญชาตญาณ** บทพิสูจน์จริงใช้ pumping lemma (Example 2.36)

สอนแทรก

- "คิด PDA ไม่ออก" ไม่ใช่บทพิสูจน์ — ถ้าโจทย์ถาม "prove" ต้องใช้ pumping lemma หรือ closure
- ข้อ 1 คือไอเดียเดียวกับ pumping lemma ของ regular (state ซ้ำ)

2.2 นิยาม 6-tuple และป้าย $a, b \to c$

โจทย์ 2.2.1 — เขียน PDA แบบ formal

เขียน PDA ของ $\{0^n 1^n \mid n \geq 0\}$ (Example 2.14) เป็น 6-tuple พร้อมตาราง δ

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

- $Q = \{q_1, q_2, q_3, q_4\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{0, \$\}$, start q_1, $F = \{q_1, q_4\}$
- δ ที่ไม่ใช่ $\emptyset$ มี 5 ค่า:

δ(state, อ่าน, pop)	ผล	ความหมาย
$\delta(q_1, \varepsilon, \varepsilon)$	$\{(q_2, \$)\}$	push \$
$\delta(q_2, 0, \varepsilon)$	$\{(q_2, 0)\}$	เก็บ 0
$\delta(q_2, 1, 0)$	$\{(q_3, \varepsilon)\}$	เจอ 1 ตัวแรก pop 0
$\delta(q_3, 1, 0)$	$\{(q_3, \varepsilon)\}$	pop 0 ต่อ 1
$\delta(q_3, \varepsilon, \$)$	$\{(q_4, \varepsilon)\}$	stack ว่าง $\to$ accept

! จุดที่คนเสียคะแนนบ่อย

- ผลของ δ ต้องเป็น**เซต** ของคู่ (state, symbol) ไม่ใช่คู่เดียว
- ลืมใส่ q_1 ใน $F \to$ ไม่รับ ε

สอบแทรก

- formal definition ต้องมีครบ 6 ส่วน + δ ทุกค่าที่ไม่ว่าง (บอกว่าที่เหลือเป็น $\emptyset$)

โจทย์ 2.2.2 — 6-tuple + ตาราง δ แบบ Sipser

PDA ของ $\{ww^{\mathcal{R}} \mid w \in \{0,1\}^*\}$ (Example 2.18) มีเส้นดังนี้

- $q_1 \xrightarrow{\varepsilon,\varepsilon\to\$} q_2$ วน q_2: $0,\varepsilon \to 0$ และ $1,\varepsilon \to 1$ $q_2 \xrightarrow{\varepsilon,\varepsilon\to\varepsilon} q_3$
- วน q_3: $0,0 \to \varepsilon$ และ $1,1 \to \varepsilon$ $q_3 \xrightarrow{\varepsilon,\$\to\varepsilon} q_4$ accept $= q_4$

เขียนเป็น 6-tuple และเขียนตาราง δ (แถว = state, หลัก = input / ยอด stack)

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

6-tuple

- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, \$\}$
- start $= q_1$
- $F = \{q_4\}$

ตาราง δ (ช่องว่าง $= \emptyset$)

input	0				1				ε			
stack	0	1	\$	ε	0	1	\$	ε	0	1	\$	ε
q_1												$\{(q_2,\$)\}$
q_2				$\{(q_2,0)\}$				$\{(q_2,1)\}$				$\{(q_3,\varepsilon)\}$
q_3	$\{(q_3,\varepsilon)\}$					$\{(q_3,\varepsilon)\}$					$\{(q_4,\varepsilon)\}$	
q_4												

- ช่อง $\delta(q_2,\varepsilon,\varepsilon) = \{(q_3,\varepsilon)\}$ คือการ**เดา**กึ่งกลาง
- ทุกช่องเป็น**เซต** แม้มีสมาชิกตัวเดียว

! จุดที่คนเสียคะแนนบ่อย

- ลืมหลัก ε ของ input หรือของ stack → เส้น push \$ และเส้นเดากึ่งกลางหาย
- Γ ลืม \$

สอบแทรก

- ตาราง Sipser: แถว = state, หลัก = (input, ยอด stack) รวมหลัก ε — ช่องว่างเขียน $\emptyset$ หรือบอกไว้ใต้ตาราง

2.3 PDA มาตรฐาน: $\{0^n1^n \mid n \geq 0\}$ + trace

โจทย์ 2.3.1 — trace

ใช้ PDA ข้อ 2.2.1 trace บน (ก) 000111 (ข) 0010 (stack ซ้ายสุด = บนสุด)

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลย (ก) 000111

state	อ่าน	ทำ	stack
q_1	—	push \$ → q_2	`$`
q_2	0,0,0	push 0 สามครั้ง	`000$`
q_2	1	pop → q_3	`00$`
q_3	1	pop	`0$`
q_3	1	pop	`$`
q_3	—	pop \$ → q_4	ว่าง

input หมด อยู่ q_4 → accept

เฉลย (ข) 0010

1. push \$, อ่าน 0, 0 → `00$`
2. อ่าน 1 → pop → q_3, `0$`
3. อ่าน 0 ที่ q_3: ไม่มีลูกศรอ่าน 0 → branch ตาย
4. ทางเลือกอื่น ($q_3 \xrightarrow{\varepsilon,\$\to\varepsilon}$) ต้องมี \$ บนสุด แต่บนสุดเป็น 0 → ไปไม่ได้

ไม่มี branch ไหนถึง accept → reject

สอนแทรก

- trace ที่ได้คะแนนเต็ม: เขียน state + input ที่เหลือ + stack ทุกก้าว และบอกเหตุผลที่ reject (ไม่มีลูกศร / stack ไม่ว่าง)

2.4 Nondeterminism: PDA “เดา” ได้

โจทย์ 2.4.1 — palindrome ทั้งหมด

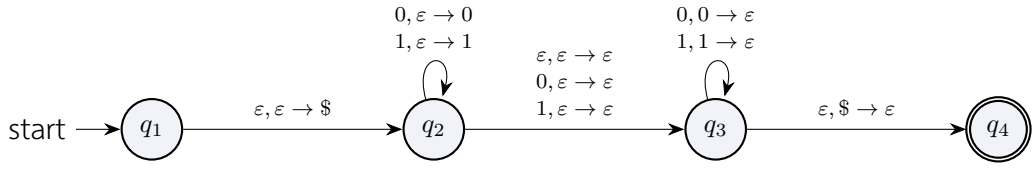
ออกแบบ PDA ของ palindrome ทั้งหมดบน $\{0,1\}$ (ยาวคู่และยาวคี่) แล้ว trace 101

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ใช้ PDA ของ $ww^{\mathcal{R}}$ แล้วเพิ่มเส้น**ข้ามตัวกลาง** 1 ตัว สำหรับกรณียาวคี่

เฉลยทีละขั้น



Trace 101 (branch ที่เดาถูก):

1. push \$ $\to$ `$`
2. อ่าน 1 push $\to$ `1$`
3. เดาว่า 0 เป็นตัวกลาง: อ่าน 0 ด้วย $0, \varepsilon \to \varepsilon \to q_3$, `1$`
4. อ่าน 1 pop 1 $\to$ `$`
5. pop \$ $\to q_4$ input หมด $\to$ accept

- branch อื่น (เช่นเดาว่ากลางอยู่หลัง 10) ตายเอง ไม่เป็นไร

สอนแทรก

- ภาษาคู่+คี่ใช้เส้นเดากลาง 3 แบบ: $\varepsilon, \varepsilon \to \varepsilon$ (คู่), $0, \varepsilon \to \varepsilon$ และ $1, \varepsilon \to \varepsilon$ (ข้ามตัวกลางของคี่)

โจทย์ 2.4.2 — Sipser Exercise 2.4(d), 2.5(d) — ยาวคี่และตัวกลางเป็น 0

$L = \{w \in \{0,1\}^* \mid |w| \text{ คี่ และตัวกลางเป็น } 0\}$ (1) ให้ CFG (2) ให้ PDA และ trace 100

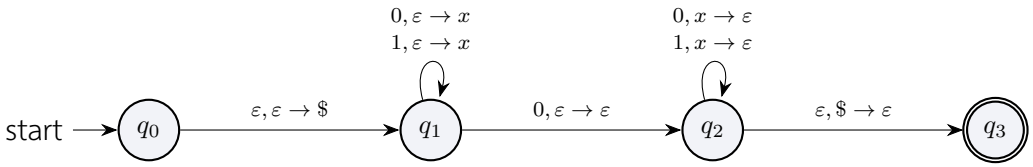
ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- สองข้างของตัวกลาง**ยาวเท่ากัน** แต่เป็นตัวอะไรก็ได้ → นับความยาว ไม่ต้องจำค่า
- PDA เดาตัวกลาง: อ่าน 0 โดยไม่แตะ stack

เฉลยทีละขั้น

CFG: $S \to 0 \mid 0S0 \mid 0S1 \mid 1S0 \mid 1S1$ (เติมทีละคู่รอบแกน 0)
PDA:



1. push x หนึ่งตัวต่อ input หนึ่งตัวของครึ่งแรก
2. **เดา**ตัวกลาง: อ่าน 0 ไม่แตะ stack
3. pop x หนึ่งตัวต่อ input หนึ่งตัวของครึ่งหลัง
4. เจอ $ ตอน input หมด → accept

Trace 100: push $ → อ่าน 1 push x → อ่าน 0 เป็นตัวกลาง → อ่าน 0 pop x → pop $ → accept

สอนแทรก

- push สัญลักษณ์ กลาง x แทน การ จำ 0/1 — ต้องการ แค่**ความ ยาว**เท่า กัน ไม่ ต้อง ตรง กัน (ต่าง จาก palindrome)

โจทย์ 2.4.3 — Sipser Exercise 2.9, 2.10 — $\{a^i b^j c^k \mid i = j \text{ หรือ } j = k\}$

(1) ให้ CFG และบอกว่า ambiguous ไหม (2) อธิบาย PDA

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- "หรือ" = union ของ $\{a^n b^n c^k\}$ กับ $\{a^i b^n c^n\}$
- string ที่ $i = j = k$ อยู่ทั้งสองฝั่ง → derivation 2 แบบ

เฉลยทีละขั้น

CFG:

$$S \to XC \mid AY \qquad X \to aXb \mid \varepsilon \quad C \to cC \mid \varepsilon \qquad A \to aA \mid \varepsilon \quad Y \to bYc \mid \varepsilon$$

Ambiguous: **ใช่** — abc มี leftmost derivation 2 แบบ

- $S \Rightarrow XC \Rightarrow aXbC \Rightarrow abC \Rightarrow abcC \Rightarrow abc$
- $S \Rightarrow AY \Rightarrow aAY \Rightarrow aY \Rightarrow abYc \Rightarrow abc$
- ภาษานี้ inherently ambiguous (Problem 2.29): ทุก grammar ของมัน ambiguous

PDA: push \$ แล้วเดา 2 กิ่ง

- กิ่ง $i = j$: push $a \to$ pop a ต่อ $b \to$ เห็น \$ pop → อ่าน c ทิ้ง
- กิ่ง $j = k$: อ่าน a ทิ้ง → push $b \to$ pop b ต่อ $c \to$ เห็น \$ pop → accept

สอนแทรก

- ภาษา "หรือ" → CFG ใช้ union, PDA ใช้เดาตั้งแต่ต้นว่าเช็กเงื่อนไขไหน

2.5 ตารางท่าออกแบบ PDA

โจทย์ 2.5.1 — Give a PDA

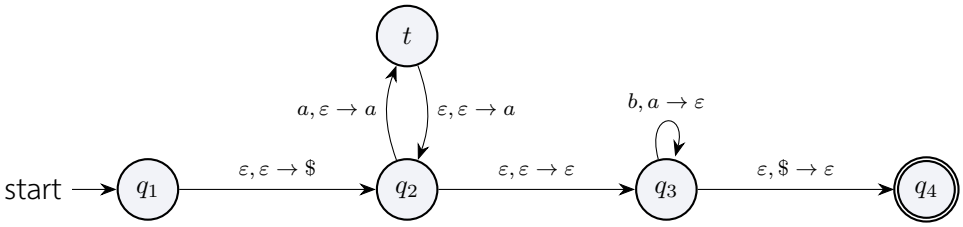
1. $\{a^n b^{2n} \mid n \geq 0\}$
2. $\{a^n b^m c^k \mid m = n + k\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

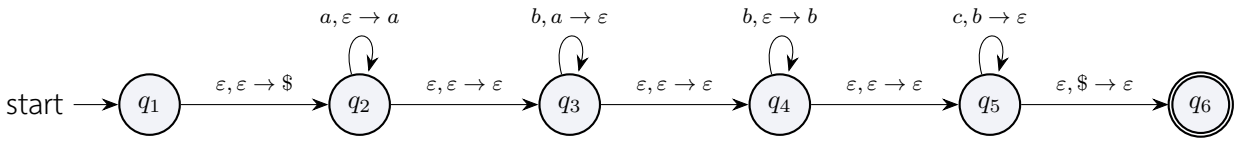
- ข้อ 1: push a **สองตัว**ต่อ a หนึ่งตัว แล้ว pop 1 ต่อ b
- ข้อ 2: $a^n b^n b^k c^k$ = สองก้อนต่อกัน → จับ a–b ก่อน แล้วจับ b–c

เฉลยข้อ 1



- $q_2 \to t \to q_2$ = อ่าน a หนึ่งตัวแล้ว push a สองตัว
- q_3 pop 1 ต่อ $b \to b$ ต้องเป็น 2 เท่าของ a

เฉลยข้อ 2



1. q_2: push a ทั้งหมด (n ตัว)
2. q_3: pop a ต่อ b (n ตัวแรกของ b)
3. q_4: push b ที่เหลือ (k ตัว)
4. q_5: pop b ต่อ c
5. เห็น \$ → accept

- ถ้าเดาย้ายจาก q_3 ไป q_4 ก่อน a หมด: b จะทับบน a แล้วสุดท้ายเจอ a แทน \$ → branch ตาย (ถูกต้อง)

สอนแทรก

- อธิบาย PDA ในข้อสอบ: วาด diagram + เขียน 3–5 ประโยคว่าแต่ละ state ทำอะไร + บอกว่าทำไม input ผิดถูก reject

โจทย์ 2.5.2 — PDA: จำนวน a เท่ากับจำนวน b (TOC_Practice 2.2)

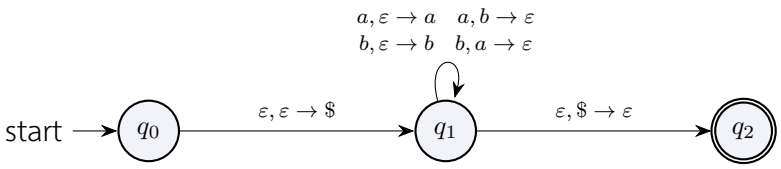
ออกแบบ PDA ของ $\{w \in \{a,b\}^* \mid \#a = \#b\}$ (ลำดับใดก็ได้) แล้ว trace $abba$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- stack เก็บ "ส่วนเกิน" ของตัวที่มีมากกว่า ณ ขณะนั้น
- อาน a: ถ้ายอดเป็น $b \to$ pop (จับคู่) ไม่งั้น push a อาน b ทำกลับกัน

เฉลยทีละขั้น



Trace $abba$ (stack ซ้ายสุด = บนสุด):

$$\$ \xrightarrow{a} a\$ \xrightarrow{b} \$ \xrightarrow{b} b\$ \xrightarrow{a} \$ \xrightarrow{\varepsilon} \text{ว่าง} \to \text{accept}$$

ทำไมถูก:

- ทุกกิ่ง: (#a ใน stack) − (#b ใน stack) = (#a ที่อ่าน) − (#b ที่อ่าน)
- pop \$ ได้เมื่อไม่มี a, b ใน stack $\to$ ผลต่าง $= 0 \to$ จำนวนเท่ากัน
- ถ้าจำนวนเท่ากัน กิ่งที่ "pop ทุกครั้งที่ pop ได้" จบด้วย stack ว่าง

สอนแทรก

- invariant "ใน stack = ผลต่างที่อ่านมา" คือเหตุผลที่ทำให้บทพิสูจน์ความถูกต้องสั้น — เขียนไว้ในคำตอบด้วย

2.6 Closure properties ของ CFL

โจทย์ 2.6.1 — Sipser Exercise 2.2

1. ใช้ $A = \{a^m b^n c^n\}$ และ $B = \{a^n b^n c^m\}$ แสดงว่า CFL ไม่ปิดภายใต้ intersection
2. ใช้ข้อ 1 และ De Morgan แสดงว่า CFL ไม่ปิดภายใต้ complement

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. A, B เป็น CFL
   - A: $S \to aS \mid T,\ T \to bTc \mid \varepsilon$
   - B: $S \to Sc \mid T,\ T \to aTb \mid \varepsilon$
   - $A \cap B = \{a^n b^n c^n\}$ ไม่ใช่ CFL (Ex 2.36) → **ไม่ปิด $\cap$**
2. สมมติ CFL ปิด complement
   - CFL ปิด union อยู่แล้ว → $\overline{\overline{A} \cup \overline{B}} = A \cap B$ เป็น CFL เสมอ
   - ขัดกับข้อ 1 → **ไม่ปิด complement**

เขียนตอบข้อสอบ (English)

1. $A = \{a^m b^n c^n\}$ and $B = \{a^n b^n c^m\}$ are context free, but $A \cap B = \{a^n b^n c^n\}$ is not (Example 2.36). So CFLs are not closed under intersection.
2. Assume CFLs are closed under complement. Since they are closed under union, $A \cap B = \overline{\overline{A} \cup \overline{B}}$ would be context free for all CFLs A, B, contradicting part 1.

สอนแทรก

- ตัวอย่างค้าน $\{a^n b^n c^m\} \cap \{a^m b^n c^n\}$ ใช้ได้ทั้งข้อ T/F และข้อพิสูจน์ — ต้องบอกว่าทั้งสองเป็น CFL เพราะอะไร (ให้ grammar)

โจทย์ 2.6.2 — Proof: CFL ∩ regular = CFL

ให้ P เป็น PDA ของ A และ D เป็น DFA ของ B พิสูจน์ว่า $A \cap B$ เป็น context free

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- product construction: state = คู่ (q, d) stack ใช้ของ PDA อย่างเดียว
- ทำไมทำกับ PDA สองตัวไม่ได้: ต้องใช้ 2 stack

เขียนตอบข้อสอบ (English)

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ and $D = (Q_D, \Sigma, \delta_D, d_0, F_D)$. Construct $P' = (Q \times Q_D, \Sigma, \Gamma, \delta', (q_0, d_0), F \times F_D)$ where

- for $a \in \Sigma$: $((r, \delta_D(d, a)), y) \in \delta'((q, d), a, x)$ whenever $(r, y) \in \delta(q, a, x)$;
- for $a = \varepsilon$: $((r, d), y) \in \delta'((q, d), \varepsilon, x)$ whenever $(r, y) \in \delta(q, \varepsilon, x)$ (the DFA does not move).

P' runs P and D in parallel on the same input, using the stack only for P. A branch of P' ends in $F \times F_D$ iff the corresponding branch of P ends in F and D ends in F_D. So $L(P') = A \cap B$, which is context free. ■

สอนแทรก

- จุดสำคัญที่ต้องเขียน: DFA **ไม่ใช้ stack** จึงรวมกับ PDA ได้ — PDA สองตัวรวมกันไม่ได้เพราะต้องใช้ 2 stack

CFG ⇔ PDA

3.1 CFG → PDA: ไอเดีย

โจทย์ 3.1.1 — stack = leftmost derivation

ให้ $S \to 0S1 \mid \varepsilon$ trace PDA ที่ได้จากการแปลงบน 0011 แล้วเทียบกับ leftmost derivation

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ทุกจังหวะ: **input ที่อ่านแล้ว + stack (ไม่นับ $)** = string ใน leftmost derivation ขั้นนั้น

เฉลยทีละขั้น

ทำ	อ่านแล้ว	stack	อ่านแล้ว + stack
push $S\$$	—	$S\$$	S
ใช้ $S \to 0S1$	—	$0S1\$$	$0S1$
จับ 0	0	$S1\$$	$0S1$
ใช้ $S \to 0S1$	0	$0S11\$$	$00S11$
จับ 0	00	$S11\$$	$00S11$
ใช้ $S \to \varepsilon$	00	$11\$$	0011
จับ 1, จับ 1	0011	$\$$	0011
pop $ → accept	0011	ว่าง	

- คอลัมน์ขวาไล่ได้ $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$ = leftmost derivation ✓

สอนแทรก

- invariant “อ่านแล้ว + stack = string ใน leftmost derivation” คือหัวใจของบทพิสูจน์ Lemma 2.21

3.2 CFG → PDA: สูตร 3 state

โจทย์ 3.2.1 — Convert CFG to PDA

แปลง $S \to aSb \mid SS \mid \varepsilon$ เป็น PDA

1. เขียนป้าย loop ทั้งหมด
2. นับจำนวน loop และจำนวน state ถ้าขยาย push หลายตัวเต็ม
3. ขยายกฎ $S \to aSb$ เป็นเส้นจริง

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. loop ที่ q_{loop}:
   - $\varepsilon, S \to aSb \quad \varepsilon, S \to SS \quad \varepsilon, S \to \varepsilon$
   - $a, a \to \varepsilon \quad b, b \to \varepsilon$

   เส้นหลัก: $q_s \xrightarrow{\varepsilon, \varepsilon \to S\$} q_{loop} \xrightarrow{\varepsilon, \$ \to \varepsilon} q_{acc}$
2. นับ
   - loop $= |R| + |\Sigma| = 3 + 2 =$ **5**
   - state เสริม: aSb ยาว 3 → 2 ตัว SS ยาว 2 → 1 ตัว $\varepsilon \to 0$ push $S\$ \to 1$ ตัว
   - state ทั้งหมด $= 3 + 2 + 1 + 0 + 1 =$ **7**
3. push จาก**ขวาไปซ้าย** ให้ a อยู่บนสุด:

$$q_{loop} \xrightarrow{\varepsilon, S \to b} r_1 \xrightarrow{\varepsilon, \varepsilon \to S} r_2 \xrightarrow{\varepsilon, \varepsilon \to a} q_{loop}$$

! จุดที่คนเสียคะแนนบ่อย

- push a ก่อน → ได้ b อยู่บนสุด → จับ input ผิดลำดับ

สอนแทรก

- ในข้อสอบวาด 3 state หลัก + loop ทุกกฎ/ทุก terminal ก็พอ ถ้าโจทย์ไม่สั่งให้ขยาย push string

โจทย์ 3.2.2 — เขียน δ แบบ formal ของ PDA จาก CFG

แปลง $S \to aSb \mid \varepsilon$ เป็น PDA แบบเต็ม (push ทีละตัว) แล้วเขียน Q, Γ, F และ δ ทุกค่าที่ไม่เป็น $\emptyset$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- push $S\$$ ต้องใช้ state เสริม 1 ตัว (m) กฎ $S \to aSb$ ยาว 3 ต้องใช้ state เสริม 2 ตัว (r_1, r_2)

เฉลยทีละขั้น

- $Q = \{q_s, m, q_{loop}, r_1, r_2, q_{acc}\}$
- $\Sigma = \{a, b\}$, $\Gamma = \{S, a, b, \$\}$, start $= q_s$, $F = \{q_{acc}\}$

δ	ผล	มาจาก
$\delta(q_s, \varepsilon, \varepsilon)$	$\{(m, \$)\}$	push \$
$\delta(m, \varepsilon, \varepsilon)$	$\{(q_{loop}, S)\}$	push S
$\delta(q_{loop}, \varepsilon, S)$	$\{(r_1, b),\ (q_{loop}, \varepsilon)\}$	กฎ $S \to aSb$ (เริ่ม) และ $S \to \varepsilon$
$\delta(r_1, \varepsilon, \varepsilon)$	$\{(r_2, S)\}$	push S
$\delta(r_2, \varepsilon, \varepsilon)$	$\{(q_{loop}, a)\}$	push a (อยู่บนสุด)
$\delta(q_{loop}, a, a)$	$\{(q_{loop}, \varepsilon)\}$	จับ a
$\delta(q_{loop}, b, b)$	$\{(q_{loop}, \varepsilon)\}$	จับ b
$\delta(q_{loop}, \varepsilon, \$)$	$\{(q_{acc}, \varepsilon)\}$	stack ว่าง → accept

- $\delta(q_{loop}, \varepsilon, S)$ มี 2 สมาชิก = PDA **เดา**ว่าจะใช้กฎไหน
- กฎ $S \to aSb$: pop S push $b \to$ push $S \to$ push a (ขวาไปซ้าย)

สอนแทรก

- ค่าที่มีหลายสมาชิกใน δ (เช่น $\delta(q_{loop}, \varepsilon, S)$) = จุดที่ PDA เดา — ชี้ให้เห็นในคำตอบ

โจทย์ 3.2.3 — CFG → PDA + trace (TOC_Practice 3.1)

G: $S \to aSb \mid T$, $T \to cT \mid \varepsilon$ (1) $L(G)$ คืออะไร (2) จำนวน state ของ PDA แบบเต็ม (3) trace acb

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. $L(G) = \{a^n c^m b^n \mid n, m \geq 0\}$
2. loop กฎ 4 เส้น + loop terminal 3 เส้น state เสริม: $S\$$ (1), aSb (2), cT (1) → $3 + 1 + 2 + 1 = 7$ **state**
3. Trace

ขั้น	input เหลือ	stack	ทำ	derivation
0	acb	$S\$$	เริ่ม	S
1	acb	$aSb\$$	$S \to aSb$	$\Rightarrow aSb$
2	cb	$Sb\$$	จับ a	
3	cb	$Tb\$$	$S \to T$	$\Rightarrow aTb$
4	cb	$cTb\$$	$T \to cT$	$\Rightarrow acTb$
5	b	$Tb\$$	จับ c	
6	b	$b\$$	$T \to \varepsilon$	$\Rightarrow acb$
7	—	$\$$	จับ b	
8	—	ว่าง	pop \$ → accept	

สอนแทรก

- ทุกครั้งที่ “ใช้กฎ” ใน trace ให้เขียน derivation ข้าง ๆ — แสดงว่าเข้าใจว่า PDA จำลอง leftmost derivation

โจทย์ 3.2.4 — Sipser Exercise 2.11 — grammar ของนิพจน์

G: $E \to E + T \mid T$, $T \to T \times F \mid F$, $F \to (E) \mid a$ (1) PDA จาก Lemma 2.21 มี loop กี่เส้น state กี่ตัว (2) แสดงการคำนวณที่ accept $a + a$

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

ข้อ 1

- loop จากกฎ 6 เส้น + จาก terminal 5 เส้น $(a, +, \times, (,))$ → **11 เส้น**
- state เสริม: $E\$$ (1), $E + T$ (2), $T \times F$ (2), (E) (2) → $3 + 7 = 10$ **state**

ข้อ 2

ขั้น	input เหลือ	stack	ทำ
0	$a + a$	$E\$$	เริ่ม
1	$a + a$	$E + T\$$	$E \to E + T$
2	$a + a$	$T + T\$$	$E \to T$
3	$a + a$	$F + T\$$	$T \to F$
4	$a + a$	$a + T\$$	$F \to a$
5–6	a	$T\$$	จับ a, จับ $+$
7–8	a	$a\$$	$T \to F$, $F \to a$
9	—	$\$$	จับ a
10	—	ว่าง	pop $ → accept

- ขั้น 1 ต้อง**เดา** $E \to E + T$ (ไม่ใช่ $E \to T$) — เดาผิดกิ่งนั้นตายเอง
- ตรงกับ leftmost derivation $E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \Rightarrow a + F \Rightarrow a + a$

สอนแทรก

- สูตรนับ: loop $= |R| + |\Sigma|$ state $= 3 + \sum(|w| - 1) + 1$ (ตัว $+1$ มาจาก push $E\$$)

โจทย์ 3.2.5 — Sipser Exercise 2.3(o), 2.12 — grammar ของ non-palindrome

G: $R \to XRX \mid S$, $S \to aTb \mid bTa$, $T \to XTX \mid X \mid \varepsilon$, $X \to a \mid b$ (1) $L(G)$ คืออะไร (2) นับ loop และ state ของ PDA (3) trace ab

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. $L(G)$
   - X = ตัวเดียว T = ทุก string
   - S = string ที่ตัวแรกกับตัวท้าย**ต่างกัน**
   - R ห่อ S ด้วยจำนวนเท่ากันสองข้าง → มีคู่สมมาตรที่ต่างกัน
   - → $\{w \in \{a,b\}^* \mid w \neq w^{\mathcal{R}}\}$ (ไม่เป็น palindrome)
2. loop จาก กฎ 9 เส้น + terminal 2 เส้น = **11 เส้น** state เสริม $R\$$ (1), XRX (2), aTb (2), bTa (2), XTX (2) → **12 state**
3. Trace ab: $R\$ \to (R \to S)\ S\$ \to (S \to aTb)\ aTb\$ \to$ จับ a: $Tb\$ \to (T \to \varepsilon)\ b\$ \to$ จับ b: $\$ \to$ pop → accept

สอนแทรก

- อ่าน grammar จาก**ล่างขึ้นบน**: $X \to T \to S \to R$ แล้วค่อยสรุปภาษา

3.3 PDA → CFG: เตรียม PDA (3 เงื่อนไข)

โจทย์ 3.3.1 — ปรับ PDA ก่อนแปลง

PDA ของ $\{0^n1^n \mid n \geq 0\}$ (Example 2.14) มี accept state 2 ตัว (q_1, q_4) ปรับให้ผ่าน 3 เงื่อนไขของ Lemma 2.27

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. **accept เดียว:** เพิ่ม q_{acc} ลากเส้นจาก q_1 และ q_4 แล้วให้ q_1, q_4 ไม่ใช่ accept
2. **stack ว่างก่อน accept:** ที่ q_1 (ยังไม่ push) และ q_4 (pop \$ แล้ว) stack ว่างอยู่แล้ว ✓
3. **push หรือ pop อย่างเดียว:** เส้นใหม่ในข้อ 1 ไม่แตะ stack → แตกเป็น push ตัวหลอกแล้ว pop
   - $q_1 \xrightarrow{\varepsilon,\varepsilon\to Z} t_1 \xrightarrow{\varepsilon,Z\to\varepsilon} q_{acc}$
   - $q_4 \xrightarrow{\varepsilon,\varepsilon\to Z} t_4 \xrightarrow{\varepsilon,Z\to\varepsilon} q_{acc}$
4. เส้นเดิมผ่านแล้ว: $\varepsilon, \varepsilon \to \$$ (push) $0, \varepsilon \to 0$ (push) $1, 0 \to \varepsilon$ (pop) $\varepsilon, \$ \to \varepsilon$ (pop)

สอนแทรก

- เส้นที่ไม่แตะ stack แก้ด้วย push ตัวหลอกแล้ว pop ทันที — ต้องใช้ state ใหม่ 1 ตัวต่อเส้น

โจทย์ 3.3.2 — ปรับ PDA ของ $ww^{\mathcal{R}}$ แล้วแปลง (TOC_Practice 4.2)

PDA ของ $ww^{\mathcal{R}}$ (Example 2.18) มีเส้น $q_2 \xrightarrow{\varepsilon,\varepsilon\to\varepsilon} q_3$ (1) เงื่อนไขไหนไม่ผ่าน (2) แก้ (3) เขียนกฎแบบ 1 แล้วลดรูป

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. accept เดียว ✓ stack ว่าง ก่อน accept ✓ **push หรือ pop อย่างเดียว** × (เส้น เดา กึ่งกลาง ไม่ แตะ stack)
2. แตกเป็น push แล้ว pop ตัวหลอก #: $q_2 \xrightarrow{\varepsilon,\varepsilon\to\#} q_5 \xrightarrow{\varepsilon,\#\to\varepsilon} q_3$
3. กฎแบบ 1

สัญลักษณ์	push	pop	กฎ
\$	$q_1 \to q_2$	$q_3 \to q_4$	$A_{14} \to A_{23}$
0	$q_2 \xrightarrow{0} q_2$	$q_3 \xrightarrow{0} q_3$	$A_{23} \to 0\,A_{23}\,0$
1	$q_2 \xrightarrow{1} q_2$	$q_3 \xrightarrow{1} q_3$	$A_{23} \to 1\,A_{23}\,1$
#	$q_2 \to q_5$	$q_5 \to q_3$	$A_{23} \to A_{55}$

- $A_{55} \to \varepsilon \to A_{23} \to \varepsilon$ แบบ 2 ($5^3 = 125$ กฎ) ไม่สร้างอะไรใหม่ → ตัด
- ตั้ง $S = A_{23}$: $S \to 0S0 \mid 1S1 \mid \varepsilon$ = grammar มาตรฐานของ $ww^{\mathcal{R}}$ ✓

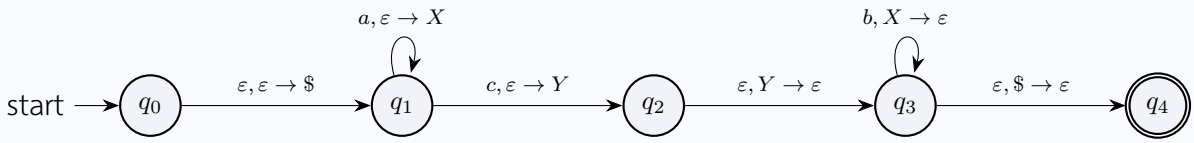
สอนแทรก

- เส้นเดากึ่งกลาง $\varepsilon, \varepsilon \to \varepsilon$ คือเส้นที่ไม่ผ่านเงื่อนไข 3 บ่อยที่สุด — เช็กเส้นนี้ก่อนเสมอ

3.4 PDA → CFG: ตัวแปร A_{pq} และกฎ 3 แบบ

โจทย์ 3.4.1 — Convert PDA to CFG

แปลง PDA นี้เป็น CFG แล้วลดรูป



ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

ขั้น 1 เช็ก 3 เงื่อนไข: accept เดียว (q_4) ✓ pop \$ ก่อนเข้า ✓ ทุกเส้นทำอย่างเดียว ✓

ขั้น 2 แยกกอง:

	push					pop			
ชื่อ	จาก	ไป	อ่าน	ใส่	ชื่อ	จาก	ไป	อ่าน	เอาออก
P1	q_0	q_1	ε	\$	Q1	q_2	q_3	ε	Y
P2	q_1	q_1	a	X	Q2	q_3	q_3	b	X
P3	q_1	q_2	c	Y	Q3	q_3	q_4	ε	\$

ขั้น 3 จับคู่สัญลักษณ์เดียวกัน (กฎแบบ 1 $A_{pq} \to aA_{rs}b$):

คู่	p	r	s	q	a, b	กฎ
P1+Q3 (\$)	q_0	q_1	q_3	q_4	ε, ε	$A_{04} \to A_{13}$
P2+Q2 (X)	q_1	q_1	q_3	q_3	a, b	$A_{13} \to a\, A_{13}\, b$
P3+Q1 (Y)	q_1	q_2	q_2	q_3	c, ε	$A_{13} \to c\, A_{22}$

ขั้น 4: แบบ 3 $A_{pp} \to \varepsilon$ (เช่น $A_{22} \to \varepsilon$) แบบ 2 $A_{pq} \to A_{pr}A_{rq}$ ทุก p, q, r (ข้อนี้ไม่มีตัวที่ใช้จริง)

ขั้น 5: start A_{04} ตั้ง $S = A_{04}$, $T = A_{13}$

$$\boxed{S \to T, \qquad T \to aTb \mid c}$$

- ภาษา $= \{a^n c b^n\}$ ตรงกับ PDA ✓ ตรวจ acb: $S \Rightarrow T \Rightarrow aTb \Rightarrow acb$
- นับ: $|Q| = 5 \to$ ตัวแปร 25, แบบ 2 125 กฎ, แบบ 3 5 กฎ

! จุดที่คนเสียคะแนนบ่อย

- P3+Q1: ไส้คือ $A_{rs} = A_{q_2 q_2}$ (ปลาย push = q_2, ต้น pop = q_2) — ไม่ใช่ $A_{q_1 q_3}$
- จับ P2 (push X) กับ Q3 (pop \$) → ผิด สัญลักษณ์ต้องตรงกัน

สอนแทรก

- เขียนตาราง push/pop แยกกองก่อนเสมอ — ลดโอกาสจับคู่ผิดและลืมคู่

โจทย์ 3.4.2 — Proof: ทุกภาษา regular เป็น CFL (Corollary 2.32 แบบสร้าง grammar ตรง)

ให้ DFA $D = (Q, \Sigma, \delta, q_0, F)$ สร้าง CFG ที่ $L(G) = L(D)$ และอธิบายว่าทำไมถูก

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- 1 ตัวแปรต่อ 1 state: R_i = string ที่พาจาก q_i ไปถึง accept state

เขียนตอบข้อสอบ (English)

Let G have a variable R_i for each state q_i, start variable R_0, and rules

- $R_i \to aR_j$ whenever $\delta(q_i, a) = q_j$;
- $R_i \to \varepsilon$ whenever $q_i \in F$.

Claim: $R_i \overset{*}{\Rightarrow} w$ iff D started in q_i on w ends in an accept state.

- By induction on $|w|$: for $w = \varepsilon$ only $R_i \to \varepsilon$ applies, which exists iff $q_i \in F$.
- For $w = aw'$ the first rule must be $R_i \to aR_j$ with $\delta(q_i, a) = q_j$, and then $R_j \overset{*}{\Rightarrow} w'$ iff D from q_j on w' accepts.

With $i = 0$, $L(G) = L(D)$. Hence every regular language is context free. ■

สอนแทรก

- grammar แบบนี้คือ right-linear grammar (Type 3 ใน Chomsky hierarchy)
- อีกวิธีที่สั้นกว่า: DFA เป็น PDA ที่ไม่ใช้ stack + Thm 2.20

โจทย์ 3.4.3 — PDA → CFG: $\{0^n 1^n \mid n \geq 1\}$ (TOC_Practice 4.1)

PDA: $p \xrightarrow{\varepsilon,\varepsilon\to\$} q$, วน q: $0,\varepsilon \to 0$, $q \xrightarrow{1,0\to\varepsilon} r$, วน r: $1,0 \to \varepsilon$, $r \xrightarrow{\varepsilon,\$\to\varepsilon} s$ (accept s) แปลงเป็น CFG แล้วลดรูป

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

ขั้น 1 เช็ก 3 เงื่อนไข

- accept state เดียว (s) ✓ pop \$ ก่อนเข้า s → stack ว่าง ✓
- ทุกเส้นทำอย่างเดียว: $\varepsilon,\varepsilon \to \$$ (push), $0,\varepsilon \to 0$ (push), $1,0 \to \varepsilon$ (pop) ×2, $\varepsilon,\$ \to \varepsilon$ (pop) ✓

ขั้น 2 แยกกอง

- push: P1 $p \to q$ (\$, อ่าน ε) P2 $q \to q$ (0, อ่าน 0)
- pop: Q1 $q \to r$ (0, อ่าน 1) Q2 $r \to r$ (0, อ่าน 1) Q3 $r \to s$ (\$, อ่าน ε)

ขั้น 3 กฎแบบ 1 $A_{\text{ต้นpush,ปลายpop}} \to a\, A_{\text{ปลายpush,ต้นpop}}\, b$

คู่	นอก	ใน	กฎ
P1+Q3 (\$)	p, s	q, r	$A_{ps} \to A_{qr}$
P2+Q1 (0)	q, r	q, q	$A_{qr} \to 0\, A_{qq}\, 1$
P2+Q2 (0)	q, r	q, r	$A_{qr} \to 0\, A_{qr}\, 1$

ขั้น 4 แบบ 2: $A_{xy} \to A_{xz}A_{zy}$ ($4^3 = 64$ กฎ) แบบ 3: $A_{xx} \to \varepsilon$ (4 กฎ) start A_{ps}

ขั้น 5 ลดรูป

- $A_{qq} \to \varepsilon \to A_{qr} \to 01$
- แบบ 2 ที่ใช้ได้เช่น $A_{qr} \to A_{qq}A_{qr}$ = $A_{qr} \to A_{qr}$ ไม่เพิ่มอะไร → ตัด
- $A_{ps} \to A_{qr}$ เป็น unit → ตั้ง $S = A_{qr}$

$$\boxed{S \to 0S1 \mid 01}$$ ตรวจ 0011 : $S \Rightarrow 0S1 \Rightarrow 0011$ ✓

สอนแทรก

- ทำไมได้ 01 ไม่ใช่ ε: ภาษาคือ $n \geq 1$ — กฎแบบ 1 ตัวแรกบังคับให้อ่าน 0 แล้วตามด้วย 1 อย่างน้อยคู่หนึ่ง
- ในข้อสอบเขียนกฎแบบ 2–3 เป็นสูตรพร้อมจำนวนก็พอ ไม่ต้องไล่ครบ 64 กฎ

ภาษาที่ไม่ใช่ CFL: Pumping Lemma

4.1 ตัวบท Pumping Lemma (CFL)

โจทย์ 4.1.1 — การแบ่งไหนถูกเงื่อนไข

ให้ $p = 4$ และ $s = a^4b^4c^4$ การแบ่ง $s = uvxyz$ ใดผ่านเงื่อนไข 2 และ 3 ของ pumping lemma

1. $v = \varepsilon,\ x = a,\ y = \varepsilon$
2. $v = \varepsilon,\ x = b,\ y = bb$
3. $v = aa,\ x = bbbb,\ y = cc$
4. $v = ab,\ x = \varepsilon,\ y = b$

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

เงื่อนไข 2: $|vy| > 0$ เงื่อนไข 3: $|vxy| \le p = 4$ ($s = aaaa\,bbbb\,cccc$)

1. $v = \varepsilon, x = a, y = \varepsilon$: $|vy| = 0 \to$ **ผิดเงื่อนไข 2** (ไม่มีอะไรให้ปั๊ม)
2. $v = \varepsilon, x = b, y = bb$: $|vy| = 2 > 0$ ✓, $|vxy| = 3 \le 4$ ✓ $\to$ **ผ่าน** (v ว่างได้ ขอแค่ v, y ไม่ว่างพร้อมกัน)
3. $v = aa, x = bbbb, y = cc$: $|vxy| = 8 > 4 \to$ **ผิดเงื่อนไข 3** (หน้าต่างกว้างเกิน p)
4. $v = ab, x = \varepsilon, y = b$: $|vy| = 3$ ✓, $|vxy| = 3$ ✓ $\to$ **ผ่าน** — $u = aaa$, $v = ab$ คร่อมรอยต่อ $a|b$

ข้อ 4 ปั๊มขึ้น: $uv^2xy^2z = aaa\,abab\,bb\,bbcccc$ มี b ก่อน $a \to$ ผิดลำดับ

สอนแทรก

- เงื่อนไข 2–3 คือ “กติกา” ที่ฝ่ายตรงข้ามต้องทำตามตอนแบ่ง — เราต้องรับมือ**ทุก**การแบ่งที่ผ่านกติกา
- ข้อ 4 แสดงว่าทำไมบทพิสูจน์ต้องมีกรณี “v หรือ y มีหลายชนิด” เสมอ

โจทย์ 4.1.2 — ทฤษฎี: ใช้ pumping lemma พิสูจน์ว่าเป็น CFL ได้ไหม

นักศึกษาเขียนว่า “ภาษา L ผ่านเงื่อนไขของ pumping lemma ทุกข้อ ดังนั้น L เป็น CFL” ถูกหรือผิด อธิบาย

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

คำตอบ: ผิด

1. pumping lemma มีรูป $P \Rightarrow Q$: “ถ้า L เป็น CFL (P) แล้วปั๊มได้ (Q)”
2. นักศึกษาใช้ $Q \Rightarrow P$ (converse) — ไม่ได้ตามมาจาก $P \Rightarrow Q$
3. มีภาษาที่**ไม่ใช่ CFL แต่ปั๊มได้** (lemma ไม่ได้บอกว่าเงื่อนไขนี้พอ)
4. ทางเดียวที่ใช้ได้คือ contrapositive $\neg Q \Rightarrow \neg P$: “ปั๊มไม่ได้ $\Rightarrow$ ไม่ใช่ CFL”
5. จะพิสูจน์ว่า**เป็น** CFL ต้อง**สร้าง** CFG หรือ PDA ให้ดู

เขียนตอบข้อสอบ (English)

The pumping lemma states that every CFL satisfies the pumping property; it does not state the converse. Some non-context-free languages also satisfy the pumping conditions, so satisfying them does not prove that L is context free. To show L is context free we must give a CFG or a PDA for L.

สอนแทรก

- ตรรกะ: $P \Rightarrow Q$ ใช้ได้แค่ contrapositive $\neg Q \Rightarrow \neg P$ — ห้ามใช้ converse $Q \Rightarrow P$

4.2 ทำไมจริง (ไอเดียพิสูจน์)

โจทย์ 4.2.1 — คำนวณ p

grammar มีตัวแปร $|V| = 3$ และด้านขวายาวสุด $b = 2$ คำนวณ pumping length ตามบทพิสูจน์ และอธิบายว่าทำไม string ยาว $\geq p$ ต้องมีตัวแปรซ้ำบน path

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **สูตร:** $p = b^{|V|+1} = 2^{3+1} =$ **16**
2. **ความยาวกับความสูง:** node หนึ่งมีลูกไม่เกิน $b = 2 \rightarrow$ tree สูง h มีใบไม่เกิน 2^h
   - ถ้า $|s| \geq 16 = 2^4 \rightarrow$ tree (ต้นที่ node น้อยสุด) ต้องสูงอย่างน้อย $4 = |V| + 1$
3. **pigeonhole:** path ยาวสุดจากรากถึงใบมีตัวแปรอย่างน้อย 4 ตัว แต่ grammar มีตัวแปรต่างกันแค่ 3 $\rightarrow$ ต้องมีตัวแปร R **ซ้ำ**บน path
4. **ปั๊ม:** R ตัวบนสร้าง vxy, R ตัวล่างสร้าง x
   - เอา subtree ของ R บนไปแทน R ล่าง $\rightarrow uv^2xy^2z$ (ทำซ้ำได้ i ครั้ง)
   - เอา subtree ของ R ล่างไปแทน R บน $\rightarrow uxz$ $(i = 0)$

สอนแทรก

- ทำไม $|vy| > 0$: เลือก tree ที่ node น้อยสุด — ถ้า v, y ว่างทั้งคู่จะตัดส่วนนั้นทิ้งได้ tree เล็กกว่า ขัดแย้ง
- ทำไม $|vxy| \leq p$: เลือก R จาก $|V| + 1$ ตัวล่างสุดของ path $\rightarrow$ subtree ของ R บนสูงไม่เกิน $|V| + 1$
- ในข้อสอบไม่ต้องคำนวณ p จริง — ใช้ p เป็นตัวแปรเสมอ

4.3 วิธีใช้: เกม 4 ตา

โจทย์ 4.3.1 — หาจุดผิดในบทพิสูจน์

นักศึกษาเขียนว่า “ให้ $p = 3$, $s = a^3b^3c^3$, เลือก $v = a$, $x = \varepsilon$, $y = b$ แล้ว $uv^2xy^2z = a^4b^4c^3 \notin L$ จึงไม่ใช่ CFL” ผิดตรงไหนบ้าง

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **กำหนด $p = 3$** — ผิด: p มาจาก lemma (ฝ่ายตรงข้าม) เราไม่รู้ค่า ต้องเขียนเป็นตัวแปรเสมอ
2. **เลือก $v = a, x = \varepsilon, y = b$ เอง** — ผิด: การแบ่งเป็นตาของฝ่ายตรงข้าม เราต้องพิสูจน์ว่า**ทุก**การแบ่งที่ $|vy| > 0$, $|vxy| \le p$ ล้มเหลว
3. **ไม่มีการแยกกรณี** — ผิด: ต้องครอบอย่างน้อย
   - (ก) v, y แต่ละตัวชนิดเดียว → จำนวนไม่เท่ากัน
   - (ข) v หรือ y ปนชนิด → ลำดับผิด
4. สิ่งที่**เราเลือกได้**มีแค่ **s (ยาว $\ge p$) และ i**

เขียนตอบข้อสอบ (English) — บทพิสูจน์ที่ถูก

Assume $L = \{a^n b^n c^n\}$ is context free with pumping length p. Let $s = a^p b^p c^p$. For any split $s = uvxyz$ with $|vy| > 0$, $|vxy| \le p$: (1) if v and y each contain one symbol type, uv^2xy^2z has unequal counts; (2) otherwise uv^2xy^2z has symbols out of order. Either way $uv^2xy^2z \notin L$, a contradiction.

สอนแทรก

- จำบทบาทในเกม: ศัตรูเลือก p และการแบ่ง เราเลือก s และ i — สลับบทบาท = ผิดทั้งข้อ

4.4 ตัวอย่างหลัก: $\{a^n b^n c^n\}$ และ $\{ww\}$

โจทย์ 4.4.1 — Prove not CF

พิสูจน์ว่า $L = \{0^n 1^n 0^n \mid n \geq 0\}$ ไม่ใช่ context free

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- 3 บล็อกยาว $p \to vxy$ แตะได้มากสุด 2 บล็อกติดกัน $\to$ อย่างน้อย 1 บล็อกไม่ถูกปั๊ม

เฉลยทีละขั้น

1. เลือก $s = 0^p 1^p 0^p \in L$, $|s| = 3p \geq p$
2. กรณี 1: v หรือ y มีทั้ง 0 และ 1 $\to uv^2xy^2z$ มีรูป $0 \ldots 1 \ldots 0 \ldots 1 \ldots$ ผิดรูปแบบ
3. กรณี 2: v, y แต่ละตัวชนิดเดียว
   - $|vxy| \leq p \to$ แตะบล็อก 0 แรกกับบล็อก 0 สุดท้ายพร้อมกันไม่ได้
   - ปั๊มขึ้น: บล็อกที่ถูกแตะยาวขึ้น บล็อกที่ไม่ถูกแตะยาว p เท่าเดิม $\to$ 3 บล็อกไม่เท่ากัน
4. ทุกกรณี $uv^2xy^2z \notin L \to$ ขัดแย้ง

เขียนตอบข้อสอบ (English)

Setup. Assume L is context free with pumping length p. Let $s = 0^p 1^p 0^p \in L$, so $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$.

- Case 1: v or y contains both 0s and 1s.
  - Then uv^2xy^2z has a 1 before a 0 before a 1, so it is not of the form $0^n 1^n 0^n$.
- Case 2: v and y each contain one type of symbol.
  - Since $|vxy| \leq p$, vxy cannot touch both the first and the last block of 0s.
  - So uv^2xy^2z lengthens at least one block but leaves another block of length p; the blocks are unequal.

Conclusion. In every case $uv^2xy^2z \notin L$, a contradiction. Therefore L is not context free. ■

สอนแทรก

- โครงเหมือน $a^n b^n c^n$ ทุกอย่าง — 2 กรณี (ปนชนิด / ชนิดเดียว) + เหตุผล $|vxy| \leq p$

โจทย์ 4.4.2 — Sipser Exercise 2.30(c) — w เป็น substring ของ t

พิสูจน์ว่า $C = \{w\#t \mid w \text{ เป็น substring ของ } t,\ w, t \in \{a, b\}^*\}$ ไม่ใช่ CFL

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- $s = a^p b^p \# a^p b^p$ กรณีที่ต้องระวัง: v อยู่ซ้าย # และ y อยู่ขวา #
- เลือกทิศปั๊มให้ถูก: ทำให้ฝั่งซ้าย**ยาวขึ้น** หรือฝั่งขวา**สั้นลง**

เขียนตอบข้อสอบ (English)

Setup. Assume C is context free with pumping length p. Let $s = a^p b^p \# a^p b^p \in C$, so $s = uvxyz$ with $|vy| > 0$, $|vxy| \leq p$.

- **Case 1:** v or y contains #. Then uv^2xy^2z has two #'s, so it is not in C.
- **Case 2:** vxy is left of #. Pump up: the left side becomes longer than the right side, so it cannot be a substring.
- **Case 3:** vxy is right of #. Pump down: the right side becomes shorter than the left side.
- **Case 4:** # is in x. Since $|vxy| \leq p$, v lies in the left b^p and y in the right a^p.
  - If $v \neq \varepsilon$: pump up; the left side has more than p b's but the right side has only p.
  - If $v = \varepsilon$ (so $y \neq \varepsilon$): pump down; the right side has fewer than p a's, so $a^p b^p$ is not its substring.

Conclusion. Every case gives a contradiction, so C is not context free. ■

สอนแทรก

- ภาษาที่มีตัวคั่น: กรณี v ซ้าย # และ y ขวา # คือกรณียากที่สุด — ต้องเลือกทิศปั๊มแยกตามว่า v ว่างหรือไม่

4.5 ภาษาที่ต้องเลือก i ตามกรณี

โจทย์ 4.5.1 — Prove not CF (Practice 5.1)

พิสูจน์ว่า $L = \{a^i b^j c^k \mid i < j < k\}$ ไม่ใช่ context free

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- เลือก $s = a^p b^{p+1} c^{p+2}$ (ห่างกันทีละ 1 → ขยับนิดเดียวก็ผิด)
- $|vxy| \le p$ → มี a กับ c พร้อมกันไม่ได้

เฉลยทีละขั้น

1. v หรือ y ปนชนิด → ปั๊มแล้วลำดับผิด
2. v, y **ไม่มี** a → ปั๊มลง $i = 0$
   - ถ้า b ลด: $b \le p = a$ → ผิด $a < b$
   - ถ้ามีแค่ c ลด: $c \le p + 1 = b$ → ผิด $b < c$
3. v, y **มี** a (จึงไม่มี c) → ปั๊มขึ้น $i = 2$
   - ถ้า b ไม่เพิ่ม: $a \ge p + 1 = b$ → ผิด $a < b$
   - ถ้า b เพิ่มด้วย: $b \ge p + 2 = c$ → ผิด $b < c$
4. ครอบทุกกรณี → ขัดแย้ง → **ไม่ใช่ CFL**

สอนแทรก

- ภาษาอสมการ ($<$, $\le$) ต้องเลือก i ต่างกันตามกรณี — เขียนตารางกรณีก่อนลงมือ

โจทย์ 4.5.2 — Proof: $\{a^{n^2} \mid n \geq 0\}$ ไม่ใช่ CFL

เขียนบทพิสูจน์เต็มว่า $L = \{a^{n^2} \mid n \geq 0\}$ ไม่ใช่ context free

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ตัวอักษรชนิดเดียว → ไม่มีกรณี “ลำดับผิด” ใช้**ความยาว**แทน
- ปั๊มขึ้นแล้วความยาวตกอยู่ระหว่างกำลังสองที่ติดกัน

เขียนตอบข้อสอบ (English)

Setup. Assume L is context free with pumping length p. Let $s = a^{p^2} \in L$, so $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$.
Pump up. uv^2xy^2z has length $p^2 + |vy|$, where $1 \leq |vy| \leq p$. So

$$p^2 < p^2 + |vy| \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2.$$

The length lies strictly between two consecutive perfect squares, so it is not a perfect square and $uv^2xy^2z \notin L$.
Conclusion. This contradicts the pumping lemma. Therefore L is not context free. ■

สอนแทรก

- ภาษาตัวอักษรเดียว: ไม่มีกรณีลำดับ ใช้ความยาวล้วน ใช้ $1 \leq |vy| \leq p$ บีบความยาวให้อยู่ระหว่างกำลังสองที่ติดกัน

โจทย์ 4.5.3 — $\{a^{2^n} \mid n \geq 0\}$ ไม่ใช่ CFL (TOC_Practice 5.2)

พิสูจน์ว่า $B = \{a^{2^n} \mid n \geq 0\}$ ไม่ใช่ context free

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ตัวอักษรชนิดเดียว → ดูแค่ความยาว $m = |vy|$ ช่องว่างระหว่างกำลังของ 2 กว้างกว่า p

เขียนตอบข้อสอบ (English)

Setup. Assume B is context free with pumping length p. Let $s = a^{2^p}$, so $s = uvxyz$ with $m = |vy|$, $1 \leq m \leq p$.
Pump up ($i = 2$). $|uv^2xy^2z| = 2^p + m$ and

$$2^p < 2^p + m \leq 2^p + p < 2^p + 2^p = 2^{p+1}$$

(using $p < 2^p$). The length is strictly between two consecutive powers of 2, so $uv^2xy^2z \notin B$.
Conclusion. Contradiction; B is not context free. ■

- B decidable: TM M_2 ของ Example 3.7

สอนแทรก

- ช่องว่างระหว่าง 2^p กับ 2^{p+1} คือ $2^p > p$ — ปั๊มขึ้นเพิ่มได้ไม่เกิน p จึงตกในช่องว่างเสมอ

โจทย์ 4.5.4 — Sipser Problem 2.31 — palindrome ที่ 0 กับ 1 เท่ากัน

ให้ B = palindrome บน $\{0,1\}$ ที่มี 0 และ 1 จำนวนเท่ากัน พิสูจน์ว่า B ไม่ใช่ CFL

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- เลือก $s = 0^p 1^{2p} 0^p$ ช่วง 1^{2p} ยาวเกิน $p \to vxy$ แตะ 0 ทั้งสองช่วงพร้อมกันไม่ได้
- ใช้ $i = 0$ (ปั๊มลง) — ช่วง 0 สั้นลงเสมอ ไม่ต้องห่วงกรณี v คร่อมรอยต่อ

เขียนตอบข้อสอบ (English)

Setup. Assume B is context free with pumping length p. Let $s = 0^p 1^{2p} 0^p \in B$ (a palindrome with $2p$ 0s and $2p$ 1s), so $s = uvxyz$ with $|vy| > 0$, $|vxy| \leq p$. Since the middle block 1^{2p} is longer than p, vxy cannot touch both blocks of 0s. Pump down to uxz.

- Case 1: vy contains only 1s. Then uxz has fewer 1s than 0s.
- Case 2: vy contains some 0s, all from one block (say the first). Then uxz starts with $p - k$ 0s ($k \geq 1$) followed by a 1, but ends with p 0s, so it is not a palindrome.

Conclusion. In every case $uxz \notin B$, a contradiction. ■

สอนแทรก

- ใช้ $i = 0$ (ปั๊มลง) เลี่ยงกรณี v คร่อมรอยต่อ — ทริกที่ใช้ได้หลายข้อ

4.6 วิธีที่สอง: Closure กับ regular

โจทย์ 4.6.1 — Sipser Problem 2.18 — Closure method

พิสูจน์ว่า $L = \{w \in \{0,1,2\}^* \mid w \text{ มี } 0,\ 1,\ 2 \text{ จำนวนเท่ากัน}\}$ ไม่ใช่ context free

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **สมมติ** L เป็น CFL (พิสูจน์แบบขัดแย้ง)
2. $R = 0^*1^*2^*$ เป็น **regular** เพราะเขียนเป็น regex ได้
3. **ทฤษฎีบท:** CFL ∩ regular เป็น CFL เสมอ → $L \cap R$ ต้องเป็น CFL
4. **คำนวณ** $L \cap R$**:**
   - อยู่ใน R → รูป $0^i1^j2^k$ (เรียงลำดับ)
   - อยู่ใน L → $i = j = k$
   - ดังนั้น $L \cap R = \{0^n1^n2^n\}$
5. $\{0^n1^n2^n\}$ **ไม่ใช่ CFL** (Example 2.36) → ขัดแย้ง → L ไม่ใช่ CFL

สอนแทรก

- ทำไมไม่ปั๊มตรง ๆ: L ไม่บังคับลำดับ → ศัตรูแบ่งแล้วปั๊มได้ string ที่ยังจำนวนเท่ากันง่าย (เช่น $v = 012$) → ปั๊มตรงยาก
- เคล็ดลับ: เลือก regular R ที่ "บังคับลำดับ" ให้เหลือภาษาที่รู้แล้วว่าไม่ใช่ CFL

! จุดที่คนเสียคะแนนบ่อย

- ใช้ "CFL ∩ CFL" แทน "CFL ∩ regular" → ผิด เพราะ CFL ไม่ปิด ∩

4.7 ตารางเลือก s

โจทย์ 4.7.1 — เลือก s

เลือก s ที่เหมาะสำหรับพิสูจน์ว่าภาษาต่อไปนี้ไม่ใช่ CFL

1. $\{a^n b^{2n} c^{3n}\}$
2. $\{a^n b^m a^n b^m\}$
3. $\{ww^{\mathcal{R}}w \mid w \in \{0,1\}^*\}$
4. $\{0^n \# 0^{2n} \# 0^{3n}\}$
5. $\{a^{n^2}\}$
6. $\{a^k \mid k \text{ prime}\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

หลักเลือก: (1) $s \in L$ (2) $|s| \geq p$ (3) ทุกบล็อกยาว $\geq p \to vxy$ แตะได้ไม่เกิน 2 บล็อกติดกัน

1. $a^p b^{2p} c^{3p}$ — ปั๊มแล้วแตะได้ไม่เกิน 2 ใน 3 บล็อก $\to$ อัตราส่วน $1:2:3$ เสีย
2. $a^p b^p a^p b^p$ — vxy แตะบล็อก a แรกกับ a สุดท้ายพร้อมกันไม่ได้ (ห่างกัน $2p$)
3. $w = 0^p 1^p \to s = 0^p 1^p\, 1^p 0^p\, 0^p 1^p$ — ถ้าใช้ $w = 0^p 1$ ศัตรูแบ่งให้ปั๊มแล้วยังเป็นรูปเดิมได้
4. $0^p \# 0^{2p} \# 0^{3p}$ — ต้องมีกรณี "v หรือ y มี #" (ปั๊มแล้ว # เกิน 2 ตัว)
5. a^{p^2} — ปั๊มขึ้นยาว $p^2 + |vy| < (p+1)^2 \to$ ไม่ใช่กำลังสอง
6. a^q ($q \geq p$ prime) — ปั๊ม $i = q+1$: ยาว $q + q|vy| = q(1+|vy|)$ เป็นผลคูณ 2 จำนวนที่ $> 1 \to$ ไม่ prime

สอนแทรก

- ภาษาตัวอักษรเดียว (ข้อ 5–6) ไม่มีเรื่องลำดับ $\to$ ใช้**ความยาว**ล้วน
- ภาษาที่มีตัวคั่น (ข้อ 4) $\to$ กรณีที่ v/y มีตัวคั่นต้องเขียนแยกเสมอ

โจทย์ 4.7.2 — เลือก s ผิดแล้วเกิดอะไร

ถ้าพิสูจน์ว่า $\{ww\}$ ไม่ใช่ CFL โดยเลือก $s = 0^p 1\, 0^p 1$ จะติดตรงไหน ยกการแบ่งที่ปั๊มแล้วยังอยู่ในภาษา

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. $s = 0^p 1\, 0^p 1$ อยู่ในภาษา ($w = 0^p 1$) และยาว $\geq p$ ✓ — แต่ศัตรูมีการแบ่งที่ชนะเรา
2. ศัตรูเลือก $u = 0^{p-1}$, $v = 0$, $x = 1$, $y = 0$, $z = 0^{p-1}1$
   - $|vy| = 2 > 0$ ✓ $|vxy| = 3 \leq p$ ✓ (ถือว่า $p \geq 3$)
3. ปั๊ม i ใด ๆ: $uv^i xy^i z = 0^{p-1+i}\, 1\, 0^{p-1+i}\, 1 = w'w'$ โดย $w' = 0^{p-1+i}1$ → **อยู่ในภาษาทุก i**
4. ไม่มีข้อขัดแย้ง → บทพิสูจน์ล้มเหลว (**ไม่ได้**แปลว่าภาษาเป็น CFL)
5. แก้: เลือก $s = 0^p 1^p 0^p 1^p$ — บล็อก 1 ยาว p ทำให้ vxy ที่คร่อมกลางถูกบีบอยู่ใน $1^p 0^p$ แล้วปั๊มลงทำลายรูป ww

สอนแทรก

- ก่อนเขียนบทพิสูจน์ ลอง “เล่นเป็นศัตรู” 30 วินาที: หาการแบ่งที่ปั๊มแล้วยังอยู่ในภาษา — เจอ = ต้องเปลี่ยน s

บทที่ 5

Turing Machines

5.1 TM ต่างจาก DFA/PDA

โจทย์ 5.1.1 — จริงหรือเท็จ

1. TM ต้องอ่าน input ให้หมดก่อนจึง accept ได้
2. TM อาจทำงานไม่หยุดบน input บางตัว
3. DFA และ PDA (ตามนิยามใน Sipser) อ่าน input ทีละตัวจากซ้ายไปขวาเท่านั้น
4. TM เขียนทับ input ได้

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **เท็จ** — TM หยุด**ทันที**ที่เข้า q_{accept} หรือ q_{reject} ไม่ต้องอ่าน input ให้หมด (ต่างจาก DFA/PDA ที่ตัดสินตอน input หมด) เช่น TM ของ “ขึ้นต้นด้วย 0” อ่านช่องเดียวแล้ว accept
2. **จริง** — หัวเดินซ้าย-ขวาได้เรื่อย ๆ จึงอาจไม่เข้า $q_{\text{accept}}/q_{\text{reject}}$ เลย ผลมี 3 แบบ: accept / reject / loop → ที่มาของ recognize กับ decide
3. **จริง** — DFA/PDA หัวเดินขวาอย่างเดียว อ่านทีละตัว (PDA อาจ ε-move แต่ไม่ถอยกลับ)
4. **จริง** — $\delta(q, a) = (r, b, \cdot)$ เขียน b ทับ a ได้ เช่นขีดฆ่า $0 \to x$ นี่คือ “หน่วยความจำ” ไม่จำกัดของ TM

สอนแทรก

- 3 สิ่งที่ทำให้ TM แรงกว่า PDA: **เขียนได้**, **เดินซ้ายได้**, **tape ไม่จำกัด** — ตัดข้อใดข้อหนึ่งออกจะอ่อนลง (เช่น stay-put แทน left → regular)

5.2 นิยาม 7-tuple

โจทย์ 5.2.1 — Sipser Exercise 3.5

ตอบพร้อมเหตุผล

1. TM เขียน $\sqcup$ ลงบน tape ได้ไหม
2. tape alphabet Γ เท่ากับ input alphabet Σ ได้ไหม
3. หัวอ่านอยู่ช่องเดิม 2 ก้าวติดกันได้ไหม
4. TM มี state เดียวได้ไหม

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ได้** — $\sqcup \in \Gamma$ และ $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\}$ เขียนตัวใดใน Γ ก็ได้ รวม $\sqcup$ (เช่น TM palindrome ลบตัวอักษรด้วย $\sqcup$)
2. **ไม่ได้** — นิยามบังคับ $\sqcup \in \Gamma$ และ $\sqcup \notin \Sigma \to \Gamma$ มีอย่างน้อย 1 ตัวที่ Σ ไม่มี $\to \Gamma \neq \Sigma$ เสมอ
3. **ได้** — ถ้าหัวอยู่ช่องซ้ายสุดแล้ว δ สั่ง L หัวอยู่ที่เดิม $\to$ configuration ถัดไปหัวอยู่ช่องเดิม
4. **ไม่ได้** — นิยามมี q_{accept} และ q_{reject} โดย $q_{\text{accept}} \neq q_{\text{reject}} \to$ อย่างน้อย 2 state

เขียนตอบข้อสอบ (English)

(a) Yes: $\sqcup \in \Gamma$, and δ may write any symbol of Γ. (b) No: $\sqcup \in \Gamma$ but $\sqcup \notin \Sigma$, so $\Gamma \neq \Sigma$. (c) Yes: if the head is on the leftmost cell and moves left, it stays on the same cell. (d) No: Q contains q_{accept} and q_{reject} with $q_{\text{accept}} \neq q_{\text{reject}}$, so $|Q| \geq 2$.

สอนแทรก

- ข้อแบบนี้ตอบจาก**นิยาม 7-tuple** ตรง ๆ — อ้างเงื่อนไขในนิยามทุกข้อ ($\sqcup \notin \Sigma$, $q_{\text{accept}} \neq q_{\text{reject}}$, ซ้ายสุดสั่ง L อยู่ที่เดิม)

โจทย์ 5.2.2 — 7-tuple + ตาราง δ ของ TM

เขียน TM ของ $\{0^n1^n \mid n \geq 0\}$ (state diagram ในเล่มสอนหัวข้อ 5.4) เป็น 7-tuple และตาราง δ

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

7-tuple

- $Q = \{q_1, q_2, q_3, q_4, q_{\text{accept}}, q_{\text{reject}}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, x, y, \sqcup\}$
- start $= q_1$, accept $= q_{\text{accept}}$, reject $= q_{\text{reject}}$

ตาราง δ (ช่องว่าง $= (q_{\text{reject}}, \cdot, \text{R})$)

	0	1	x	y	$\sqcup$
q_1	(q_2, x, R)			(q_4, y, R)	$(q_{\text{accept}}, \sqcup, \text{R})$
q_2	$(q_2, 0, \text{R})$	(q_3, y, L)		(q_2, y, R)	
q_3	$(q_3, 0, \text{L})$		(q_1, x, R)	(q_3, y, L)	
q_4				(q_4, y, R)	$(q_{\text{accept}}, \sqcup, \text{R})$

- ป้าย "$0 \to$ R" ในรูป = เขียนตัวเดิม $\to$ ในตารางเขียน $(q_2, 0, \text{R})$ ให้ครบ 3 ส่วน
- q_{accept}, q_{reject} ไม่มีแถว เพราะเครื่องหยุดทันที

! จุดที่คนเสียคะแนนบ่อย

- Σ ใส่ $\sqcup$ หรือ $x, y \to$ ผิด (Σ มีแค่ตัวอักษรของ input)
- ผลของ δ ของ TM เป็น**สามสิ่ง** $(q, b, \text{L/R})$ ไม่ใช่เซต (deterministic)

สอบแทรก

- แปลงจาก diagram เป็นตาราง: ลูกศร "$0 \to$ R" = $(q, 0, \text{R})$ — ต้องเขียนตัวเดิมกลับลงไปให้ครบ 3 ส่วน

5.3 Configuration $u\,q\,v$

อ่าน สัญลักษณ์: $\vdash$ อ่านว่า **"yields"** = TM ทำตาม δ **1 ก้าว** จาก configuration ซ้ายไปขวา แต่ละก้อน $u\,q\,v$ = tape uv, state q, หัวที่**ตัวแรกหลัง** q เช่น $q_0\,abc \vdash x\,q_1\,bc$: $\delta(q_0,a) = (q_1,x,\mathrm{R})$ เขียน x แทน a แล้วเลื่อนขวา (ต่างจาก $\Rightarrow$ ของ CFG = ใช้กฎ 1 ครั้ง)

โจทย์ 5.3.1 — ก้าวถัดไป

configuration ปัจจุบัน $10\,q_3\,01$ เขียน configuration ถัดไป เมื่อ

1. $\delta(q_3,0) = (q_5,1,\mathrm{L})$
2. $\delta(q_3,0) = (q_5,1,\mathrm{R})$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- หัวอยู่ที่ตัวแรกหลัง q_3 คือ 0 ขั้นตอน: (1) เขียน 1 ทับ 0 (2) ย้าย state ไปตามทิศ

เฉลยละเอียด

แยก $10\,q_3\,01$ เป็น $u\,a\,q_3\,b\,v$ โดย $u=1$, $a=0$ (ตัวทางซ้ายหัว), $b=0$ (ตัวใต้หัว), $v=1$

1. $\delta(q_3,0) = (q_5,1,\mathrm{L})$: สูตร $ua\,q_i\,bv \vdash u\,q_j\,acv$
   - เขียน $c=1$ แทน b $\rightarrow$ tape $1\,0\,1\,1$
   - เลื่อนซ้าย $\rightarrow$ state ไปอยู่**หน้า** a
   - ได้ $1\,q_5\,011$ (หัวอยู่ที่ 0 ตัวที่ 2)
2. $\delta(q_3,0) = (q_5,1,\mathrm{R})$: สูตร $ua\,q_i\,bv \vdash uac\,q_j\,v$
   - เขียน 1 แทน 0 $\rightarrow$ tape $1\,0\,1\,1$
   - เลื่อนขวา $\rightarrow$ state ไปอยู่**หลัง**ตัวที่เพิ่งเขียน
   - ได้ $101\,q_5\,1$ (หัวอยู่ที่ 1 ตัวสุดท้าย)

สอนแทรก

- เช็กคำตอบเร็ว ๆ: เนื้อหา tape (ลบ state ออก) ต้องเท่ากันทั้งสองกรณี = 1011 ต่างกันแค่ตำแหน่ง state
- พลาดบ่อย: เลื่อนซ้ายแล้วเขียน $10\,q_5\,11$ (ลืมว่า state ต้องข้ามไปหน้าตัวทางซ้าย)

โจทย์ 5.3.2 — Sipser Exercise 3.1 — configuration ของ M_2 (0^{2^n})

M_2 (Example 3.7) มีตาราง δ ด้านล่าง เขียนลำดับ configuration บน (a) 0 (b) 00 (c) 000

	0	x	⊔
q_1	$(q_2, ⊔, \mathrm{R})$	$(q_{\text{reject}}, x, \mathrm{R})$	$(q_{\text{reject}}, ⊔, \mathrm{R})$
q_2	(q_3, x, R)	(q_2, x, R)	$(q_{\text{accept}}, ⊔, \mathrm{R})$
q_3	$(q_4, 0, \mathrm{R})$	(q_3, x, R)	$(q_5, ⊔, \mathrm{L})$
q_4	(q_3, x, R)	(q_4, x, R)	$(q_{\text{reject}}, ⊔, \mathrm{R})$
q_5	$(q_5, 0, \mathrm{L})$	(q_5, x, L)	$(q_2, ⊔, \mathrm{R})$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- q_1 เขียน ⊔ ทับ 0 ตัวแรก (หมุดขอบซ้าย และนับ 0 ตัวนั้นด้วย)
- q_2 หา 0 ตัวแรกที่เหลือ q_3/q_4 สลับกันขีด 0 “ตัวเว้นตัว” และจำคู่/คี่ q_5 เดินกลับ

เฉลยละเอียด

(a) input 0

1. $q_1\,0 \vdash ⊔\,q_2\,⊔$ — $\delta(q_1, 0) = (q_2, ⊔, \mathrm{R})$
2. $\vdash ⊔⊔\,q_{\text{accept}}$ — q_2 เจอ ⊔ = เหลือ 0 ตัวเดียว ($1 = 2^0$) → **accept**

(b) input 00

1. $q_1\,00 \vdash ⊔\,q_2\,0$ — ลบ 0 แรก
2. $\vdash ⊔x\,q_3\,⊔$ — q_2 ขีด 0 ตัวที่สองเป็น x ไป q_3
3. $\vdash ⊔\,q_5\,x$ — q_3 เจอ ⊔ (รอบนี้จำนวนคู่) → เดินกลับ
4. $\vdash q_5\ ⊔\,x \vdash ⊔\,q_2\,x$ — ถึงหมุดซ้าย เริ่มรอบใหม่
5. $\vdash ⊔x\,q_2\,⊔ \vdash ⊔x\,⊔\,q_{\text{accept}}$ — ข้าม x เจอ ⊔ → **accept**

(c) input 000

1. $q_1\,000 \vdash ⊔\,q_2\,00 \vdash ⊔x\,q_3\,0$
2. $\vdash ⊔x0\,q_4\,⊔$ — q_3 อ่าน 0 ไม่ขีด ไป q_4 (นับคี่)
3. $\vdash ⊔x0\,⊔\,q_{\text{reject}}$ — q_4 เจอ ⊔ = จำนวน 0 ในรอบเป็นคี่และ > 1 → **reject**

สอนแทรก

- ทุกรอบขีด 0 ครึ่งหนึ่ง (หาร 2) เป็นกำลังของ 2 ⇔ หารลงตัวไปเรื่อย ๆ จนเหลือ 1
- เขียน configuration: บอก transition ที่ใช้ทุกก้าว อาจารย์ให้คะแนนแม้ก้าวท้าย ๆ ผิด

โจทย์ 5.3.3 — Sipser Exercise 3.2 — configuration ของ M_1 ($w\#w$)

M_1 (Example 3.9) มีตารางด้านล่าง (ช่องว่าง = reject, R/L เดี่ยว = state เดิมไม่เปลี่ยน tape) เขียนลำดับ configuration บน (a) 11 (b) 1#1 (c) 1##1

	0	1	#	x	⊔
q_1	(q_2, x, R)	(q_3, x, R)	$(q_8, \#, \mathrm{R})$		
q_2	R	R	$(q_4, \#, \mathrm{R})$		
q_3	R	R	$(q_5, \#, \mathrm{R})$		
q_4	(q_6, x, L)			R	
q_5		(q_6, x, L)		R	
q_6	L	L	$(q_7, \#, \mathrm{L})$	L	
q_7	L	L		(q_1, x, R)	
q_8				R	$(q_{\text{accept}}, ⊔, \mathrm{R})$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- q_2/q_3 = "จำว่าขีด 0 / 1 ไป" แล้ววิ่งหา # q_4/q_5 = หาตัวแรกที่ยังไม่ขีดหลัง # ต้องตรงกัน q_6, q_7 = เดินกลับ q_8 = เช็กว่าฝั่งขวาขีดหมด

เฉลยละเอียด

(a) 11

1. $q_1\,11 \vdash x\,q_3\,1$ — ขีด 1 จำด้วย q_3
2. $\vdash x1\,q_3\,⊔$ — q_3 ข้าม 1 หา #
3. $\vdash x1\,⊔\ q_{\text{reject}}$ — เจอ ⊔ ไม่มี # → **reject**

(b) 1#1

1. $q_1\,1\#1 \vdash x\,q_3\,\#1 \vdash x\#\,q_5\,1$ — ขีด 1 ข้าม #
2. $\vdash x\,q_6\,\#x$ — ตรงกัน ขีด 1 ฝั่งขวา เดินกลับ
3. $\vdash q_7\,x\#x \vdash x\,q_1\,\#x$ — กลับถึง x ตัวขวาสุดฝั่งซ้าย
4. $\vdash x\#\,q_8\,x \vdash x\#x\,q_8\,⊔ \vdash x\#x\,⊔\ q_{\text{accept}}$ — ฝั่งซ้ายหมด ฝั่งขวาเหลือแต่ x → **accept**

(c) 1##1

1. $q_1\,1\#\#1 \vdash x\,q_3\,\#\#1 \vdash x\#\,q_5\,\#1$
2. $\vdash x\#\#\,q_{\text{reject}}\,1$ — q_5 ต้องการ 1 หรือ x แต่เจอ # ตัวที่สอง ไม่มีเส้น → **reject**

สอนแทรก

- ช่อง "R" เดี่ยว ใน ตาราง = $(q, \text{ตัวเดิม}, \mathrm{R})$ อยู่ state เดิม ไม่ เปลี่ยน tape — เขียน configuration ต้อง คงตัวอักษรเดิม
- ช่องว่าง = ไป q_{reject} → configuration สุดท้ายคือ state q_{reject} ณ ตำแหน่งนั้น

โจทย์ 5.3.4 — Configuration ของ TM $a^n b^n c^n$ (TOC_Practice 6.2)

ใช้ TM ของ $\{a^n b^n c^n\}$ (โจทย์ 5.6 ถัดไป / ตารางในเล่มสอน 5.4) เขียนลำดับ configuration บน (1) abc (2) ab

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

(1) abc

ก้าว	configuration	transition
0	$q_0\, abc$	start
1	$x\, q_1\, bc$	$\delta(q_0, a) = (q_1, x, \mathrm{R})$
2	$xy\, q_2\, c$	$\delta(q_1, b) = (q_2, y, \mathrm{R})$
3	$x\, q_3\, yz$	$\delta(q_2, c) = (q_3, z, \mathrm{L})$
4	$q_3\, xyz$	$\delta(q_3, y) = (q_3, y, \mathrm{L})$
5	$x\, q_0\, yz$	$\delta(q_3, x) = (q_0, x, \mathrm{R})$
6	$xy\, q_4\, z$	$\delta(q_0, y) = (q_4, y, \mathrm{R})$
7	$xyz\, q_4\, \sqcup$	$\delta(q_4, z) = (q_4, z, \mathrm{R})$
8	$xyz\, \sqcup\, q_{\text{accept}}$	$\delta(q_4, \sqcup) = (q_{\text{accept}}, \sqcup, \mathrm{R})$

→ **accept ใน 8 ก้าว**

(2) ab: $q_0\, ab \vdash x\, q_1\, b \vdash xy\, q_2\, \sqcup \vdash xy\, \sqcup\, q_{\text{reject}}$ — q_2 หา c แต่เจอ $\sqcup$ ($\delta(q_2, \sqcup)$ ไม่กำหนด) → **reject**

สอนแทรก

- ก้าว 3 (เลื่อนซ้าย): เขียน z แทน c แล้ว state ไปอยู่**หน้า** y
- head เลยตัวสุดท้าย (ก้าว 7): เขียน $\sqcup$ ต่อท้ายหลัง state ให้เห็นว่าหัวอยู่บน blank

5.4 TM ตัวอย่าง: $\{0^n1^n\}$ (ขีดฆ่า x/y)

โจทย์ 5.4.1 — เขียนลำดับ configuration

ใช้ TM ของ $\{0^n1^n\}$ ในเล่มสอนหัวข้อ 5.4 เขียนลำดับ configuration บน 0101 แล้วบอกผล

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

ก้าว	configuration	ทำอะไร
0	$q_1\,0101$	start
1	$x\,q_2\,101$	q_1 อ่าน 0 → เขียน x, ไปขวา, หา 1
2	$q_3\,xy01$	q_2 อ่าน 1 → เขียน y, ไปซ้าย (หัวกลับมาที่ x)
3	$x\,q_1\,y01$	q_3 อ่าน x → ไปขวา กลับ q_1
4	$xy\,q_4\,01$	q_1 อ่าน y → “0 หมดแล้ว” ไป q_4 เช็กว่าเหลือแต่ y
5	q_{reject}	q_4 อ่าน 0: ไม่มีเส้น → **reject**

- ถูกต้องเพราะ 0101 ไม่ใช่ 0^n1^n — มี 0 อยู่หลัง 1

สอนแทรก

- TM นี้ไม่ได้เช็กรูปแบบ 0^*1^* ก่อน แต่จับได้ตอน q_4 เจอ 0 — ถ้าเขียน implementation เองควรใส่ขั้น “scan check form” ไว้ก่อนเพื่อให้อธิบายง่าย

โจทย์ 5.4.2 — Proof: TM นี้ decide $\{0^n1^n\}$ จริง

พิสูจน์ว่า TM ของ $\{0^n1^n\}$ (หัวข้อ 5.4 ในเล่มสอน) **หยุดทุก input** และ accept พอดีกับ string ใน $\{0^n1^n\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- หยุดเสมอ: ทุกรอบ ($q_1 \to q_2 \to q_3 \to q_1$) ขีด 0 หนึ่งตัว → รอบมีไม่เกิน $|w|$ ในรอบหัวเดินไม่เกิน $|w|+1$ ช่อง
- ถูกต้อง: แต่ละรอบจับ 0 ซ้ายสุดกับ 1 ซ้ายสุดที่ยังไม่ขีด

เขียนตอบข้อสอบ (English)

Halting.

- Each pass $q_1 \to q_2 \to q_3 \to q_1$ changes one 0 to x, so there are at most $|w|$ passes.
- Within a pass the head moves right to the first 1 and back to the last x, at most $|w| + 1$ cells each way; if no 1 is found, the machine rejects.
- State q_4 only moves right over y's and stops at the first other symbol.

So M halts on every input.

Correctness.

- After k passes, the tape is $x^k\,0^a\,y^k\,1^b\cdots$ when the input has the form 0^*1^*; each pass pairs one 0 with one 1.
- M reaches q_{accept} only when all 0s are crossed (q_1 sees y or $\sqcup$) and only y's remain before $\sqcup$; this happens iff the input is 0^n1^n.
- Any 1 before a 0, extra 0s, or extra 1s leads to a missing transition, so M rejects.

Hence M decides $\{0^n1^n\}$. ■

สอนแทรก

- พิสูจน์ TM เป็น decider = 2 ส่วนเสมอ: **หยุดทุก input** (นับรอบ) + **ตอบถูก** (accept ⇔ อยู่ในภาษา)

5.5 บรรยาย TM 3 ระดับ

โจทย์ 5.5.1 — Implementation-level description

เขียน TM ระดับ implementation ที่ decide $C = \{a^i b^j c^k \mid i \times j = k,\ i, j, k \geq 1\}$ (Example 3.11)

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- คูณ = บวกซ้ำ: ทุก a หนึ่งตัว ขีด c ไป j ตัว (หนึ่งตัวต่อ b แต่ละตัว) แล้วคืน b

เขียนตอบข้อสอบ (English)

M = "On input w:

1. Scan the input and check that it is a member of $a^+b^+c^+$; *reject* if it isn't.
2. Return the head to the left-hand end of the tape.
3. Cross off an a and scan to the right until a b occurs. Shuttle between the b's and the c's, crossing off one of each until all b's are gone. If all c's have been crossed off and some b's remain, *reject*.
4. Restore the crossed-off b's and repeat stage 3 if there is another a to cross off. If all a's have been crossed off, check whether all c's also have been crossed off. If yes, *accept*; otherwise, *reject*."

สอนแทรก

- implementation-level = บอกว่าหัวเดินอย่างไร เขียนอะไร แต่ไม่ต้องระบุ state — ระดับที่ข้อสอบส่วนใหญ่ต้องการ

5.6 เทคนิคออกแบบ TM

โจทย์ 5.6.1 — Sipser Exercise 3.8(b)

เขียน TM ระดับ implementation ที่ decide $\{w \in \{0,1\}^* \mid w$ มี 0 เป็นสองเท่าของ $1\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ขีด 1 หนึ่งตัวคู่กับ 0 สองตัวต่อรอบ ทุกรอบขีดอย่างน้อย 1 ตัว → หยุดเสมอ

เขียนตอบข้อสอบ (English)

$M =$ "On input w:

1. Scan the tape and cross off the first 1 that has not been crossed off. If no such 1 is found, go to stage 5.
2. Return to the left end; scan and cross off the first uncrossed 0. If none is found, *reject*.
3. Return to the left end; scan and cross off the next uncrossed 0. If none is found, *reject*.
4. Return the head to the left end and go to stage 1.
5. Scan the tape. If any uncrossed 0 remains, *reject*; otherwise, *accept*."

! จุดที่คนเสียคะแนนบ่อย

- ลืมขั้นตอนสุดท้ายที่เช็กว่าไม่มี 0 เหลือ → รับ 000 ผิด ๆ

สอบแทรก

- ทุกรอบขีด 1 หนึ่งตัวกับ 0 สองตัว → เหลือ 0 = มี 0 เกิน → reject; ไม่มี 0 ให้ขีด = 0 ขาด → reject

โจทย์ 5.6.2 — ออกแบบ TM + 7-tuple + ตาราง + trace

ออกแบบ TM ที่ decide $\{0^n 1^{2n} \mid n \geq 0\}$

1. เขียน 7-tuple และตาราง δ
2. เขียนลำดับ configuration บน 011 และบน 01

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ทุกรอบ: ขีด 0 หนึ่งตัวเป็น x แล้วขีด 1 **สองตัวติดกัน**เป็น y แล้ววิ่งกลับ
- 0 หมด (q_1 เจอ y) → เช็กว่าเหลือแต่ y แล้วเจอ $\sqcup$

เฉลยทีละขั้น

7-tuple: $Q = \{q_1, \ldots, q_5, q_{\text{accept}}, q_{\text{reject}}\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, x, y, \sqcup\}$, start q_1

	0	1	x	y	$\sqcup$	หน้าที่
q_1	(q_2, x, R)			(q_5, y, R)	$(q_{\text{accept}}, \sqcup, \text{R})$	หา 0 ตัวถัดไป
q_2	$(q_2, 0, \text{R})$	(q_3, y, R)		(q_2, y, R)		หา 1 ตัวแรก
q_3		(q_4, y, L)				1 ตัวที่สองต้องติดกัน
q_4	$(q_4, 0, \text{L})$		(q_1, x, R)	(q_4, y, L)		วิ่งกลับหา x
q_5				(q_5, y, R)	$(q_{\text{accept}}, \sqcup, \text{R})$	เช็กว่าเหลือแต่ y

บน 011:

$$q_1\,011 \vdash x\,q_2\,11 \vdash xy\,q_3\,1 \vdash x\,q_4\,yy \vdash q_4\,xyy \vdash x\,q_1\,yy \vdash xy\,q_5\,y \vdash xyy\,q_5\,\sqcup \vdash xyy\,\sqcup\,q_{\text{accept}}$$

→ accept

บน 01:

$$q_1\,01 \vdash x\,q_2\,1 \vdash xy\,q_3\,\sqcup$$

→ q_3 อ่าน $\sqcup$ ไม่มีเส้น → reject

- หยุดเสมอ: ทุกรอบขีดอย่างน้อย 1 ตัว และไม่มีรอบไหนวนโดยไม่ขีด

สอนแทรก

- ออกแบบ TM: เขียน algorithm ภาษาคน → กำหนด state ตามงาน → ใส่ลูกศรเฉพาะที่ถูก → trace 1 ตัวที่ accept และ 1 ตัวที่ reject

โจทย์ 5.6.3 — ออกแบบ TM: $\{a^n b^n c^n\}$ (TOC_Practice 6.1)

ออกแบบ TM ที่ decide $\{a^n b^n c^n \mid n \geq 0\}$ ให้ทั้ง implementation-level และตาราง δ

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- รอบละ: ขีด $a \to x$, $b \to y$, $c \to z$ อย่างละตัว แล้วเดินกลับ a หมด $\to$ เช็กว่าเหลือแต่ y, z

เขียนตอบข้อสอบ (English)

$M =$ "On input w:

1. If the current cell is $\sqcup$, *accept*.
2. Cross off the leftmost a as x.
3. Move right over a's and y's; cross off the first b as y. If none, *reject*.
4. Move right over b's and z's; cross off the first c as z. If none, *reject*.
5. Move left to the rightmost x and step right. If the cell is a, go to stage 2.
6. If it is y, scan right: if only y's and z's remain before $\sqcup$, *accept*; otherwise *reject*."

ตาราง δ (ช่องว่าง = q_{reject})

	a	b	c	x	y	z	$\sqcup$
q_0	(q_1, x, R)				(q_4, y, R)		$(q_{\text{accept}}, \sqcup, \mathrm{R})$
q_1	(q_1, a, R)	(q_2, y, R)			(q_1, y, R)		
q_2		(q_2, b, R)	(q_3, z, L)			(q_2, z, R)	
q_3	(q_3, a, L)	(q_3, b, L)		(q_0, x, R)	(q_3, y, L)	(q_3, z, L)	
q_4					(q_4, y, R)	(q_4, z, R)	$(q_{\text{accept}}, \sqcup, \mathrm{R})$

- หยุดเสมอ: รอบไม่เกิน $|w|$ รอบ แต่ละรอบเดินไม่เกิน $2|w| + 2$ ขั้น
- ลำดับผิด เช่น $abcabc$: รอบสุดท้าย q_4 เจอ $a \to$ reject

สอนแทรก

- ภาษานี้คือ witness "decidable แต่ไม่ CFL" — TM ทำได้เพราะ**เขียนทับและเดินกลับ**ได้ ซึ่ง PDA ทำไม่ได้

โจทย์ 5.6.4 — ออกแบบ TM: palindrome (TOC_Practice 6.3)

ออกแบบ TM ที่ decide palindrome บน $\{a, b\}$ แล้วเขียนลำดับ configuration บน aba

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ลบตัวซ้ายสุด **จำค่าใน state** (q_a/q_b) → วิ่งไปขวาสุด เช็กว่าตรงแล้วลบ → กลับซ้าย ทำซ้ำ
- TM เดินกลับได้ จึงไม่ต้องเดากึ่งกลางเหมือน PDA

เฉลยทีละขั้น

	a	b	$\sqcup$	หน้าที่
q_0	$(q_a, \sqcup, \mathrm{R})$	$(q_b, \sqcup, \mathrm{R})$	$(q_{\text{accept}}, \sqcup, \mathrm{R})$	ลบตัวซ้ายสุด
q_a	(q_a, a, R)	(q_a, b, R)	$(p_a, \sqcup, \mathrm{L})$	จำ a วิ่งขวาสุด
q_b	(q_b, a, R)	(q_b, b, R)	$(p_b, \sqcup, \mathrm{L})$	จำ b วิ่งขวาสุด
p_a	$(q_r, \sqcup, \mathrm{L})$		$(q_{\text{accept}}, \sqcup, \mathrm{R})$	ตัวขวาสุดต้องเป็น a
p_b		$(q_r, \sqcup, \mathrm{L})$	$(q_{\text{accept}}, \sqcup, \mathrm{R})$	ตัวขวาสุดต้องเป็น b
q_r	(q_r, a, L)	(q_r, b, L)	$(q_0, \sqcup, \mathrm{R})$	เดินกลับซ้ายสุด

Trace aba:

$$q_0\,aba \vdash \sqcup\, q_a\, ba \vdash \sqcup b\, q_a\, a \vdash \sqcup ba\, q_a\, \sqcup \vdash \sqcup b\, p_a\, a \vdash \sqcup\, q_r\, b \vdash q_r\, \sqcup b$$

$$\vdash \sqcup\, q_0\, b \vdash \sqcup\sqcup\, q_b\, \sqcup \vdash \sqcup\, p_b\, \sqcup \vdash \sqcup\sqcup\, q_{\text{accept}}$$

- p_b อ่าน $\sqcup$ = ตัวที่ลบเป็น**ตัวกลาง**ของ string ยาวคี่ → accept
- หยุดเสมอ: รอบละลบ 2 ตัว เวลาประมาณ n^2

สอนแทรก

- เทคนิค “จำค่าใน state” (q_a/q_b) ใช้เมื่อข้อมูลที่ต้องจำมีจำนวนจำกัด

บทที่ 6

Variants of Turing Machines

6.1 “เท่ากัน” คืออะไร + Stay-put

โจทย์ 6.1.1 — Doubly infinite tape (Sipser Problem 3.11)

TM ที่ tape ยาวไม่จำกัด**ทั้งซ้ายและขวา** รู้จำภาษาชุดเดียวกับ TM ธรรมดา — พิสูจน์

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ทิศง่าย: TM ธรรมดาจำลองบน tape สองทางได้ (ใส่เครื่องหมายขอบซ้าย ไม่ไปทางซ้ายของมัน)
- ทิศยาก: TM ธรรมดาจำลอง tape สองทางด้วย 2 tape (ขวา + ซ้ายกลับหลัง) แล้วใช้ Thm 3.13

เขียนตอบข้อสอบ (English)

- ($\Rightarrow$) A doubly-infinite-tape TM simulates a standard TM M: it writes a special end-marker to the left of the input and treats it as the left end of M's tape (moving left onto it means staying put on the first cell).
- ($\Leftarrow$) A standard 2-tape TM simulates a doubly-infinite-tape TM D: tape 1 holds the cells from the starting position rightward; tape 2 holds the cells to the left in reverse order. When D's head crosses the starting cell, the simulator switches tapes and reverses direction.

By Theorem 3.13 the 2-tape TM has an equivalent single-tape TM, so the models are equivalent.

สอบแทรก

- โจทย์ “equivalent” ต้องสองทิศเสมอ — ทิศง่าย (กรณีพิเศษ) ห้ามข้าม เขียน 1–2 ประโยคก็พอ

6.2 Multitape TM → single tape

โจทย์ 6.2.1 — วาง tape เดียว

TM 2 tape: tape 1 = $1\,0\,\underline{1}\,1$ (หัวที่ตัวที่ 3), tape 2 = $\underline{a}\,b$ (หัวที่ตัวแรก)

1. เขียน tape เดียวที่ single-tape TM ใช้แทน
2. ถ้า 2-tape TM ใช้เวลา n^2 single-tape ใช้เวลาเท่าไร (Big-O)

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **เรียง tape**: # + tape 1 + # + tape 2 + # ใส่จุดบนตัวที่หัวชี้

$$\#\,1\,0\,\dot{1}\,1\,\#\,\dot{a}\,b\,\#$$

   - $\dot{1}$ = หัวของ tape 1 อยู่ตัวที่ 3 $\dot{a}$ = หัวของ tape 2 อยู่ตัวแรก
   - จุดเป็น tape symbol ใหม่ ($\dot{0}, \dot{1}, \dot{a}, \cdots \in \Gamma'$)
2. **เวลา**: 1 ก้าวของ 2-tape TM ใช้ 2 รอบสแกนทั้ง tape (ยาว $O(t(n))$)
   - $t(n)$ ก้าว $\times$ $O(t(n))$ ต่อก้าว = $O(t^2(n))$ $t(n) = n^2 \to$ $O(n^4)$

สอนแทรก

- 1 ก้าวจำลอง: (1) สแกนซ้าย→ขวาอ่านตัวที่มีจุดทุก tape (2) สแกนอีกรอบเขียน/ย้ายจุดตาม δ (3) ถ้าจุดชน # → เลื่อนทุกอย่างทางขวา 1 ช่องแล้วใส่ ⊔
- ต่างกันแค่กำลังสอง (polynomial) → P ไม่เปลี่ยนเมื่อเปลี่ยนจาก multitape เป็น single-tape

6.3 Nondeterministic TM → deterministic

โจทย์ 6.3.1 — NTM + BFS

1. เขียน NTM ที่ decide $COMPOSITES = \{\langle x \rangle \mid x = pq,\ p, q > 1\}$
2. ทำไม DTM ที่จำลอง NTM ต้องค้นแบบ BFS ไม่ใช่ DFS

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. NTM ของ COMPOSITES

1. N = "On input $\langle x \rangle$:
2. Nondeterministically choose p with $1 < p < x$ (เขียน p ในฐาน 2 ทีละบิต → แต่ละบิตเป็นจุดแตกกิ่ง)
3. If p divides x, *accept*; otherwise, *reject*."

- x ประกอบ ⇒ มี p ที่หาร x ลงตัว ⇒ มีกิ่ง accept ⇒ N accept
- x ไม่ประกอบ ⇒ ทุกกิ่งหาร x ไม่ลงตัว ⇒ ทุกกิ่ง reject
- ทุกกิ่งหยุด (เลือก p จำกัดบิต + หาร) ⇒ N เป็น decider

2. ทำไม BFS

- tree การคำนวณอาจมีกิ่ง**ยาวไม่สิ้นสุด**
- DFS ลงกิ่งแรกจนสุด → ถ้ากิ่งนั้น loop จะไม่กลับมา → พลาดกิ่ง accept ที่อยู่ข้าง ๆ
- BFS ไล่ทีละ ระดับ (address $\varepsilon, 1, 2, \ldots, 11, 12, \ldots$) → กิ่ง accept ที่ลึก d ถูกเจอ หลังไล่ครบ ระดับ d ซึ่งมีจำนวนจำกัด

สอนแทรก

- รูปแบบ **guess and verify**: NTM "เดา" คำตอบ แล้ว "ตรวจ" แบบ deterministic — ใช้อีกครั้งตอนนิยาม NP (บทที่ 11)

โจทย์ 6.3.2 — Proof: recognizable ปิดภายใต้ concatenation (ใช้ NTM)

ให้ M_1, M_2 recognize L_1, L_2 พิสูจน์ว่า L_1L_2 Turing-recognizable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ถ้าลองทุกจุดตัดทีละจุดแบบ decidable: M_1 อาจ loop ที่จุดตัดผิด → ไม่ได้ลองจุดถัดไป
- NTM **เดา**จุดตัด → แต่ละกิ่งรันจุดตัดเดียว ไม่ขวางกัน

เขียนตอบข้อสอบ (English)

N = "On input w:

1. Nondeterministically split w as $w = xy$.
2. Run M_1 on x. If it rejects, *reject*.
3. If M_1 accepts, run M_2 on y. If it accepts, *accept*; if it rejects, *reject*."

- $w \in L_1L_2 \Rightarrow$ some split has $x \in L_1$, $y \in L_2 \Rightarrow$ that branch accepts $\Rightarrow$ N accepts.
- $w \notin L_1L_2 \Rightarrow$ no branch accepts (each either rejects or loops).

So N recognizes L_1L_2, and by Corollary 3.18 a deterministic TM recognizes it too. ■

สอนแทรก

- ทำไม NTM แก้ปัญหา loop: แต่ละกิ่งลองจุดตัดเดียว กิ่งที่ loop ไม่ขวางกิ่งอื่น (BFS จำลองได้)

โจทย์ 6.3.3 — Sipser Exercise 3.3 — พิสูจน์ Corollary 3.19

ปรับบทพิสูจน์ Thm 3.16 เพื่อแสดง: decidable $\iff$ มี NTM ที่ decide (ใช้ได้: tree ที่ทุก node มีลูกจำกัด และทุกกิ่งจำกัด มี node จำกัด)

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- D ของ Thm 3.16 ค้น BFS ไม่จบถ้าไม่มีกิ่ง accept → เพิ่มขั้น "ทุกกิ่ง reject แล้ว → reject"
- NTM decider: ทุกกิ่งหยุด → tree จำกัด → BFS ค้นครบได้

เขียนตอบข้อสอบ (English)

($\Rightarrow$) A deterministic decider is a special case of a nondeterministic decider.
($\Leftarrow$) Let NTM N decide L. Modify D from Theorem 3.16 to D':

1. Stages 1–3 as before (copy input, simulate the branch given by the address on tape 3).
2. **New stage:** if every branch of length ℓ has ended (rejected or the address is invalid) before using all addresses of length ℓ, *reject*.
3. Otherwise advance to the next address and repeat.

- If N accepts w, D' eventually finds the accepting branch.
- If not, every branch halts and rejects; each node has at most b children, so the tree is finite and D' finishes searching it and rejects.

So D' is a decider for L. ■

สอนแทรก

- ประเด็นหลักที่ต้องเขียน: tree ของ NTM decider **จำกัด** (ทุกกิ่งจำกัด + ลูกจำกัด) จึงค้นครบแล้ว reject ได้

6.4 Enumerator

โจทย์ 6.4.1 — Sipser Problem 3.18

พิสูจน์: ภาษา A decidable **ก็ต่อเมื่อ** มี enumerator ที่พิมพ์ A **ตามลำดับมาตรฐาน** (string order)

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ($\Rightarrow$) decider หยุดเสมอ $\rightarrow$ ไล่ $s_1, s_2, \ldots$ ทีละตัวได้เลย ไม่ต้อง dovetail
- ($\Leftarrow$) พิมพ์ตามลำดับ $\rightarrow$ ถ้าพิมพ์ตัวที่ "เลย w ไปแล้ว" แปลว่า w ไม่มีวันถูกพิมพ์

เขียนตอบข้อสอบ (English)

($\Rightarrow$) Let M decide A and $s_1, s_2, \ldots$ be all strings in string order. E = "Ignore the input.
1. For $i = 1, 2, 3, \ldots$: run M on s_i.
2. If M accepts, print s_i."

M always halts, so every s_i is tested in turn and A is printed in string order.

($\Leftarrow$) Let E enumerate A in string order.

- If A is finite, it is decidable (finite languages are regular).
- Otherwise M = "On input w: Run E until it prints w or a string after w in string order. If it printed w, *accept*; otherwise, *reject*."

Since A is infinite, E eventually prints a string after w, so M always halts and decides A. ■

สอนแทรก

- string order = เรียงตามความยาวแล้วพจนานุกรม — ทิศ ($\Leftarrow$) อาศัยว่า "พิมพ์เลย w ไปแล้ว" = w ไม่มีวันถูกพิมพ์

โจทย์ 6.4.2 — Sipser Exercise 3.6 — ทำไม enumerator แบบง่ายใช้ไม่ได้

ในทิศ ($\Rightarrow$) ของ Thm 3.21 ทำไมใช้ E = "for $i = 1, 2, \ldots$: run M on s_i; if it accepts, print s_i" ไม่ได้

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. M เป็นแค่ recognizer $\rightarrow$ อาจ loop บน s_i ที่ไม่อยู่ในภาษา
2. ถ้า loop บน s_i $\rightarrow$ ไม่ได้ทดสอบ $s_{i+1}, s_{i+2}, \ldots$ เลย $\rightarrow$ พิมพ์ไม่ครบ
3. ตัวอย่าง: M loop บน $s_1 = \varepsilon$ แต่ accept ทุกตัวอื่น $\rightarrow$ E ไม่พิมพ์อะไรเลย ทั้งที่ $L(M) = \Sigma^* \setminus \{\varepsilon\}$
4. วิธี ถูก: dovetail — รอบ i รัน M แค่ i ก้าว บน $s_1..s_i$ $\rightarrow$ ถ้า M accept s_j ใน k ก้าว จะ พิมพ์ ใน รอบ $\max(j, k)$

- ถ้า M เป็น decider แบบง่ายใช้ได้ (และพิมพ์ตาม string order — Problem 3.18)

สอนแทรก

- dovetailing คือท่าเดียวกับที่ใช้พิสูจน์ $\overline{E_{\text{TM}}}$ recognizable — ใช้ทุกครั้งที่ต้อง "ลองหลายตัวที่อาจ loop"

6.5 Variant อื่น ๆ

โจทย์ 6.5.1 — เท่ากับ TM หรืออ่อนกว่า

1. PDA ที่มี 2 stack
2. TM ที่หัวเดินขวาอย่างเดียว
3. TM ที่มี stay put **แทน** left
4. queue automaton
5. TM หลายหัวบน tape เดียว
6. PDA ที่มี 1 stack

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **2-stack PDA = TM** — stack ซ้าย = tape ทางซ้ายของหัว, stack ขวา = ตัวใต้หัว + ทางขวา; เดินขวา/ซ้าย = ย้ายยอดระหว่าง stack
2. **หัวเดินขวาอย่างเดียว → regular** — อ่านแต่ละช่องครั้งเดียว สิ่งที่เขียนไม่มีวันถูกอ่าน ความจำมีแค่ state = DFA
3. **stay-put แทน left → regular** — ถอยกลับไม่ได้ เหมือนข้อ 2 (Problem 3.13)
4. **queue automaton = TM** — วนคิวจำลองการเดินบน tape ได้ (เอาตัวหน้าไปต่อท้าย = เลื่อนหัว)
5. **หลายหัวบน tape เดียว = TM** — เก็บตำแหน่งหัวด้วยจุดแบบ multitape
6. **1-stack PDA → CFL** (อ่อนกว่า) — ทำ $\{a^n b^n c^n\}$ ไม่ได้

สอนแทรก

- กฎจำ: variant ที่ “**เพิ่ม**ความ สามารถ” (tape, หัว, nondeterminism, stack ที่ 2) → **เท่ากับ TM** variant ที่ “**ตัด**การกลับไปอ่าน” → **อ่อนลง**

โจทย์ 6.5.2 — Proof: PDA ที่มี 2 stack เท่ากับ TM

พิสูจน์ว่าภาษาเป็น Turing-recognizable ก็ต่อเมื่อมี 2-stack PDA ที่รู้จำภาษานั้น (บรรยายระดับ simulation)

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- tape ของ TM = stack ซ้าย (ส่วนทางซ้ายของหัว) + stack ขวา (ตัวใต้หัวและทางขวา, ตัวใต้หัวอยู่บนสุด)
- เดินขวา = ย้ายยอดจาก stack ขวาไป stack ซ้าย เดินซ้าย = ย้ายกลับ

เขียนตอบข้อสอบ (English)

(2-stack PDA $\Rightarrow$ TM) A 2-tape TM keeps each stack on its own tape (top at the right end) and simulates each push/ pop by writing or erasing at that end. By Theorem 3.13 it has a single-tape equivalent.

(TM $\Rightarrow$ 2-stack PDA) Given TM M:

1. First read the whole input, pushing it on stack L; then pop it all onto stack R, so the first input symbol is on top of R (the head cell).
2. To simulate $\delta(q,a) = (r,b,\mathrm{R})$ with a on top of R: pop a from R and push b on L. If R becomes empty, push $\sqcup$ on R.
3. To simulate $\delta(q,a) = (r,b,\mathrm{L})$: pop a from R, push b on R, then move the top of L (if any) onto R.
4. Accept when M enters q_{accept}.

Each move of M is simulated by finitely many stack operations, so the two models recognize the same languages. ■

สอนแทรก

- stack ซ้าย + stack ขวา = tape ทั้งเส้น — ภาพนี้อธิบายได้ทันทีว่าทำไม 2 stack แรงกว่า 1 stack มาก

โจทย์ 6.5.3 — Sipser Problem 3.13 — stay put แทน left

TM ที่ $\delta : Q \times \Gamma \to Q \times \Gamma \times \{R, S\}$ (เลื่อนซ้ายไม่ได้) แสดงว่าไม่ equivalent กับ TM ปกติ และ recognize ภาษาชั้นไหน

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- กลับไปซ้ายไม่ได้ → สิ่งที่เขียนไว้ไม่มีวันถูกอ่านอีก → ความจำจริงมีแค่ state → เหมือน DFA

เฉลยทีละขั้น

คำตอบ: recognize ได้พอดี regular languages

1. **TM แบบนี้ → DFA:** state ของ DFA = state ของ M (+ state ดักตาย)
   - $\delta_D(q, a)$ = state ตอน M **เลื่อนขวาออก**จากช่องที่มี a เมื่อเข้ามาด้วย q (ไม่ขึ้น S ต่อกัน)
   - คู่ (state, symbol) จำกัด → วน ซ้ำ คู่ เดิม = loop → ไป state ดัก ตาย เจอ q_{accept} → accept แบบดูดกลืน
   - accept state ของ D = q ที่ M จะ accept เมื่อที่เหลือเป็น $\sqcup$ ทั้งหมด
2. **DFA → TM แบบนี้:** DFA = TM ที่เลื่อนขวาอย่างเดียวและไม่เขียน
3. ไม่ equivalent: TM ปกติ decide $\{0^n 1^n\}$ ได้ แต่ variant นี้ไม่ได้ (ไม่ regular)

สอนแทรก

- เทียบกับ Problem 3.11: **เพิ่ม** tape ทางซ้ายไม่ทำให้แรงขึ้น แต่**ตัด**การเดินซ้ายทำให้อ่อนลงจนเหลือ regular

บทที่ 7

Church–Turing Thesis

7.1 Thesis และทำไมเป็น thesis

โจทย์ 7.1.1 — อธิบาย

1. Church–Turing thesis พูดว่าอะไร
2. ทำไมเป็น “thesis” ไม่ใช่ “theorem”
3. ผลของ thesis ต่อการเขียนบทพิสูจน์ในข้อสอบ

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ตัว thesis**: แนวคิด algorithm ตาม สามัญสำนึก = algorithm ที่ ทำ บน TM ได้ (Church: λ-calculus, Turing: TM, 1936 — พบว่าเท่ากัน)
2. **ทำไมเป็น thesis**:
   - “algorithm ตามสามัญสำนึก” ไม่มีนิยามทางคณิต → พิสูจน์เท่ากับ TM ทางคณิตไม่ได้
   - หลักฐาน: ทุกโมเดลที่เสนอมา (multitape, NTM, RAM, ภาษาโปรแกรม) พิสูจน์แล้วว่าเท่ากับ TM
3. **ผลต่อข้อสอบ**:
   - เขียน TM เป็น high-level description (algorithm ภาษาอังกฤษ) ได้ ไม่ต้องวาด state ทุกครั้ง
   - พิสูจน์ว่าไม่มี TM decide → สรุปได้ว่า**ไม่มี algorithm/โปรแกรมใด**แก้ได้

เขียนตอบข้อสอบ (English)

The Church–Turing thesis states that the intuitive notion of an algorithm is equivalent to Turing machine algorithms. It is a thesis, not a theorem, because “algorithm in the intuitive sense” has no formal definition, so it cannot be proved; it is supported by the fact that every reasonable model of computation proposed is equivalent to TMs. Consequently we may describe TMs by high-level algorithms, and showing that no TM decides a language shows that no algorithm decides it.

สอบแทรก

- ข้อสอบมักถาม “why is it called a thesis” — คำตอบสั้นที่สุด: “algorithm” ไม่มีนิยามทางคณิต จึงพิสูจน์ไม่ได้

โจทย์ 7.1.2 — ทฤษฎี: thesis กับการพิสูจน์ undecidability

อธิบาย (ภาษาอังกฤษ 3–4 ประโยค) ว่าทำไมการพิสูจน์ว่า “ไม่มี TM ใด decide A_{TM}” จึงสรุปได้ว่า “ไม่มีโปรแกรมคอมพิวเตอร์ใดตัดสิน A_{TM} ได้”

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

1. Theorem 4.11 is a mathematical statement about Turing machines only.
2. The Church–Turing thesis says that every algorithm, in the intuitive sense, can be carried out by a Turing machine.
3. A computer program that decides A_{TM} would be such an algorithm, so it would give a TM that decides A_{TM}.
4. No such TM exists, so no program in any language, on any hardware, can decide A_{TM}.

สอนแทรก

- thesis เป็นสะพานจาก “ไม่มี TM” (คณิต) ไป “ไม่มีโปรแกรม” (โลกจริง)

7.2 Hilbert's 10th problem

โจทย์ 7.2.1 — ตัวแปรเดียว

ให้ $p(x) = x^3 - 2x^2 - 5x + 6$

1. ใช้ขอบเขต $\pm k\, c_{\max}/|c_1|$ หาช่วงที่ต้องลอง
2. หารากจำนวนเต็ม
3. ทำไม D (หลายตัวแปร) แค่ recognizable แต่ D_1 (ตัวแปรเดียว) decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ขอบเขต**
   - k = จำนวนพจน์ $= 4$ (x^3, $-2x^2$, $-5x$, 6)
   - $c_{\max}$ = สัมประสิทธิ์ค่าสัมบูรณ์มากสุด $= 6$ c_1 = สัมประสิทธิ์ดีกรีสูงสุด $= 1$
   - รากจำนวนเต็มต้องอยู่ใน $\pm k\, c_{\max}/|c_1| = \pm 24 \to$ ลอง $x = -24, \ldots, 24$ (49 ค่า)
2. **ราก**: $p(1) = 1 - 2 - 5 + 6 = 0$ $p(3) = 27 - 18 - 15 + 6 = 0$ $p(-2) = -8 - 8 + 10 + 6 = 0 \to$ $1,\ 3,\ -2$ (ตรวจ: $(x-1)(x-3)(x+2) = x^3 - 2x^2 - 5x + 6$ ✓)
3. **ทำไมต่างกัน**
   - D (หลายตัวแปร): ไม่มีขอบเขต $\to$ "ลองทุกชุด" ถ้าไม่มีรากจะลองไม่จบ $\to$ recognizer เท่านั้น Matijasevič (1970) พิสูจน์ว่า undecidable
   - D_1 (ตัวแปรเดียว): ขอบเขตจำกัด $\to$ ลองครบแล้ว reject ได้ $\to$ decider

สอนแทรก

- บทเรียนที่ใช้ทั้งวิชา: **ค้นไม่จำกัด = recognizer หาขอบเขตการค้นได้ = decider** (เช่น A_{CFG} ใช้ $2n - 1$, A_{LBA} ใช้ qng^n)

7.3 Encoding + high-level description

โจทย์ 7.3.1 — Sipser Exercise 3.7

อธิบายว่าทำไมคำบรรยายนี้**ไม่ใช่** TM ที่ถูกต้อง
M_{bad} = "On input $\langle p \rangle$, a polynomial over variables $x_1, \ldots, x_k$: 1. Try all possible settings of $x_1, \ldots, x_k$ to integer values. 2. Evaluate p on all of these settings. 3. If any of these settings evaluates to 0, *accept*; otherwise, *reject*."

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. ขั้น 1 สั่ง "try **all** settings" — ค่าจำนวนเต็มของ $x_1..x_k$ มี**อนันต์**ชุด
2. ทุกขั้นของ TM ต้องทำเสร็จในจำนวนก้าวจำกัด → ขั้น 1–2 ทำไม่เสร็จ → ไม่ใช่คำบรรยาย TM ที่ถูกต้อง
3. ผลคือไม่มีวันถึงขั้น 3 ส่วน "otherwise, reject" → คำตอบ reject ไม่มีทางเกิด
4. **แก้ให้ถูก (recognizer)**: "For $i = 1, 2, \ldots$: evaluate p on the ith integer setting; if it is 0, *accept*." เครื่องนี้ถูกต้องตามนิยาม แต่ loop เมื่อไม่มีราก

เขียนตอบข้อสอบ (English)

M_{bad} is not a legitimate TM description because stage 1 asks it to try infinitely many settings, which cannot be completed in finitely many steps; hence it can never reach the "otherwise, reject" part. A legitimate recognizer instead tries the settings one at a time and accepts as soon as one evaluates to 0.

สอนแทรก

- กฎ: ทุกขั้นใน TM description ต้องทำเสร็จได้ในเวลาจำกัด — "try all" บนเซตอนันต์ผิดทันที

โจทย์ 7.3.2 — High-level description

เขียน TM ระดับ high-level ที่ decide $\{\langle G\rangle \mid G$ เป็นกราฟไม่มีทิศที่มี node ซึ่งต่อกับ node อื่นทุกตัว$\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- "มี node ที่ต่อกับทุก node" = มี node ที่ degree $= m - 1$ ไล่ทุก node แล้วนับ edge

เขียนตอบข้อสอบ (English)

$M =$ "On input $\langle G\rangle$, where G is an undirected graph with m nodes:
1. Check that $\langle G\rangle$ is a valid encoding; if not, *reject*.
2. For each node v of G:
3. Count the edges incident to v. If v is adjacent to all other $m - 1$ nodes, *accept*.
4. If no such node is found, *reject*."

Halts: the loop runs m times and each count scans the finite edge list. **Correct:** M accepts iff some node has degree $m - 1$.

สอนแทรก

- high-level description = เขียนเป็น algorithm ได้เลย ไม่ต้องพูดถึงหัวอ่าน (Church–Turing)
- ต้องมีเหตุผล "halts" เสมอถ้าโจทย์บอก "decide"

Decidable Languages

8.1 ชื่อปัญหามาตรฐาน

โจทย์ 8.1.1 — เขียนเป็นเซต

เขียนภาษาต่อไปนี้เป็นเซต: E_{CFG}, EQ_{DFA}, ALL_{NFA}, A_{TM}

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

- $E_{\text{CFG}} = \{\langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset\}$ — E = Emptiness
- $EQ_{\text{DFA}} = \{\langle A, B \rangle \mid A, B \text{ are DFAs and } L(A) = L(B)\}$ — EQ = Equivalence (input เป็นคู่)
- $ALL_{\text{NFA}} = \{\langle N \rangle \mid N \text{ is an NFA and } L(N) = \Sigma^*\}$ — รับทุก string
- $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ — A = Acceptance (input เป็นคู่ เครื่อง + string)

อ่านรูปแบบ: $\{\langle$ของที่รับ$\rangle \mid$ ชนิดของสิ่งนั้น และเงื่อนไข yes$\}$

สอนแทรก

- ต้องเขียน "G is a CFG" ด้วย — ไม่งั้น string ที่ไม่ใช่ encoding ของ CFG ก็จะถือว่าอยู่ในภาษา
- ชื่อบอกความหมาย: A_X (accept), E_X (empty), EQ_X (equal), ALL_X (all)

8.2 ภาษา regular: decidable ทั้งหมด

โจทย์ 8.2.1 — Subset ของ DFA

แสดงว่า $SUBSET_{\text{DFA}} = \{\langle A, B\rangle \mid A, B \text{ DFA และ } L(A) \subseteq L(B)\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- $L(A) \subseteq L(B) \iff$ ไม่มี string ที่อยู่ใน A แต่ไม่อยู่ใน $B \iff L(A) \cap \overline{L(B)} = \emptyset$

เขียนตอบข้อสอบ (English)

$M =$ "On input $\langle A, B\rangle$:

1. Construct DFA C with $L(C) = L(A) \cap \overline{L(B)}$ (complement and intersection constructions).
2. Run the decider T for E_{DFA} on $\langle C\rangle$.
3. If T accepts, *accept*; otherwise, *reject*."

- Halts: the construction is finite and T is a decider.
- Correct: $L(A) \subseteq L(B)$ iff $L(C) = \emptyset$ iff T accepts.

สอนแทรก

- ท่ามาตรฐาน: เปลี่ยนคำถามเป็น "ภาษาว่างไหม" ด้วย closure ($\cap$, complement) แล้วเรียก E_{DFA}

โจทย์ 8.2.2 — Proof: A_{NFA} decidable (Theorem 4.2)

พิสูจน์ว่า $A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ is an NFA that accepts } w\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- แปลงเป็นปัญหาที่แก้ได้แล้ว: NFA $\to$ DFA (subset construction) แล้วเรียก decider ของ A_{DFA}

เขียนตอบข้อสอบ (English)

N = "On input $\langle B, w\rangle$, where B is an NFA and w is a string:
1. Convert B to an equivalent DFA C using the subset construction.
2. Run the decider M for A_{DFA} (Theorem 4.1) on $\langle C, w\rangle$.
3. If M accepts, *accept*; otherwise, *reject*."

- Halts: the subset construction produces at most $2^{|Q|}$ states in finitely many steps, and M is a decider.
- Correct: $L(C) = L(B)$, so B accepts w iff C accepts w iff M accepts.

สอนแทรก

- ท่า "แปลงเป็นสิ่งที่แก้ได้แล้ว" — NFA→DFA, regex→NFA ใช้ซ้ำได้ทุกข้อเกี่ยวกับ regular

โจทย์ 8.2.3 — Sipser Exercise 4.2 — DFA กับ regex เท่ากันไหม

เขียนปัญหา “DFA A กับ regex R equivalent ไหม” เป็นภาษา แล้วแสดงว่า decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

$EQ_{\text{DFA,REX}} = \{\langle A, R\rangle \mid A \text{ is a DFA, } R \text{ is a regular expression, } L(A) = L(R)\}$.
$X =$ “On input $\langle A, R\rangle$:

1. Convert R to an equivalent NFA N (Lemma 1.55), then to an equivalent DFA B (Theorem 1.39).
2. Run the decider F for EQ_{DFA} on $\langle A, B\rangle$.
3. If F accepts, *accept*; otherwise, *reject*.”

- **Halts:** both conversions are algorithms, and F is a decider.
- **Correct:** $L(B) = L(R)$, so F accepts iff $L(A) = L(R)$.

สอนแทรก

- DFA, NFA, regex แปลงหากันได้ด้วย algorithm → สำหรับ decidability ให้แบบไหนมาก็เท่ากัน

โจทย์ 8.2.4 — Sipser Problem 4.13 — subset ของ regex

แสดงว่า $\{\langle R, S\rangle \mid R, S \text{ เป็น regex และ } L(R) \subseteq L(S)\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- $L(R) \subseteq L(S) \iff L(R) \cap \overline{L(S)} = \emptyset$
- ต้องแปลงเป็น DFA ก่อน complement — สลับ accept ของ NFA ไม่ได้ complement

เขียนตอบข้อสอบ (English)

X = "On input $\langle R, S\rangle$:
1. Convert R and S to DFAs D_R, D_S.
2. Build $\overline{D_S}$ by swapping accept states, then DFA C with $L(C) = L(D_R) \cap L(\overline{D_S})$.
3. Run the decider T for E_{DFA} on $\langle C\rangle$. If T accepts, *accept*; otherwise, *reject*."

T accepts iff $L(R) \cap \overline{L(S)} = \emptyset$ iff $L(R) \subseteq L(S)$; every step halts.

! จุดที่คนเสียคะแนนบ่อย

- ปัญหาเดียวกันกับ CFG ($L(G) \subseteq L(H)$) ไม่ decidable

สอบแทรก

- $L(R) \subseteq L(S) \Leftrightarrow L(R) \cap \overline{L(S)} = \emptyset$ — สูตร subset ที่ใช้บ่อย
- กับ CFG ทำไม่ได้ (CFL ไม่ปิด complement/∩) → undecidable

8.3 ภาษา CFL

โจทย์ 8.3.1 — รัน decider ของ E_{CFG}

ใช้ algorithm mark ของ Thm 4.8 กับ G: $S \to AB \mid C,\ A \to aA \mid a,\ B \to bB,\ C \to cC \mid c$ แล้วบอกว่า decider ตอบอะไร

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

Algorithm (Thm 4.8): mark terminal → mark ตัวแปร A ถ้ามีกฎ $A \to U_1 \cdots U_k$ ที่ทุก U_i ถูก mark → ทำซ้ำ → start ไม่ถูก mark = ภาษาว่าง

1. **รอบ 0**: mark a, b, c
2. **รอบ 1**: $A \to a$ (ทุกตัว marked) → mark A $C \to c$ → mark C $B \to bB$ ต้องมี B marked ก่อน → ยังไม่ได้
3. **รอบ 2**: $S \to C$ → mark S B ยัง mark ไม่ได้
4. **รอบ 3**: ไม่มีตัวใหม่ → หยุด
5. start S ถูก mark → $L(G) \neq \emptyset$ → decider ของ E_{CFG} **reject**

- ยืนยัน: $S \Rightarrow C \Rightarrow c \to c \in L(G)$ จริง
- B ไม่ถูก mark = B ไม่มีวันจบเป็น terminal ล้วน ($B \to bB \to bbB \to \cdots$) → $S \to AB$ ใช้ไม่ได้ แต่ไม่มีผล เพราะ $S \to C$ ใช้ได้

สอนแทรก

- "mark" = "ตัวนี้สร้าง string ของ terminal ล้วนได้" — ไล่จากล่างขึ้นบน
- ระวังทิศคำตอบ: E_{CFG} **accept เมื่อว่าง** → start ถูก mark (ไม่ว่าง) = reject

โจทย์ 8.3.2 — ทำไมไม่ simulate PDA

ทำไม decider ของ A_{CFG} ต้องแปลงเป็น CNF ก่อน แทนที่จะแปลงเป็น PDA แล้ว simulate

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ปัญหาของ PDA**: PDA เป็น nondeterministic และมี ε-move ได้
   - บางกิ่ง push/pop วนไปเรื่อย ๆ โดย**ไม่อ่าน input** (เช่น $\varepsilon, \varepsilon \to X$ วนตัวเอง)
   - simulate ทุกกิ่ง $\to$ กิ่งพวกนี้ไม่จบ $\to$ simulator ไม่หยุด $\to$ ไม่ใช่ decider
2. **ปัญหาของ grammar ทั่วไป**: derivation มีได้ไม่จำกัด (เช่น $S \to SS \mid \varepsilon$) $\to$ ลองไปเรื่อย ๆ ไม่จบ
3. **CNF แก้ได้**: ทุก derivation ของ $|w| = n$ ยาว $2n - 1$ พอดี
   - derivation ยาว $2n - 1$ มีจำนวน**จำกัด** (กฎจำกัด $\times$ ขั้นจำกัด)
   - ลองครบแล้วตอบได้เสมอ $\to$ หยุดแน่

เขียนตอบข้อสอบ (English)

A PDA may have branches that push and pop forever without reading input, so simulating it might never halt. Converting G to Chomsky normal form guarantees that every derivation of a string of length $n \geq 1$ has exactly $2n - 1$ steps, so only finitely many derivations need to be checked, and the decider always halts.

สอนแทรก

- คำถาม “why” แบบนี้ออกบ่อย — ตอบ 2 ประเด็น: PDA อาจไม่หยุด + CNF ให้ขอบเขต $2n - 1$

โจทย์ 8.3.3 — Sipser Exercise 4.4 — CFG สร้าง ε ได้ไหม

แสดงว่า $\{\langle G\rangle \mid G$ เป็น CFG ที่สร้าง $\varepsilon\}$ decidable (2 วิธี)

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด — 2 วิธี

วิธี 1: เรียก decider เดิม (สั้นสุด)

1. ให้ S = decider ของ A_{CFG} (Thm 4.7)
2. M = "On input $\langle G\rangle$: run S on $\langle G, \varepsilon\rangle$; accept iff S accepts."
3. หยุดเสมอเพราะ S เป็น decider ถูกเพราะ S accept $\langle G, \varepsilon\rangle \Leftrightarrow G$ สร้าง ε

วิธี 2: หาตัวแปร nullable

1. mark ทุก A ที่มีกฎ $A \to \varepsilon$
2. ทำซ้ำ: mark A ถ้ามีกฎ $A \to U_1 \cdots U_k$ ที่ทุก U_i เป็น**ตัวแปร**ที่ mark แล้ว (ห้ามมี terminal)
3. start ถูก mark $\to$ accept ไม่งั้น reject
4. วนไม่เกิน $|V|$ รอบ (แต่ละรอบ mark ใหม่อย่างน้อย 1 หรือหยุด) $\to$ หยุดเสมอ

เขียนตอบข้อสอบ (English)

Let S be the decider for A_{CFG} (Theorem 4.7). M = "On input $\langle G\rangle$: Run S on $\langle G, \varepsilon\rangle$. If S accepts, *accept*; otherwise, *reject*." M halts because S is a decider, and M accepts iff G generates ε.

สอนแทรก

- โจทย์ "show decidable" ส่วนใหญ่ตอบได้ด้วย การ**เรียก decider ที่รู้แล้ว** (รายการ: A/E_{DFA}, EQ_{DFA}, A/E_{CFG}) — ลองคิดท่านี้ก่อน

โจทย์ 8.3.4 — Sipser Problem 4.14 — CFG สร้าง string ใน 1^* ได้ไหม

แสดงว่า $\{\langle G\rangle \mid G$ เป็น CFG บน $\{0,1\}$ และ $1^* \cap L(G) \neq \emptyset\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- 1^* regular → CFL ∩ regular = CFL สร้างได้ → ถาม E_{CFG} แล้ว**กลับคำตอบ**
- ทางลัด: ลบทุกกฎที่มี terminal 0 ด้านขวา (string ใน 1^* ใช้กฎพวกนั้นไม่ได้อยู่แล้ว)

เขียนตอบข้อสอบ (English)

$X =$ "On input $\langle G\rangle$:
1. Build CFG H with $L(H) = 1^* \cap L(G)$ (e.g. delete every rule of G whose right-hand side contains 0).
2. Run the decider R for E_{CFG} on $\langle H\rangle$.
3. If R accepts, *reject*; if R rejects, *accept*."

R rejects iff $L(H) \neq \emptyset$ iff $1^* \cap L(G) \neq \emptyset$; every step halts.

! จุดที่คนเสียคะแนนบ่อย

- "ลอง 1^n ทุกตัวด้วย A_{CFG}" → มีไม่จำกัด ได้แค่ recognizer

สอนแทรก

- "ไม่ว่าง" = กลับคำตอบของ E_{CFG} — เช็กทิศคำตอบทุกครั้งก่อนเขียนขั้นสุดท้าย

8.4 สูตรพิสูจน์ decidable + แบบฝึก

โจทย์ 8.4.1 — DFA ที่ accept string ยาวคู่

แสดงว่า $\{\langle A\rangle \mid A$ เป็น DFA ที่ accept string ยาวคู่อย่างน้อย 1 ตัว$\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- "มี string แบบนี้ใน $L(A)$" = $L(A) \cap L(E) \neq \emptyset$ โดย E = DFA ของ string ยาวคู่ → ถาม E_{DFA} แล้ว**กลับคำตอบ**

เขียนตอบข้อสอบ (English)

Let E be a DFA recognizing all strings of even length. $M =$ "On input $\langle A\rangle$:

1. Construct DFA C with $L(C) = L(A) \cap L(E)$.
2. Run the decider T for E_{DFA} on $\langle C\rangle$.
3. If T accepts, *reject*; if T rejects, *accept*."

- **Halts:** the construction is finite and T is a decider.
- **Correct:** A accepts some even-length string iff $L(C) \neq \emptyset$ iff T rejects iff M accepts.

! จุดที่คนเสียคะแนนบ่อย

- ตอบตาม T โดยไม่กลับ → ได้ภาษาตรงข้าม (DFA ที่**ไม่** accept string ยาวคู่เลย)

สอนแทรก

- แม่แบบ "DFA accept **บาง** string ที่มีคุณสมบัติ P": $\cap$ กับ DFA ของ $P \to E_{\text{DFA}} \to$ กลับคำตอบ

โจทย์ 8.4.2 — Sipser Exercise 4.3 — ALL_{DFA}

แสดงว่า $ALL_{DFA} = \{\langle A \rangle \mid A \text{ เป็น DFA และ } L(A) = \Sigma^*\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- "รับทุก string" ⇔ "complement ว่าง" → แปลงเป็นปัญหา E_{DFA} ที่แก้ได้แล้ว

เขียนตอบข้อสอบ (English)

M = "On input $\langle A \rangle$, where A is a DFA:
1. Construct DFA A' by swapping the accept and non-accept states of A, so $L(A') = \overline{L(A)}$.
2. Run the decider T for E_{DFA} on $\langle A' \rangle$.
3. If T accepts, *accept*; otherwise, *reject*."

- **Halts:** the construction is finite and T is a decider.
- **Correct:** $L(A) = \Sigma^*$ iff $\overline{L(A)} = \emptyset$ iff T accepts $\langle A' \rangle$.

สอนแทรก

- สลับ accept state ได้ complement **เฉพาะ DFA** — กับ NFA ใช้ไม่ได้ (ต้องแปลงเป็น DFA ก่อน)
- ALL_{CFG} ทำแบบนี้**ไม่ได้** (CFL ไม่ปิด complement) → undecidable (Thm 5.13)

โจทย์ 8.4.3 — Sipser Problem 4.10 — $INFINITE_{DFA}$

แสดงว่า $\{\langle A\rangle \mid A$ เป็น DFA และ $L(A)$ อนันต์$\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- A มี k state: $L(A)$ อนันต์ $\iff$ A accept บาง string ยาว $\geq k$
- ($\Leftarrow$) pumping lemma ($p = k$) ปั๊มได้ไม่จำกัด ($\Rightarrow$) ภาษาอนันต์มี string ยาวเท่าไรก็ได้

เขียนตอบข้อสอบ (English)

$I =$ "On input $\langle A\rangle$, where A has k states:
1. Construct DFA D accepting all strings of length $\geq k$.
2. Construct DFA C with $L(C) = L(A) \cap L(D)$.
3. Run the decider T for E_{DFA} on $\langle C\rangle$. If T accepts, *reject*; if T rejects, *accept*."

$L(C) \neq \emptyset$ iff A accepts a string of length $\geq k$ iff $L(A)$ is infinite. Every step halts.

สอนแทรก

- ขอบเขต k (จำนวน state) มาจาก pumping lemma ของ regular — ท่า "จำกัดขอบเขตการค้น"

โจทย์ 8.4.4 — Sipser Problem 4.12 — DFA ที่ไม่ accept string ที่มี 1 จำนวนคี่

แสดงว่า $\{\langle M\rangle \mid M$ เป็น DFA ที่ไม่ accept string ใดที่มี 1 จำนวนคี่$\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. สร้าง DFA D ของ "1 จำนวนคี่" (2 state: คู่/คี่ — อ่าน 1 สลับ อ่าน 0 อยู่ที่เดิม)
2. สร้าง B ที่ $L(B) = L(M) \cap L(D)$
3. รัน T (E_{DFA}) บน $\langle B\rangle \rightarrow$ **ตอบตาม** T (ไม่กลับ)

แม่แบบ "DFA M (ไม่) accept string ที่มีคุณสมบัติ P"

- P ต้อง regular $\rightarrow$ สร้าง DFA ของ $P \rightarrow \cap$ กับ $L(M) \rightarrow E_{\text{DFA}}$
- "**ไม่** accept string ใดที่มี P" $\rightarrow$ ตอบตาม T "accept **บาง** string ที่มี P" $\rightarrow$ กลับคำตอบ

สอนแทรก

- "ไม่ accept string ใดที่มี P" = ตอบ**ตาม** E_{DFA} (ไม่กลับ) — คู่กับข้อ "ยาวคู่" ที่ต้องกลับ

8.5 Closure: decidable vs recognizable

โจทย์ 8.5.1 — star และ intersection

1. แสดงว่า decidable ปิดภายใต้ star
2. แสดงวา recognizable ปิดภายใต intersection
3. ทำไมพิสูจน์ “recognizable ปิด complement” ด้วยการสลับ accept/reject ไม่ได้

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English) — 1. Decidable languages are closed under star

Let M_1 decide L. $M =$ “On input w:
1. If $w = \varepsilon$, *accept*.
2. For each way to cut w into nonempty pieces $w = w_1 w_2 \cdots w_k$:
3. Run M_1 on each w_i. If M_1 accepts all of them, *accept*.
4. If no cut works, *reject*.”

- Halts: there are $2^{|w|-1}$ cuts and M_1 is a decider.
- Correct: M accepts iff $w \in L^*$.

เขียนตอบข้อสอบ (English) — 2. Recognizable languages are closed under intersection

Let M_1, M_2 recognize L_1, L_2. $M =$ “On input w:
1. Run M_1 on w. If it rejects, *reject*.
2. If M_1 accepts, run M_2 on w. If M_2 accepts, *accept*; if it rejects, *reject*.”

- $w \in L_1 \cap L_2 \Rightarrow$ both accept $\Rightarrow M$ accepts.
- $w \notin L_1 \cap L_2 \Rightarrow$ some machine rejects or loops $\Rightarrow M$ does not accept.

เฉลยข้อ 3

- recognizer อาจ loop บน $w \notin L$
- สลับ accept/reject แล้ว w นั้นยัง loop $\rightarrow$ ไม่ถูก accept ทั้งที่ $w \in \overline{L}$
- (และ A_{TM} recognizable แต่ $\overline{A_{\text{TM}}}$ ไม่ recognizable $\rightarrow$ ไม่ปิดจริง)

สอบแทรก

- star: ลองทุกวิธีตัด ($2^{|w|-1}$ แบบ) recognizable: รันต่อกันได้กับ $\cap$ แต่ไม่ได้กับ $\cup$

โจทย์ 8.5.2 — Proof: decidable ปิดภายใต้ complement และ intersection

ให้ M_1, M_2 decide L_1, L_2 พิสูจน์ว่า $\overline{L_1}$ และ $L_1 \cap L_2$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

Complement. M = "On input w: Run M_1 on w. If it accepts, *reject*; if it rejects, *accept*."

- M_1 always halts, so M always halts, and M accepts iff $w \notin L_1$.

Intersection. M = "On input w:

1. Run M_1 on w. If it rejects, *reject*.
2. Run M_2 on w. If it accepts, *accept*; otherwise, *reject*."

- Both are deciders, so M always halts.
- M accepts iff both accept iff $w \in L_1 \cap L_2$.

สอนแทรก

- complement ใช้ได้เพราะ decider**หยุดเสมอ** — เขียนเหตุผลนี้ให้ชัด (คือจุดที่ต่างจาก recognizable)

โจทย์ 8.5.3 — Sipser Problem 3.15(b), 3.16(a) — concatenation และ union

(1) decidable ปิดภายใต้ concatenation (2) recognizable ปิดภายใต้ union

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English) — 1. Decidable — concatenation

M = "On input w:
1. For each way to cut $w = xy$ ($|w| + 1$ ways): run M_1 on x and M_2 on y; if both accept, *accept*.
2. If no cut works, *reject*."

Finitely many cuts and both are deciders, so M halts; it accepts iff $w \in L_1 L_2$.

เขียนตอบข้อสอบ (English) — 2. Recognizable — union

U = "On input w: Run M_1 and M_2 on w in parallel (alternate steps, two tapes). If either accepts, *accept*."

- $w \in L_1 \cup L_2 \Rightarrow$ one machine accepts in finitely many steps $\Rightarrow U$ accepts.
- $w \notin L_1 \cup L_2 \Rightarrow$ neither accepts $\Rightarrow U$ does not accept.

Running one after the other fails: if M_1 loops, M_2 never runs.

สอนแทรก

- decidable: ลองทุกจุดตัดทีละจุดได้ recognizable: ต้องขนาน/NTM เดา — ข้อสอบชอบถามความต่างนี้

บทที่ 9

Undecidability

9.1 A_{TM} และ universal TM

โจทย์ 9.1.1 — สมาชิกของ A_{TM}

ให้ M = "On input x: ถ้า $|x|$ เป็นคู่ accept ถ้า $|x|$ คี่ วนไม่หยุด"

1. $\langle M, 01\rangle \in A_{\text{TM}}$ ไหม $\langle M, 0\rangle \in A_{\text{TM}}$ ไหม
2. U ทำอะไรบน $\langle M, 0\rangle$
3. ทำไมแก้ U ให้ "รันแค่ 10^6 ก้าว ไม่จบให้ reject" ไม่ได้

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **สมาชิก**
   - $\langle M, 01\rangle$: $|01| = 2$ คู่ $\to$ M accept $\to$ $\in A_{\text{TM}}$
   - $\langle M, 0\rangle$: $|0| = 1$ คี่ $\to$ M loop $\to$ loop **ไม่ใช่** accept $\to$ $\notin A_{\text{TM}}$
2. **U บน $\langle M, 0\rangle$**: U จำลอง M ทีละก้าว $\to$ M ไม่หยุด $\to$ **U loop ตาม**
   - ยังถูกนิยาม recognizer: $\langle M, 0\rangle \notin A_{\text{TM}}$ และ recognizer ได้รับอนุญาตให้ loop เมื่อคำตอบเป็น no
   - แต่ทำให้ U **ไม่ใช่** decider
3. **ทำไมจำกัดก้าวไม่ได้**
   - สร้าง M_t = "เดิน 10^6 ก้าวแล้ว accept" $\to$ $\langle M_t, w\rangle \in A_{\text{TM}}$
   - U ที่ตัดที่ 10^6 ก้าวจะ reject ผิด ตั้งขีดจำกัด k เท่าไรก็มีเครื่องที่ accept ก้าว $k+1$
   - แก้ไม่ได้ด้วยขีดจำกัดใด ๆ $\to$ ตรงกับ Thm 4.11 ว่า A_{TM} undecidable

สอนแทรก

- "loop" กับ "reject" ต่างกันมาก: reject = หยุดและตอบ no loop = ไม่หยุด ไม่ตอบอะไร — ทั้งคู่ไม่ใช่ accept

9.2 Countable และ Diagonalization

โจทย์ 9.2.1 — countable / uncountable

1. แสดงว่า $\mathbb{Z}$ countable
2. แสดงว่า $\mathbb{N} \times \mathbb{N}$ countable
3. แสดงว่าเซต $\mathcal{B}$ ของลำดับ 0/1 อนันต์ uncountable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. เรียง $0, 1, -1, 2, -2, 3, -3, \ldots$ คือ $f(1) = 0,\ f(2k) = k,\ f(2k+1) = -k$ — ทุกจำนวนเต็มมีตำแหน่งเดียว (correspondence)
2. เขียนตาราง แถว i หลัก j = (i, j) แล้วไล่**แนวทแยง**ที่ $i + j$ คงที่: $(1,1), (1,2), (2,1), (1,3), (2,2), (3,1), \ldots$ — ทุกคู่ถูกนับครั้งเดียว
3. สมมติเรียงได้ $b_1, b_2, b_3, \ldots$
   - สร้าง d: บิตที่ i ของ d = กลับบิตที่ i ของ b_i
   - $d \neq b_i$ ทุก i (ต่างที่บิต i) แต่ $d \in \mathcal{B} \rightarrow$ ขัดแย้ง

ผลตามมา

ภาษาแต่ละภาษา = ลำดับ 0/1 อนันต์ (characteristic sequence) → เซตของภาษา uncountable เซตของ TM countable → **มีภาษาที่ไม่ recognizable**

สอนแทรก

- countable = เรียงเป็นลำดับได้ครบโดย**ทุกตัวอยู่ที่ตำแหน่งจำกัด**
- uncountable พิสูจน์ด้วย diagonalization: สมมติเรียงได้ → สร้างตัวที่ต่างจากทุกแถวในแนวทแยง

โจทย์ 9.2.2 — Proof: มีภาษาที่ไม่ Turing-recognizable (Corollary 4.18)

พิสูจน์ด้วยการนับ: (1) เซตของ TM ทั้งหมด countable (2) เซตของภาษาทั้งหมดบน Σ uncountable (3) สรุป

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

1. **TMs are countable.** Σ^* is countable (list strings by length, then lexicographically). Every TM M has an encoding $\langle M \rangle \in \Sigma^*$; omitting strings that are not legal encodings gives a list of all TMs.
2. **Languages are uncountable.** Let $s_1, s_2, \ldots$ list Σ^*. Map each language A to its characteristic sequence χ_A, whose ith bit is 1 iff $s_i \in A$; this is a correspondence with the set $\mathcal{B}$ of infinite binary sequences. $\mathcal{B}$ is uncountable by diagonalization: given any list $b_1, b_2, \ldots$, the sequence whose ith bit differs from the ith bit of b_i is not in the list.
3. **Conclusion.** Each TM recognizes one language, and there are only countably many TMs but uncountably many languages. So some language is recognized by no TM. ■

สอนแทรก

- บทพิสูจน์นี้ non-constructive — บอกว่ามี แต่ไม่บอกว่าภาษาไหน (ตัวอย่างจริงคือ $\overline{A_{\text{TM}}}$)

โจทย์ 9.2.3 — Sipser Exercise 4.6 — one-to-one, onto, correspondence

$X = \{1, .., 5\}$, $Y = \{6, .., 10\}$ $f = (6, 7, 6, 7, 6)$ และ $g = (10, 9, 8, 7, 6)$ (ค่าที่ $n = 1..5$) แต่ละตัว one-to-one / onto / correspondence ไหม

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

นิยาม: one-to-one = input ต่างให้ output ต่าง onto = ทุกตัวใน Y ถูกชี้ correspondence = ทั้งสองอย่าง

(a) f one-to-one? **ไม่** — $f(1) = f(3) = 6$ (input ต่าง output เดียวกัน)

(b) f onto? **ไม่** — ไม่มี n ที่ $f(n) = 8$ (และ 9, 10)

(c) f correspondence? **ไม่** — ไม่ผ่าน (a) และ (b)

(d) g one-to-one? **ใช่** — $10, 9, 8, 7, 6$ ต่างกันหมด

(e) g onto? **ใช่** — ทุกตัวใน $\{6, .., 10\}$ ถูกชี้

(f) g correspondence? **ใช่** — ผ่าน (d)(e); $g(n) = 11 - n$

สอนแทรก

- X, Y **ขนาดเท่ากัน** เพราะ**มี** correspondence อย่างน้อย 1 ตัว (g) — f ไม่เป็นก็ไม่เปลี่ยนข้อสรุป
- จะพิสูจน์ว่าขนาด**ไม่เท่า** ต้องแสดงว่า**ไม่มี**ฟังก์ชันใดเลยเป็น correspondence → ใช้ diagonalization (เช่น $\mathbb{N}$ กับ $\mathbb{R}$)

โจทย์ 9.2.4 — Sipser Exercise 4.8 — $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$ countable

แสดงว่า $T = \{(i, j, k) \mid i, j, k \in \mathbb{N}\}$ countable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

วิธี 1: ไล่ตามผลรวม $s = i + j + k = 3, 4, 5, \ldots$

1. แต่ละกลุ่ม s มี triple จำนวนจำกัด (ไม่เกิน s^3)
2. เรียงกลุ่ม $s = 3, 4, 5, \ldots$ ในกลุ่มเรียงแบบพจนานุกรม: $(1,1,1), (1,1,2), (1,2,1), (2,1,1), (1,1,3), \ldots$
3. ทุก triple อยู่ที่ตำแหน่ง**จำกัด** และไม่ซ้ำ $\rightarrow$ correspondence กับ $\mathbb{N}$

วิธี 2: ใช้ correspondence $h : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ สองครั้ง: $(i, j, k) \mapsto h(h(i, j), k)$

! จุดที่คนเสียคะแนนบ่อย

- ไล่ $(1,1,1), (1,1,2), (1,1,3), \ldots \rightarrow$ ไม่มีวันถึง $(1,2,1)$ ทุกตัวต้องมีตัวก่อนหน้า**จำกัด**

สอนแทรก

- ใช้ได้กับ $\mathbb{N}^k$ ทุก k และเซตของ tuple จำกัดทั้งหมด

9.3 A_{TM} undecidable (Theorem 4.11)

โจทย์ 9.3.1 — ถ้า D ไม่กลับคำตอบ

ในบทพิสูจน์ Thm 4.11 ถ้าเปลี่ยน D ให้ตอบ**ตาม** H (ไม่กลับ) จะยังได้ข้อขัดแย้งไหม แล้วเขียนบทพิสูจน์ที่ถูกต้อง

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

- $D'(\langle D' \rangle)$ = accept $\iff$ D' accept $\langle D' \rangle \to$ **ไม่ขัดแย้ง** (จริงได้ทั้งสองแบบ)
- ต้อง **ตอบตรงข้าม** ถึงจะได้ "accept $\iff$ ไม่ accept"

เขียนตอบข้อสอบ (English) — Theorem 4.11

Assume A_{TM} is decidable, with decider H: $H(\langle M, w \rangle)$ accepts if M accepts w and rejects otherwise.
Construct $D =$ "On input $\langle M \rangle$:

1. Run H on $\langle M, \langle M \rangle \rangle$.
2. If H accepts, *reject*; if H rejects, *accept*."

Run D on $\langle D \rangle$: D accepts $\langle D \rangle$ iff D does not accept $\langle D \rangle$ — a contradiction. So H cannot exist, and A_{TM} is undecidable. ■

สอนแทรก

- การ "ตอบตรงข้าม" คือหัวใจของ diagonalization — ต้องเขียนให้เห็นว่า $D(\langle D \rangle)$ accept $\Leftrightarrow$ $D(\langle D \rangle)$ ไม่ accept

9.4 Co-recognizable และ Theorem 4.22

โจทย์ 9.4.1 — ใช้ Thm 4.22

1. พิสูจน์ทิศ (⇐) ของ Thm 4.22
2. E_{TM} เป็น co-recognizable ไหม

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English) — 1. Theorem 4.22 (⇐)

Let M_1 recognize A and M_2 recognize $\overline{A}$. $M =$ "On input w:
1. Run M_1 and M_2 on w in parallel (alternate steps).
2. If M_1 accepts, *accept*; if M_2 accepts, *reject*."

Every w is in A or $\overline{A}$, so one machine accepts; hence M always halts and decides A. ■

เฉลยข้อ 2

- $\overline{E_{\text{TM}}} = \{\langle M\rangle \mid L(M) \neq \emptyset\}$ recognizable ด้วย dovetailing (รัน M บน $s_1..s_i$ คนละ i ก้าว)
- $\rightarrow E_{\text{TM}}$ **เป็น co-recognizable**

สอนแทรก

- Thm 4.22 ใช้ 2 ทาง: (1) recognizable + co-recognizable ⇒ decidable (2) recognizable + undecidable ⇒ complement ไม่ recognizable

โจทย์ 9.4.2 — ทฤษฎี: recognizable + co-recognizable

ให้ L เป็น Turing-recognizable และ $\overline{L}$ เป็น Turing-recognizable ทำไม การ “รัน M_1 ให้ จบ ก่อน แล้ว ค่อย รัน M_2” จึงใช้สร้าง decider ไม่ได้ และวิธีที่ถูกคืออะไร

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

ทำไม “รัน M_1 จบก่อนแล้วค่อย M_2” ใช้ไม่ได้

1. M_1 recognize $L \rightarrow$ บน $w \notin L$ (คือ $w \in \overline{L}$) M_1 อาจ loop
2. ถ้า M_1 loop $\rightarrow$ ไม่มีวันได้รัน M_2 ทั้งที่ M_2 จะ accept w ได้
3. เครื่องรวมจึงไม่หยุดบน $w \rightarrow$ ไม่ใช่ decider

วิธีที่ถูก: รันสลับทีละก้าว (parallel)

1. M = “On input w: รัน M_1 1 ก้าว, M_2 1 ก้าว, สลับไปเรื่อย ๆ (ใช้ 2 tape)”
2. M_1 accept $\rightarrow$ accept M_2 accept $\rightarrow$ reject
3. ทุก w อยู่ใน L หรือ $\overline{L}$ อย่างใดอย่างหนึ่ง $\rightarrow$ เครื่องใดเครื่องหนึ่ง accept ในเวลาจำกัด $\rightarrow$ M **หยุดเสมอ**
4. M accept $\Leftrightarrow$ M_1 accept $\Leftrightarrow$ $w \in L \rightarrow M$ decide L (Thm 4.22)

เขียนตอบข้อสอบ (English)

Running M_1 to completion first fails because M_1 may loop on $w \in \overline{L}$, so M_2 would never run. Instead run M_1 and M_2 alternately, one step each. Since every w is in L or $\overline{L}$, one of them accepts after finitely many steps; accept if M_1 accepts and reject if M_2 accepts. This machine always halts and decides L.

สอนแทรก

- “รันขนาน” ใช้ทุกครั้งที่มี recognizer หลายตัวซึ่งอาจ loop

โจทย์ 9.4.3 — Sipser Exercise 4.5 — $\overline{E_{\text{TM}}}$ recognizable (บทพิสูจน์เต็ม)

แสดงว่า $\overline{E_{\text{TM}}}$ Turing-recognizable แล้วสรุปเรื่อง E_{TM}

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

Let $s_1, s_2, \ldots$ list Σ^*. $R =$ "On input $\langle M \rangle$:

1. Repeat for $i = 1, 2, 3, \ldots$:
2. Run M for i steps on each of $s_1, \ldots, s_i$.
3. If M accepts any of them, *accept*."

- $L(M) \neq \emptyset \Rightarrow M$ accepts some s_j in k steps $\Rightarrow R$ accepts at round $\max(j, k)$.
- $L(M) = \emptyset \Rightarrow R$ never accepts.

So R recognizes $\overline{E_{\text{TM}}}$. Since E_{TM} is undecidable (Thm 5.2), Theorem 4.22 gives that E_{TM} is **not** Turing-recognizable.

- ทางเลือก NTM: "เดา w แล้ว simulate M บน w accept ถ้า M accept"

สอนแทรก

- ผลตามมาที่ข้อสอบชอบถามต่อ: E_{TM} **ไม่** recognizable (Thm 4.22 + Thm 5.2)

9.5 $\overline{A_{\text{TM}}}$ ไม่ recognizable (Cor 4.23)

โจทย์ 9.5.1 — $\overline{HALT_{\text{TM}}}$

แสดงว่า $\overline{HALT_{\text{TM}}}$ ไม่ Turing-recognizable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- สูตร เดียว กับ Cor 4.23: "ตัว มัน recognizable + ตัว มัน undecidable ⇒ complement ไม่ recognizable"

เฉลยละเอียด

1. $HALT_{\text{TM}}$ **recognizable**: R = "simulate M บน w; ถ้า M หยุด (accept หรือ reject) → accept"
   - M หยุด → R accept ✓ M loop → R loop (ได้ เพราะคำตอบเป็น no)
2. $HALT_{\text{TM}}$ **undecidable**: Theorem 5.1 (reduce จาก A_{TM})
3. **สมมติ** $\overline{HALT_{\text{TM}}}$ recognizable → $HALT_{\text{TM}}$ เป็นทั้ง recognizable และ co-recognizable
4. Thm 4.22 → $HALT_{\text{TM}}$ decidable → ขัดกับข้อ 2
5. ดังนั้น $\overline{HALT_{\text{TM}}}$ **ไม่ Turing-recognizable**

เขียนตอบข้อสอบ (English)

$HALT_{\text{TM}}$ is Turing-recognizable: simulate M on w and accept if it halts. $HALT_{\text{TM}}$ is undecidable (Theorem 5.1). If $\overline{HALT_{\text{TM}}}$ were Turing-recognizable, $HALT_{\text{TM}}$ would be both Turing-recognizable and co-Turing-recognizable, hence decidable by Theorem 4.22 — a contradiction. So $\overline{HALT_{\text{TM}}}$ is not Turing-recognizable. ■

สอนแทรก

- ใช้สูตรเดียวกันได้กับทุกภาษาที่ recognizable แต่ undecidable: A_{TM}, $HALT_{\text{TM}}$, $\overline{E_{\text{TM}}}$, PCP

บทที่ 10

Reducibility

10.1 Reduction และทิศของมัน

โจทย์ 10.1.1 — จริงหรือเท็จ (ให้ $A \leq B$)

1. B decidable $\Rightarrow$ A decidable
2. A decidable $\Rightarrow$ B decidable
3. A undecidable $\Rightarrow$ B undecidable
4. B undecidable $\Rightarrow$ A undecidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

อ่าน $A \leq B$ ว่า "A ไม่ยากกว่า B" (แก้ B ได้ $\Rightarrow$ แก้ A ได้)

1. **จริง** — มี decider ของ $B \rightarrow$ คำนวณ $f(w)$ แล้วถาม decider ของ $B \rightarrow$ ได้ decider ของ A (Thm 5.22)
2. **เท็จ** — A ง่ายไม่ได้บอกอะไรเกี่ยวกับ B ตัวอย่างค้าน: $A = \{0\}$ decidable และ $A \leq_m A_{\text{TM}}$ (ส่ง 0 ไปสมาชิกตายตัวของ A_{TM} ตัวอื่นไปไม่ใช่สมาชิก) แต่ A_{TM} undecidable
3. **จริง** — contrapositive ของข้อ 1: ถ้า B decidable จะได้ A decidable ขัดแย้ง (Cor 5.23) นี่คือท่าพิสูจน์ undecidable: $A = A_{\text{TM}}$
4. **เท็จ** — ตัวอย่างค้านเดียวกับข้อ 2 ($B = A_{\text{TM}}$ undecidable แต่ $A = \{0\}$ decidable)

สอนแทรก

- "ง่ายไหลลง (B $\rightarrow$ A) ยากไหลขึ้น (A $\rightarrow$ B)" — อีก 2 ข้อที่เหลือ (ไหลผิดทิศ) เท็จเสมอ
- ข้อสอบมักถามข้อ 4 หลอก — คนชอบคิดว่า "reduce ถึงกัน = ยากเท่ากัน"

10.2 สูตรพิสูจน์ undecidable แบบขัดแย้ง

โจทย์ 10.2.1 — หาจุดผิด

นักศึกษา พิสูจน์ ว่า B undecidable ดังนี้ S = “On input $\langle M, w\rangle$: 1. Run M on w. 2. If M accepts, run R on $\langle M\rangle$ ...” ผิดตรงไหน

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ผิดที่ขั้น 1** “Run M on w”
   - S ต้องเป็น **decider** ของ A_{TM} (หยุดทุก input) ถึงจะได้ข้อขัดแย้ง
   - ถ้า M loop บน $w \to S$ ติดที่ขั้น 1 ตลอดไป $\to S$ **ไม่ใช่ decider** $\to$ ไม่ขัดกับ Thm 4.11
2. **วิธีที่ถูก**
   - S แค่**สร้าง description** ของ M' ที่ “ข้างใน” มีคำสั่ง run M on w
   - การเขียน description เป็นงานจำกัดเสมอ (ไม่ได้รันจริง)
   - ส่ง $\langle M'\rangle$ ให้ R — R เป็นตัวที่ต้อง “ฉลาด” ซึ่งเราสมมติว่ามี

สอนแทรก

- ทดสอบตัวเอง: ทุกขั้นของ S ต้อง “จบแน่” — “run M” ปรากฏได้เฉพาะ**ข้างใน** M' หรือหลังจากรู้แล้วว่า M หยุด (แบบ Thm 5.1)

10.3 $HALT_{TM}$, E_{TM}, $REGULAR_{TM}$, EQ_{TM}

โจทย์ 10.3.1 — accept ε

พิสูจน์ว่า $E_\varepsilon = \{\langle M\rangle \mid M \text{ is a TM that accepts } \varepsilon\}$ undecidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ให้ M' ไม่สน input แล้วรัน M บน $w \to M'$ accept $\varepsilon \iff M$ accept w

เขียนตอบข้อสอบ (English)

Assume R decides E_ε. $S =$ "On input $\langle M, w\rangle$:

1. Construct $M' =$ 'On input x: Run M on w. If M accepts, *accept*.'
2. Run R on $\langle M'\rangle$.
3. If R accepts, *accept*; if R rejects, *reject*."

- M accepts $w \Rightarrow L(M') = \Sigma^* \Rightarrow M'$ accepts $\varepsilon \Rightarrow S$ accepts
- M does not accept $w \Rightarrow L(M') = \emptyset \Rightarrow S$ rejects

Contradiction. S only constructs M', so it always halts; S decides A_{TM}. Therefore E_ε is undecidable. ■

สอนแทรก

- M' แบบ "ไม่สน x รัน M บน w" ให้ Σ^* หรือ $\emptyset$ — ใช้ได้กับคุณสมบัติที่ Σ^* มีแต่ $\emptyset$ ไม่มี

โจทย์ 10.3.2 — $HALT_{TM}$ undecidable — Theorem 5.1 (TOC_Practice 9.5)

พิสูจน์ว่า $HALT_{TM} = \{\langle M, w\rangle \mid M \text{ หยุดบน } w\}$ undecidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ถ้ารู้ว่า M หยุดบน $w \to$ simulate ได้ปลอดภัย แล้วดูว่า accept ไหม

เขียนตอบข้อสอบ (English)

Assume R decides $HALT_{TM}$. $S =$ "On input $\langle M, w\rangle$:

1. Run R on $\langle M, w\rangle$. If R rejects, *reject*.
2. If R accepts, simulate M on w until it halts.
3. If M accepted, *accept*; if it rejected, *reject*."

Halts: stage 2 ends because R guarantees M halts on w.

Contradiction: S decides A_{TM}. So $HALT_{TM}$ is undecidable. ■

สอนแทรก

- ต่างจากข้ออื่น: ที่นี่ S **รัน** M บน w ได้ เพราะ R รับประกันแล้วว่าหยุด

10.4 ตารางออกแบบ M'

โจทย์ 10.4.1 — ภาษาจำกัด

พิสูจน์ว่า $\mathit{FINITE}_{\text{TM}} = \{\langle M \rangle \mid L(M) \text{ is finite}\}$ undecidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. เปิดตาราง: คุณสมบัติ "จำกัด" → M' ไม่สน x รัน M บน w
2. M accept $w \Rightarrow L(M') = \Sigma^*$ (อนันต์) ไม่ accept $\Rightarrow L(M') = \emptyset$ (จำกัด)
3. "จำกัด" ตรงกับ "ไม่ accept" → S ต้อง **กลับคำตอบ** ของ R

เขียนตอบข้อสอบ (English)

Assume R decides $\mathit{FINITE}_{\text{TM}}$. S = "On input $\langle M, w \rangle$:

1. Construct M' = 'On input x: Run M on w. If M accepts, *accept*.'
2. Run R on $\langle M' \rangle$.
3. If R accepts, *reject*; if R rejects, *accept*."

- M accepts $w \Rightarrow L(M') = \Sigma^*$ is infinite $\Rightarrow R$ rejects $\Rightarrow S$ accepts
- M does not accept $w \Rightarrow L(M') = \emptyset$ is finite $\Rightarrow R$ accepts $\Rightarrow S$ rejects

Contradiction. S decides A_{TM}. ■

สอนแทรก

- เช็กทิศคำตอบด้วยตาราง 2 แถว (M accept / ไม่ accept) ก่อนเขียนขั้น "If R accepts ..."

10.5 Rice's theorem

โจทย์ 10.5.1 — ใช้ Rice ได้ไหม

ภาษาใดพิสูจน์ undecidable ด้วย Rice's theorem ได้

1. $\{\langle M\rangle \mid L(M)$ มี string อย่างน้อย 2 ตัว$\}$
2. $\{\langle M\rangle \mid M$ มี state มากกว่า 10 ตัว$\}$
3. $\{\langle M\rangle \mid L(M) = \Sigma^*\}$
4. $\{\langle M\rangle \mid M$ หยุดบน ε ภายใน 100 ก้าว$\}$
5. $\{\langle M\rangle \mid L(M)$ เป็น context-free$\}$

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

Rice ใช้ได้เมื่อ (1) เป็นคุณสมบัติของ**ภาษา** $L(M)$ (2) nontrivial — มีเครื่องที่มีและไม่มีคุณสมบัตินี้

1. **ได้** — ขึ้นกับ $L(M)$ อย่างเดียว; มีเครื่องที่ $L = \Sigma^*$ (มีหลาย string) และ $L = \emptyset$ (ไม่มี) $\to$ nontrivial
2. **ไม่ได้** — "มี state > 10" เป็นคุณสมบัติของ**ตัวเครื่อง** (2 เครื่องภาษาเดียวกันมี state ต่างกันได้) และ decidable: อ่าน $\langle M\rangle$ นับ state
3. **ได้** — ขึ้นกับภาษา; มีเครื่องที่รับทุกอย่างและไม่รับอะไร
4. **ไม่ได้** — พูดถึงการทำงาน 100 ก้าวของเครื่อง ไม่ใช่ภาษา decidable: simulate 100 ก้าวแล้วดู
5. **ได้** — ขึ้นกับภาษา; $\{0^n1^n\}$ เป็น CFL, $\{0^n1^n2^n\}$ ไม่เป็น $\to$ nontrivial

สอนแทรก

- เช็ก "คุณสมบัติของภาษา" ด้วยคำถาม: "ถ้า $L(M_1) = L(M_2)$ แล้วสองเครื่องต้องตอบเหมือนกันไหม" — ถ้าไม่จำเป็น = คุณสมบัติของเครื่อง $\to$ Rice ใช้ไม่ได้
- ถ้าอาจารย์ไม่ให้อ้าง Rice ต้องเขียน reduction เต็ม (ตาราง M' หัวข้อ 10.4)

10.6 Computation history, LBA, PCP

โจทย์ 10.6.1 — LBA และ PCP

1. LBA มี 5 state, tape alphabet 3 ตัว, input ยาว 4 มี configuration ได้กี่แบบ
2. หา match ของ domino $\left[\frac{ab}{a}\right], \left[\frac{b}{bb}\right]$
3. ทำไม $\left[\frac{a}{aa}\right], \left[\frac{b}{bb}\right]$ ไม่มี match

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **จำนวน configuration ของ LBA** $= q \cdot n \cdot g^n$
   - $q = 5$ ทางเลือกของ state $n = 4$ ตำแหน่งหัว (หัวออกนอก input ไม่ได้) $g^n = 3^4 = 81$ เนื้อหา tape
   - $5 \cdot 4 \cdot 81 =$ **1620**
   - ใช้ทำอะไร: ถ้า LBA เดินเกิน 1620 ก้าว ต้องมี configuration ซ้ำ → loop แน่ → A_{LBA} decidable
2. **match**: ตัวแรกต้องขึ้นต้นตรงกัน → $[ab/a]$ (บน a, ล่าง a ✓)
   - หลังตัว 1: บน ab ล่าง a → บนเกิน b → ต้องการตัวที่ล่างขึ้นต้นด้วย b → $[b/bb]$
   - หลังตัว 2: บน abb ล่าง abb → **เท่ากัน = match (ลำดับ 1, 2)**
3. **ไม่มี match**: ทุก domino บนสั้นกว่าล่างอย่างน้อย 1 ตัว ($|a| < |aa|$, $|b| < |bb|$)
   - ต่อ l ตัว → ความยาวบนน้อยกว่าล่างอย่างน้อย $l \geq 1$ → ไม่มีวันเท่ากัน

สอนแทรก

- ข้อ 3 เป็นกรณีพิเศษที่ “ตอบได้ง่าย” — PCP undecidable หมายถึงไม่มี algorithm ที่ตอบได้**ทุก**ชุด ไม่ได้แปลว่าตอบชุดไหนไม่ได้เลย

10.7 Mapping reducibility $\leq_m$

โจทย์ 10.7.1 — Show $A_{TM} \leq_m B$

ให้ $B = \{\langle M \rangle \mid M \text{ accepts } 0\}$ แสดงว่า $A_{TM} \leq_m B$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ต้องการ $f(\langle M, w \rangle) = \langle M' \rangle$ ที่ "M accept w" $\Leftrightarrow$ "M' accept 0"
- M' ไม่สน input: รัน M บน $w \to$ ถ้า accept ก็ accept ทุกอย่าง (รวม 0)

เขียนตอบข้อสอบ (English)

$F =$ "On input $\langle M, w \rangle$:

1. Construct $M' =$ 'On input x: Run M on w. If M accepts, *accept*.'
2. Output $\langle M' \rangle$."

- **Computable:** F only constructs a description (it never runs M), so it always halts.
- **($\Rightarrow$)** M accepts $w \Rightarrow M'$ accepts every input, in particular $0 \Rightarrow \langle M' \rangle \in B$.
- **($\Leftarrow$)** M does not accept $w \Rightarrow M'$ accepts nothing $\Rightarrow \langle M' \rangle \notin B$.

Hence $\langle M, w \rangle \in A_{TM} \iff f(\langle M, w \rangle) \in B$, so $A_{TM} \leq_m B$. ■

สอนแทรก

- ผลตามมา: B undecidable (5.23) และ B ไม่ co-recognizable (เพราะ $\overline{A_{TM}} \leq_m \overline{B}$)
- mapping reduction ต้องมีครบ 3 ส่วน: F computable, ($\Rightarrow$), ($\Leftarrow$) — ขาดส่วนใดส่วนหนึ่งเสียคะแนน

โจทย์ 10.7.2 — Proof: Theorem 5.22 และ $\leq_m$ transitive

1. พิสูจน์: ถ้า $A \leq_m B$ และ B decidable แล้ว A decidable
2. พิสูจน์: ถ้า $A \leq_m B$ และ $B \leq_m C$ แล้ว $A \leq_m C$

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

1. Let M decide B and f be the reduction from A to B. $N =$ "On input w:
   1. Compute $f(w)$.
   2. Run M on $f(w)$ and output whatever M outputs."

f is computable and M is a decider, so N halts. $w \in A \iff f(w) \in B \iff M$ accepts $f(w)$, so N decides A.

2. Let f reduce A to B and g reduce B to C. Then $h = g \circ f$ is computable (compute $f(w)$, then g of it), and

$$w \in A \iff f(w) \in B \iff g(f(w)) \in C.$$

So h is a mapping reduction from A to C. ■

สอนแทรก

- transitive ใช้บ่อย: reduce จากภาษาที่รู้ว่ายากตัวไหนก็ได้ ไม่จำเป็นต้องเริ่มจาก A_{TM} (เช่น EQ_{TM} จาก E_{TM})

10.8 EQ_{TM} ไม่ recognizable ทั้งสองแบบ (Thm 5.30)

โจทย์ 10.8.1 — E_{TM} ไม่ recognizable

แสดงว่า E_{TM} ไม่ Turing-recognizable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- สูตร 9 วิธี A: complement recognizable + ตัวเอง undecidable → Thm 4.22

เขียนตอบข้อสอบ (English)

1. $\overline{E_{TM}}$ is Turing-recognizable: dovetail — for $i = 1, 2, \ldots$ run M for i steps on $s_1, \ldots, s_i$ and *accept* if any accepts.
2. E_{TM} is undecidable (Theorem 5.2).
3. If E_{TM} were Turing-recognizable, it would be both recognizable and co-recognizable, hence decidable by Theorem 4.22 — a contradiction.

Therefore E_{TM} is not Turing-recognizable. ■

สอนแทรก

- สูตร 9 วิธี A — ต้องอ้างครบ 3 ชิ้น: complement recognizable, ตัวเอง undecidable, Thm 4.22

โจทย์ 10.8.2 — EQ_{TM} undecidable (Thm 5.4)

พิสูจน์ว่า EQ_{TM} undecidable โดย reduce จาก E_{TM} (ไม่ต้องเริ่มจาก A_{TM})

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- "$L(M) = \emptyset$" = "$L(M)$ เท่ากับภาษาของเครื่องที่ reject ทุกอย่าง"

เขียนตอบข้อสอบ (English)

Assume R decides EQ_{TM}. S = "On input $\langle M \rangle$:

1. Let M_1 be a TM that rejects all inputs.
2. Run R on $\langle M, M_1 \rangle$.
3. If R accepts, *accept*; if R rejects, *reject*."

- $L(M) = \emptyset \Leftrightarrow L(M) = L(M_1) \Leftrightarrow R$ accepts.
- So S decides E_{TM}, which is undecidable (Theorem 5.2) — a contradiction.

Therefore EQ_{TM} is undecidable. ■

สอนแทรก

- reduce จาก E_{TM} ด้วยการเทียบกับเครื่องที่ reject ทุกอย่าง — "ว่าง" = "เท่ากับภาษาว่าง"

โจทย์ 10.8.3 — Proof: ALL_{TM} ไม่ Turing-recognizable

พิสูจน์ว่า $ALL_{\text{TM}} = \{\langle M \rangle \mid L(M) = \Sigma^*\}$ ไม่ Turing-recognizable โดยแสดง $\overline{A_{\text{TM}}} \leq_{\text{m}} ALL_{\text{TM}}$

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ต้องการ M' ที่ "accept ทุกอย่าง" เมื่อ M **ไม่** accept w
- ใช้ความยาวของ input เป็นตัวจำกัดเวลา: M' บน x รัน M บน w แค่ $|x|$ ก้าว

เขียนตอบข้อสอบ (English)

$F =$ "On input $\langle M, w \rangle$:

1. Construct $M' =$ 'On input x: Run M on w for $|x|$ steps. If M has accepted within these steps, *reject*; otherwise, *accept*.'
2. Output $\langle M' \rangle$."

- **Computable:** F only builds M'; and M' always halts.
- **M does not accept w** $\Rightarrow$ M' accepts every x $\Rightarrow$ $\langle M' \rangle \in ALL_{\text{TM}}$.
- **M accepts w in t steps** $\Rightarrow$ M' rejects every x with $|x| \geq t$ $\Rightarrow$ $\langle M' \rangle \notin ALL_{\text{TM}}$.

So $\overline{A_{\text{TM}}} \leq_{\text{m}} ALL_{\text{TM}}$. Since $\overline{A_{\text{TM}}}$ is not Turing-recognizable, neither is ALL_{TM} (Corollary 5.29). ■

สอนแทรก

- ทริก "รัน M บน w แค่ $|x|$ ก้าว" ทำให้ M' หยุดเสมอ และกลับด้านผลได้ — ใช้ในบทพิสูจน์ EQ_{TM} แบบอื่นด้วย

บทที่ 11

Basic Complexity: P, NP, NP-complete

11.1 Running time และ Big-O

โจทย์ 11.1.1 — วิเคราะห์เวลา M_1 ของ $\{0^k1^k\}$ (Sipser Example 7.4)

M_1 = “On input w: 1. Scan across the tape and reject if a 0 is found to the right of a 1. 2. Repeat if both 0s and 1s remain on the tape: 3. Scan across the tape, crossing off a single 0 and a single 1. 4. If 0s still remain after all the 1s have been crossed off, or if 1s still remain after all the 0s have been crossed off, reject. Otherwise, if neither 0s nor 1s remain on the tape, accept.”

1. วิเคราะห์เวลาทีละ stage แล้วสรุป Big-O
2. สรุปว่า $\{0^k1^k\}$ อยู่ใน TIME$(\cdot)$ อะไร

ลองทำเองก่อนดูแนวคิด/เฉลย

วิเคราะห์เวลา TM / algorithm: 4 ขั้น

1. แบ่ง algorithm เป็น **stage** ที่มีเลขกำกับ
2. แต่ละ stage: **หนึ่งครั้งใช้กี่ก้าว** (สแกน tape ยาว n = $O(n)$)
3. แต่ละ stage: **ทำกี่ครั้ง** (วนกี่รอบ — มักนับจากจำนวนตัวที่ขีดฆ่าต่อรอบ)
4. รวม: $\sum$(ครั้ง $\times$ ก้าวต่อครั้ง) แล้วเก็บพจน์ที่โตเร็วสุด

เฉลยละเอียด

ให้ $n = |w|$

stage	1 ครั้งใช้	ทำกี่ครั้ง	รวม
1: สแกนเช็กรูปแบบ แล้วกลับซ้าย	$2n$ ก้าว	1	$O(n)$
2–3: สแกนขีด 0 หนึ่งตัว + 1 หนึ่งตัว	$O(n)$	$\leq n/2$ รอบ (ขีด 2 ตัว/รอบ)	$O(n^2)$
4: สแกนเช็กว่าเหลือไหม	$O(n)$	1	$O(n)$

1. รวม $O(n) + O(n^2) + O(n) =$ $O(n^2)$ — เก็บพจน์ที่โตเร็วสุด
2. ดังนั้น $\{0^k1^k\} \in$ TIME(n^2)

เขียนตอบข้อสอบ (English)

Let $n = |w|$. Stage 1 scans the tape once and returns the head, using $O(n)$ steps. Each execution of stage 3 scans the tape, using $O(n)$ steps, and crosses off two symbols, so stages 2–3 repeat at most $n/2$ times, for $O(n^2)$ steps in total. Stage 4 uses $O(n)$ steps. The total is $O(n)+O(n^2)+O(n) = O(n^2)$, so $\{0^k1^k\} \in$ TIME(n^2).

สอนแทรก

- นับ “ทำกี่ครั้ง” จาก**จำนวนตัวที่ขีดฆ่าต่อรอบ**: ขีด c ตัว/รอบ $\rightarrow$ ไม่เกิน n/c รอบ = $O(n)$ รอบ
- ไม่ต้องนับก้าวเป๊ะ ๆ — Big-O ตัดค่าคงที่ (เช่น $2n$ = $O(n)$)

โจทย์ 11.1.2 — ทำให้เร็วขึ้น: $O(n \log n)$ และ 2-tape $O(n)$ (Sipser Example 7.4 ต่อ)

1. M_2 (1 tape): ทุกรอบเช็กว่าจำนวน 0 กับ 1 ที่เหลือ **คู่/คี่ตรงกันไหม** แล้ว**ขีด 0 ทุกตัวเว้นตัว และ 1 ทุกตัวเว้นตัว** วิเคราะห์เวลา และอธิบายว่าทำไมถูก
2. M_3 (2 tape): คัดลอก 0 ไป tape 2 แล้วจับคู่กับ 1 วิเคราะห์เวลา
3. สรุป: ทำไมเวลาขึ้นกับโมเดล แต่ decidability ไม่ขึ้น

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. M_2: $O(n \log n)$
   - 1 รอบ = สแกน 1–2 ครั้ง $\rightarrow O(n)$
   - ขีดทุกตัวเว้นตัว $\rightarrow$ จำนวนที่เหลือ**ลดครึ่ง**ทุกรอบ $\rightarrow$ จาก n เหลือ 0 ใช้ $1 + \log_2 n$ รอบ = $O(\log n)$
   - รวม $O(n) \times O(\log n) = O(n \log n)$
   - **ทำไมถูก**: parity ของจำนวนที่เหลือในแต่ละรอบ = บิตของจำนวนในฐาน 2 (จากบิตต่ำไปสูง) $\rightarrow$ ถ้า parity ตรงกันทุกรอบ แปลว่าเลขฐาน 2 ของ #0 กับ #1 เท่ากันทุกบิต $\rightarrow$ #0 = #1
2. M_3 (2 tape): $O(n)$
   - สแกน tape 1: คัดลอก 0 ทุกตัวไป tape 2 ($O(n)$)
   - สแกน 1 บน tape 1 พร้อมขีด 0 บน tape 2 ทีละตัว ($O(n)$)
   - เช็กว่าหมดพร้อมกัน $\rightarrow$ รวม $O(n)$ — สแกนทางเดียว ไม่ต้องวิ่งไปกลับ
3. สรุป
   - เวลาขึ้นกับโมเดล: 1 tape $O(n \log n)$ 2 tape $O(n)$ (และพิสูจน์ได้ว่า 1 tape ทำเร็วกว่า $o(n \log n)$ ไม่ได้สำหรับภาษาที่ไม่ regular)
   - decidability ไม่ขึ้นกับโมเดล (ทุก variant เท่ากับ TM) แต่เวลาต่างกันได้ถึงกำลังสอง (Thm 7.8)

สอนแทรก

- "ลดครึ่งทุกรอบ" = $O(\log n)$ รอบ — รูปแบบเดียวกับ binary search และ Euclid (RELPRIME)

โจทย์ 11.1.3 — วิเคราะห์เวลา TM ของ $\{a^n b^n c^n\}$ และ palindrome

1. TM ของ $\{a^n b^n c^n\}$ (โจทย์ 5.6): รอบละขีด a, b, c อย่างละตัวแล้วเดินกลับ วิเคราะห์เวลา
2. TM ของ palindrome (โจทย์ 5.6): รอบละลบตัวซ้ายสุดกับขวาสุด วิเคราะห์เวลา
3. ถ้าใช้ 2 tape ทำ palindrome ได้เร็วแค่ไหน

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. $a^n b^n c^n$
   - 1 รอบ: วิ่งขวาหา b แล้ว c แล้ววิ่งกลับหา $x \to$ ไม่เกิน $2n$ ก้าว = $O(n)$
   - ขีด 3 ตัว/รอบ $\to$ ไม่เกิน $n/3$ รอบ = $O(n)$ รอบ
   - รอบสุดท้าย (q_4 เช็ก y, z): $O(n)$
   - รวม $O(n) \cdot O(n) + O(n) = O(n^2)$
2. palindrome (1 tape)
   - 1 รอบ: วิ่งไปขวาสุด + วิ่งกลับซ้ายสุด $\to O(n)$
   - ลบ 2 ตัว/รอบ $\to$ ไม่เกิน $\lceil n/2 \rceil + 1$ รอบ
   - รวม $O(n^2)$
3. palindrome (2 tape)
   - คัดลอก input ไป tape 2 ($O(n)$) $\to$ เลื่อนหัว tape 1 กลับซ้ายสุด ($O(n)$)
   - อ่าน tape 1 จากซ้ายไปขวา พร้อม tape 2 จากขวาไปซ้าย เทียบทีละตัว ($O(n)$)
   - รวม $O(n)$

เขียนตอบข้อสอบ (English) — TM ของ $\{a^n b^n c^n\}$

Let $n = |w|$. Each pass crosses off one a, one b and one c and moves the head across the tape and back, using $O(n)$ steps. There are at most $n/3$ passes, for $O(n^2)$ steps. The final check of the remaining y's and z's takes $O(n)$ steps. Hence the TM runs in time $O(n^2)$, and $\{a^n b^n c^n\} \in \text{TIME}(n^2) \subseteq \text{P}$.

สอนแทรก

- TM แบบ "ขีดฆ่าแล้ววิ่งไปกลับ" บน 1 tape เกือบทุกตัวได้ $O(n^2)$ — สูตรจำ: (จำนวนรอบ $O(n)$) $\times$ (ความยาวรอบ $O(n)$)
- ข้อนี้แสดงว่า $\{a^n b^n c^n\} \in$ P แม้จะไม่ใช่ CFL — P ใหญ่กว่า CFL

โจทย์ 11.1.4 — วิเคราะห์เวลา M_2 ของ $\{0^{2^n}\}$ (Sipser Example 3.7)

M_2 = “1. Sweep left to right across the tape, crossing off every other 0. 2. If in stage 1 the tape contained a single 0, accept. 3. If in stage 1 the tape contained more than a single 0 and the number of 0s was odd, reject. 4. Return the head to the left-hand end of the tape. 5. Go to stage 1.” วิเคราะห์เวลา

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

stage	1 ครั้งใช้	ทำกี่ครั้ง	เหตุผล
1 (sweep ขีดเว้นตัว)	$O(n)$	$\le 1+\log_2 n$	0 ที่เหลือลดครึ่งทุกรอบ
2–3 (เช็ก)	$O(1)$ (จำใน state ระหว่าง sweep)	$\le 1+\log_2 n$	ทำรอบละครั้ง
4 (กลับซ้ายสุด)	$O(n)$	$\le 1+\log_2 n$	รอบละครั้ง

- รวม $O(\log n)\times O(n) = O(n\log n)$
- หยุดเสมอ: จำนวน 0 ลดครึ่งทุกรอบ $\rightarrow$ ไม่เกิน $\log_2 n+1$ รอบก็เหลือ 1 ตัวหรือเจอเลขคี่

สอนแทรก

- เทคนิคเดียวกับ M_2 ของ 0^k1^k: “ขีดทุกตัวเว้นตัว” = หาร 2 ทุกรอบ = $O(\log n)$ รอบ

โจทย์ 11.1.5 — ข้อเสริม: Big-O / small-o (แนว Sipser Exercise 7.1, 7.2)

จริงหรือเท็จ

1. $3n^4 + 2n\log n + 7 = O(n^4)$
2. $n\log n = o(n^2)$
3. $2^n = O(n^{10})$
4. $n^2 = o(n^2)$
5. $\log_2 n = O(\log_{10} n)$
6. $n^{100} = O(2^n)$

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

นิยาม: $f = O(g) \Leftrightarrow$ มี c, n_0 ที่ $f(n) \le c\,g(n)$ ทุก $n \ge n_0$ $\quad f = o(g) \Leftrightarrow \lim f/g = 0$

1. **จริง** — $3n^4 + 2n\log n + 7 \le 3n^4 + 2n^4 + 7n^4 = 12n^4$ ทุก $n \ge 1 \to c = 12,\ n_0 = 1$
2. **จริง** — $\dfrac{n\log n}{n^2} = \dfrac{\log n}{n} \to 0$
3. **เท็จ** — $\dfrac{2^n}{n^{10}} \to \infty$ ไม่มี c ใดคุมได้ (exponential โตเร็วกว่า polynomial)
4. **เท็จ** — $\dfrac{n^2}{n^2} = 1 \not\to 0$ $\quad (n^2 = O(n^2)$ จริง แต่ o ต้อง “โตช้ากว่าอย่างแท้จริง”)
5. **จริง** — $\log_2 n = \log_{10} n \cdot \log_2 10 \approx 3.32\log_{10} n$ ต่างแค่ค่าคงที่
6. **จริง** — $\dfrac{n^{100}}{2^n} \to 0$ (แม้ช่วงต้น n^{100} ใหญ่กว่ามาก)

สอนแทรก

- O = “ไม่โตเร็วกว่า” (เหมือน $\le$) $\quad o$ = “โตช้ากว่าแท้จริง” (เหมือน $<$)
- ลำดับความเร็วโต: $\log n \ll n \ll n\log n \ll n^2 \ll n^k \ll 2^n$

11.2 เวลาข้ามโมเดล

โจทย์ 11.2.1 — แปลงเวลา

1. 3-tape TM ใช้เวลา $O(n^2)$ → single-tape ใช้เท่าไร
2. NTM ใช้เวลา $O(n)$ → deterministic TM ใช้เท่าไร
3. ข้อไหนทำให้ภาษายังอยู่ใน P แน่นอน

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. multitape → single-tape (Thm 7.8): $t(n) \to O(t^2(n))$
   - tape ของเครื่องเดียวยาว $O(t(n))$ จำลอง 1 ก้าวใช้ $O(t(n))$ $t(n)$ ก้าว → $O(t^2(n))$
   - $t(n) = n^2 \to O(n^4)$
2. NTM → DTM (Thm 7.11): $t(n) \to 2^{O(t(n))}$
   - tree สูง $t(n)$ แต่ละ node ลูกไม่เกิน b → ใบไม่เกิน $b^{t(n)}$ → ลองทุกกิ่ง
   - $t(n) = n \to 2^{O(n)}$
3. อยู่ใน P แน่: **ข้อ 1** — $O(n^4)$ เป็น polynomial
   - ข้อ 2 สรุปได้แค่ว่าอยู่ใน NP (NTM polynomial) — จำลองด้วยวิธีที่รู้ได้ exponential ซึ่งไม่ใช่ P

สอนแทรก

- นี่ คือ เหตุผล ที่ P “robust”: เปลี่ยน โมเดล deterministic (1 tape / หลาย tape / RAM) ต่าง แค่ polynomial
- ช่องว่าง polynomial vs exponential ของ NTM → DTM = ที่มาของคำถาม P vs NP

11.3 คลาส P

โจทย์ 11.3.1 — Triangle $\in$ P

แสดงว่า *TRIANGLE* $= \{\langle G\rangle \mid G$ เป็นกราฟไม่มีทิศที่มี 3-clique$\} \in$ P

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ลองทุก 3 node: มี $\binom{m}{3} = O(m^3)$ ชุด $\rightarrow$ polynomial (ต่างจากลองทุก subset ที่เป็น 2^m)

เขียนตอบข้อสอบ (English)

$M =$ "On input $\langle G\rangle$, where G has m nodes:
1. For each triple of distinct nodes u, v, w of G:
2. If (u, v), (v, w) and (u, w) are all edges of G, *accept*.
3. *Reject*."

- Stage 1 loops over $\binom{m}{3} = O(m^3)$ triples.
- Stage 2 checks 3 edges, each in polynomial time.
- Total time is polynomial in $|\langle G\rangle|$, so *TRIANGLE* $\in$ P.

! จุดที่คนเสียคะแนนบ่อย

- "ลอง หาร ทุก ตัว ถึง x" ของ RELPRIME ดู เหมือน poly แต่ x ใหญ่ได้ถึง $2^n \rightarrow$ exponential นับ เวลา เป็น ฟังก์ชันของ**ความยาว input**

สอนแทรก

- ลองทุก k-subset เมื่อ k **คงที่** = poly ($O(m^k)$) แต่ถ้า k เป็น input (CLIQUE) = exponential

โจทย์ 11.3.2 — Proof: P ปิดภายใต้ union และ complement

ให้ $L_1, L_2 \in \mathsf{P}$ พิสูจน์ว่า $L_1 \cup L_2 \in \mathsf{P}$ และ $\overline{L_1} \in \mathsf{P}$

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

Let M_1, M_2 decide L_1, L_2 in time $O(n^k)$ and $O(n^l)$.

- Union: M = "On input w: Run M_1 on w; if it accepts, *accept*. Run M_2 on w; if it accepts, *accept*; otherwise, *reject*." Time $O(n^k) + O(n^l) = O(n^{\max(k,l)})$.
- Complement: M = "On input w: Run M_1 on w; if it accepts, *reject*; otherwise, *accept*." Time $O(n^k)$.

Both are polynomial time deciders, so $L_1 \cup L_2 \in \mathsf{P}$ and $\overline{L_1} \in \mathsf{P}$. ■

สอนแทรก

- poly + poly = poly poly × poly = poly poly ของ poly = poly — 3 ข้อ นี้ พอ สำหรับ ทุก บท พิสูจน์ "in P"

11.4 คลาส NP

โจทย์ 11.4.1 — SUBSET-SUM $\in$ NP

แสดงว่า *SUBSET-SUM* $= \{\langle S, t\rangle \mid S = \{x_1, \ldots, x_k\}$ มี subset ที่ผลรวม $= t\} \in$ NP และอธิบายว่าทำไมยังไม่รู้ว่า $\overline{\textit{SUBSET-SUM}} \in$ NP

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

The certificate is a subset $c = \{y_1, \ldots, y_l\}$. $V =$ "On input $\langle\langle S, t\rangle, c\rangle$:

1. Test whether c is a collection of numbers that sum to t.
2. Test whether S contains all the numbers in c.
3. If both pass, *accept*; otherwise, *reject*."

- $|c| \leq |S|$, and each test is polynomial, so V runs in polynomial time.
- $\langle S, t\rangle \in$ *SUBSET-SUM* iff some c makes V accept.

เฉลยส่วน complement

- $\overline{\textit{SUBSET-SUM}}$ = "**ไม่มี** subset ไหนรวมได้ t"
- ยังไม่มีใครรู้ certificate สั้น ๆ ที่พิสูจน์ "ไม่มี" ได้ (ต้องตรวจทุก 2^k subset)
- จึงยังไม่รู้ว่า NP $=$ coNP ไหม

สอบแทรก

- บทพิสูจน์ "in NP" ต้องบอก: certificate คืออะไร, ยาว poly, ตรวจได้ใน poly, และ $w \in L \Leftrightarrow$ มี certificate

โจทย์ 11.4.2 — Proof: $P \subseteq NP$ และ NP ปิดภายใต้ union

1. พิสูจน์ว่า $P \subseteq NP$
2. พิสูจน์ว่าถ้า $A, B \in NP$ แล้ว $A \cup B \in NP$

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

1. Let $A \in P$ with polynomial time decider M. Define V = "On input $\langle w, c \rangle$: Run M on w and output its answer (ignore c)." V runs in polynomial time and $w \in A$ iff V accepts $\langle w, c \rangle$ for some c (e.g. $c = \varepsilon$). So $A \in NP$.
2. Let N_A, N_B be polynomial time NTMs for A, B. N = "On input w: Nondeterministically choose to run N_A or N_B on w, and accept if it accepts." Each branch runs in polynomial time, and some branch accepts iff $w \in A \cup B$. So $A \cup B \in NP$. ■

สอนแทรก

- verifier "ไม่สน certificate" คือเหตุผลมาตรฐานของ $P \subseteq NP$

11.5 $\leq_P$ และ *3SAT* $\leq_P$ *CLIQUE*

โจทย์ 11.5.1 — สร้างกราฟ

ให้ $\phi = (x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3)$

1. สร้างกราฟ G ตาม reduction และบอก k
2. หา k-clique แล้วแปลงเป็นการกำหนดค่าที่ทำให้ ϕ จริง

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

1. node: triple 1 = $\{x_1, x_2, \overline{x_3}\}$, triple 2 = $\{\overline{x_1}, x_2, x_3\}$, $k =$ **2** (จำนวน clause)
2. edge: เชื่อมทุกคู่**ข้าม triple** ยกเว้นคู่ที่ขัดกัน
   - คู่ข้าม triple มี $3 \times 3 = 9$ คู่
   - ตัดคู่ขัด x_1–$\overline{x_1}$ และ $\overline{x_3}$–x_3
   - เหลือ **7 edge**: x_1x_2, x_1x_3, $x_2\overline{x_1}$, x_2x_2, x_2x_3, $\overline{x_3}\overline{x_1}$, $\overline{x_3}x_2$
3. 2-clique เช่น $\{x_2$ (triple 1), x_2 (triple 2)$\}$
   - ตั้ง $x_2 =$ TRUE (ตัวอื่นอะไรก็ได้) $\rightarrow$ ทั้ง 2 clause จริง ✓

สอนแทรก

- ตรวจ reduction ด้วยตัวอย่างเล็ก ๆ แบบนี้ก่อนเขียนบทพิสูจน์ทั่วไป — เห็นชัดว่าทำไม 2 ทิศถูก

11.6 NP-completeness

โจทย์ 11.6.1 — INDEPENDENT-SET เป็น NP-complete

$IS = \{\langle G, k\rangle \mid G$ มี independent set ขนาด $k\}$ (เซต node ที่ไม่มี edge ระหว่าง กัน เลย) พิสูจน์ ว่า NP-complete โดยใช้ว่า CLIQUE NP-complete

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- clique ใน G = independent set ใน**กราฟ complement** $\overline{G}$ (สลับมี edge / ไม่มี edge)

เขียนตอบข้อสอบ (English)

Part 1: $IS \in$ NP.
- The certificate is a set c of k nodes.
- V checks that c has k nodes of G and that no edge of G joins two nodes of c.
- Both checks take polynomial time.

Part 2: $CLIQUE \leq_P IS$.
- Construction: $f(\langle G, k\rangle) = \langle \overline{G}, k\rangle$, where $\overline{G}$ has the same nodes and an edge exactly where G has none.
- Polynomial time: $\overline{G}$ has at most m^2 edges to write.
- ($\Rightarrow$) If C is a k-clique in G, every pair in C is an edge of G, so no pair is an edge of $\overline{G}$; C is an independent set in $\overline{G}$.
- ($\Leftarrow$) If C is an independent set of size k in $\overline{G}$, every pair in C is a non-edge of $\overline{G}$, so an edge of G; C is a k-clique in G.

Conclusion. IS is NP-complete by Theorem 7.36. ■

! จุดที่คนเสียคะแนนบ่อย

- reduce $IS \leq_P CLIQUE$ แทน $\rightarrow$ พิสูจน์ผิดทิศ (ได้แค่ IS ไม่ยากกว่า CLIQUE)
- ลืม Part 1 $\rightarrow$ ได้แค่ NP-hard

สอนแทรก

- NP-complete = 2 ส่วนเสมอ: in NP + NP-hard NP-hard ต้อง reduce **จาก** NPC ที่รู้แล้ว **ไป** ภาษาใหม่

โจทย์ 11.6.2 — Proof: Theorems 7.35 และ 7.36

1. ถ้า B NP-complete และ $B \in$ P แล้ว P $=$ NP
2. ถ้า B NP-complete, $B \leq_{\text{P}} C$ และ $C \in$ NP แล้ว C NP-complete

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ข้อ 1 ใช้ Thm 7.31 ($A \leq_{\text{P}} B$, $B \in$ P $\Rightarrow A \in$ P)
- ข้อ 2 ใช้ว่า $\leq_{\text{P}}$ transitive: ประกอบ poly กับ poly ได้ poly

เขียนตอบข้อสอบ (English)

1. P $\subseteq$ NP always. Let $A \in$ NP. Since B is NP-complete, $A \leq_{\text{P}} B$; since $B \in$ P, Theorem 7.31 gives $A \in$ P. So NP $\subseteq$ P, and P $=$ NP.
2. $C \in$ NP is given. Let $A \in$ NP; then $A \leq_{\text{P}} B$ by f and $B \leq_{\text{P}} C$ by g. The composition $g \circ f$ is computable in polynomial time: $|f(w)|$ is polynomial in $|w|$, and g runs in time polynomial in $|f(w)|$. Also $w \in A \iff f(w) \in B \iff g(f(w)) \in C$. So $A \leq_{\text{P}} C$ for every $A \in$ NP, and C is NP-complete. ■

สอนแทรก

- 7.36 คือเหตุผลที่พิสูจน์ NP-complete ด้วย reduction **ตัวเดียว**ได้ ไม่ต้อง reduce ทุกภาษาใน NP

บทที่ 12

เห็นโจทย์แล้วใช้สูตรไหน

12.1 ตารางเลือกสูตร

โจทย์ 12.1.1 — เลือกสูตร

แต่ละโจทย์ใช้วิธีไหน (ตอบสั้น ๆ)

1. Show that $\{a^n b^n c^n d^n\}$ is not context free.
2. Show that $\{\langle A\rangle \mid A \text{ is a DFA and } L(A) = \Sigma^*\}$ is decidable.
3. Show that $\{\langle M\rangle \mid L(M) \text{ is infinite}\}$ is undecidable.
4. Show that $\overline{E_{\text{TM}}}$ is Turing-recognizable.
5. Show that *VERTEX-COVER* $\in$ NP.
6. Show that a TM with a doubly infinite tape is equivalent to a standard TM.
7. Show that EQ_{TM} is not Turing-recognizable.

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **สูตร 1 (pumping)** — $s = a^p b^p c^p d^p$; vxy แตะได้ไม่เกิน 2 บล็อกติดกัน → อีก 2 บล็อกยาวเท่าเดิม
2. **สูตร 4 (decidable)** — สลับ accept state ได้ $\overline{L(A)}$ แล้วรัน decider E_{DFA}
3. **สูตร 7 (undecidable)** — M' ไม่สน x รัน M บน w: accept ⇒ $L(M') = \Sigma^*$ (อนันต์) / ไม่ ⇒ $\emptyset$ (จำกัด) → ตอบตาม R
4. **สูตร 5 (recognizable)** — dovetail: รอบ i รัน M บน $s_1..s_i$ คนละ i ก้าว accept ถ้ามีตัวไหน accept
5. **สูตร 11 (NP)** — certificate = เซต k node; ตรวจว่าทุก edge แตะเซต (poly)
6. **สูตร 3 (variant)** — ($\Rightarrow$) TM ธรรมดาเป็นกรณีพิเศษ ($\Leftarrow$) จำลองด้วย 2 tape แล้วใช้ Thm 3.13
7. **สูตร 9 (not recognizable)** — $A_{\text{TM}} \leq_{\text{m}} \overline{EQ_{\text{TM}}}$ (M_1 reject ทุกอย่าง, M_2 รัน M บน w) → $\overline{A_{\text{TM}}} \leq_{\text{m}} EQ_{\text{TM}}$

สอนแทรก

- อ่าน คำ กริยา ใน โจทย์ ก่อน: “not context free” / “decidable” / “undecidable” / “recognizable” / “in NP” / “equivalent” / “not recognizable” → เปิดสูตรได้ทันที

12.2 สูตร 1–3: not CFL, TM, variant

โจทย์ 12.2.1 — เติมสูตร 1

ใช้สูตร 1 พิสูจน์ว่า $\{a^n b^n c^n d^n \mid n \geq 0\}$ ไม่ใช่ context free

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

Setup. Assume L is context free with pumping length p. Let $s = a^p b^p c^p d^p \in L$, so $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$. Because $|vxy| \leq p$, vxy touches at most two adjacent blocks.

- Case 1: v or y contains two kinds of symbols.
  - Then uv^2xy^2z has symbols out of order, so it is not in L.
- Case 2: v and y each contain one kind of symbol.
  - At most two of the four blocks grow in uv^2xy^2z; at least two stay of length p.
  - So the block lengths are unequal and $uv^2xy^2z \notin L$.

Conclusion. Every case contradicts the pumping lemma. Therefore L is not context free. ■

สอนแทรก

- ภาษา k บล็อกเท่ากัน: vxy แตะได้ไม่เกิน 2 บล็อกติดกัน → บล็อกที่เหลือยาว p เท่าเดิม — ใช้ได้ทุก $k \geq 3$

12.3 สูตร 4–6: decidable, recognizable, closure

โจทย์ 12.3.1 — DFA ∩ CFG

แสดงว่า $\{\langle A, G\rangle \mid A$ เป็น DFA, G เป็น CFG, $L(A) \cap L(G) = \emptyset\}$ decidable

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- CFL ∩ regular = CFL → สร้าง CFG ของ $L(A) \cap L(G)$ ได้ แล้วถาม E_{CFG}

เขียนตอบข้อสอบ (English)

$M =$ "On input $\langle A, G\rangle$:
1. Convert G to a PDA P; build the product PDA P' of P and A, so $L(P') = L(A) \cap L(G)$.
2. Convert P' to an equivalent CFG G'.
3. Run the decider R for E_{CFG} on $\langle G'\rangle$. If it accepts, *accept*; otherwise, *reject*."

- Halting: every construction is finite and R is a decider.
- Correctness: $L(A) \cap L(G) = \emptyset$ iff $L(G') = \emptyset$ iff R accepts.

สอบแทรก

- ท่า "สร้างเครื่องใหม่ด้วย closure แล้วถาม emptiness" ใช้ได้ทั้งกับ DFA และ CFG — ขอแค่ operation นั้นปิด

12.4 สูตร 7–9: undecidable, $\leq_m$, not recognizable

โจทย์ 12.4.1 — ALL_{TM}

ใช้สูตร 7 พิสูจน์ว่า $ALL_{TM} = \{\langle M \rangle \mid L(M) = \Sigma^*\}$ undecidable

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

Assume R decides ALL_{TM}. S = "On input $\langle M, w \rangle$:

1. Construct M' = 'On input x: Run M on w. If M accepts, *accept*.'
2. Run R on $\langle M' \rangle$.
3. If R accepts, *accept*; if R rejects, *reject*."

- M accepts $w \Rightarrow L(M') = \Sigma^* \Rightarrow S$ accepts
- M does not accept $w \Rightarrow L(M') = \emptyset \neq \Sigma^* \Rightarrow S$ rejects

Contradiction. S always halts and decides A_{TM}. Therefore ALL_{TM} is undecidable. ■

สอนแทรก

- ข้อนี้ undecidable และไม่ recognizable ด้วย (ดูโจทย์ 10.8) — แต่ถ้าโจทย์ถามแค่ undecidable ใช้สูตร 7 พอ

โจทย์ 12.4.2 — Proof: $REGULAR_{TM}$ undecidable (Theorem 5.3)

ใช้สูตร 7 เขียนบทพิสูจน์เต็มโดยไม่ดูเล่มสอน

ลองทำเองก่อนดูแนวคิด/เฉลย

แนวคิด

- ตาราง M': คุณสมบัติ "regular" $\to x = 0^n1^n$ accept ไม่งั้นรัน M บน $w \to \Sigma^* \,/\, \{0^n1^n\}$

เขียนตอบข้อสอบ (English)

Assume R decides $REGULAR_{TM}$. S = "On input $\langle M, w\rangle$:

1. Construct M_2 = 'On input x: If x has the form 0^n1^n, *accept*. Otherwise run M on w and *accept* if M accepts.'
2. Run R on $\langle M_2\rangle$.
3. If R accepts, *accept*; if R rejects, *reject*."

- M accepts $w \Rightarrow L(M_2) = \Sigma^*$, regular $\Rightarrow S$ accepts
- M does not accept $w \Rightarrow L(M_2) = \{0^n1^n\}$, not regular $\Rightarrow S$ rejects

Contradiction. S always halts and decides A_{TM}. ■

สอนแทรก

- เลือกภาษาไม่ regular ที่ "ตรวจง่าย" (0^n1^n) ใส่ใน M_2 — ใช้แบบเดียวกันกับ "CFL" (ใส่ $0^n1^n2^n$)

12.5 สูตร 10–11: P, NP

โจทย์ 12.5.1 — COMPOSITES $\in$ NP

ใช้สูตร 11 แสดงว่า $\mathit{COMPOSITES} = \{\langle x\rangle \mid x = pq,\ p, q > 1\} \in \mathrm{NP}$

ลองทำเองก่อนดูแนวคิด/เฉลย

เขียนตอบข้อสอบ (English)

The certificate c is a number p, whose length is at most $|\langle x\rangle|$. V = "On input $\langle x, c\rangle$:

1. Test whether $1 < c < x$.
2. Test whether c divides x (long division in binary).
3. If both tests pass, *accept*; otherwise, *reject*."

- Each test runs in polynomial time in the number of bits.
- x is composite $\Rightarrow$ a factor c makes V accept; x is not composite $\Rightarrow$ no c does.

Hence $\mathit{COMPOSITES} \in \mathrm{NP}$.

สอนแทรก

- วัดเวลาเป็นฟังก์ชันของ**จำนวนบิต** — การหารเลขฐาน 2 แบบ long division ใช้เวลา poly ในจำนวนบิต

12.6 True / False ที่ออกบ่อย

โจทย์ 12.6.1 — T/F ชุดใหม่

1. ทุก regular เป็น decidable
2. complement ของ CFL เป็น decidable เสมอ
3. ถ้า $A \leq_{\text{m}} B$ และ B CFL แล้ว A CFL
4. $HALT_{\text{TM}}$ เป็น co-recognizable
5. ถ้า L และ $\overline{L}$ เป็น CFL แล้ว L decidable
6. ทุกภาษาจำกัดเป็น decidable
7. ถ้า P $=$ NP แล้ว $SAT \in$ P
8. NP-hard ทุกตัวอยู่ใน NP

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. T — regular $\subseteq$ CFL $\subseteq$ decidable (DFA หยุดเสมอ)
2. T — L CFL $\Rightarrow$ L decidable (Thm 4.9) $\Rightarrow$ $\overline{L}$ decidable (decidable ปิด complement) ($\overline{L}$ อาจไม่ใช่ CFL แต่ decidable แน่)
3. F — f เป็น computable function ใด ๆ ซึ่ง แรง กว่า PDA มาก $\rightarrow A$ อาจ ยาก กว่า CFL ตัวอย่าง ค้าน: $\{a^n b^n c^n\} \leq_{\text{m}} \{0\}$ โดย $f(w) = 0$ ถ้า $w \in \{a^n b^n c^n\}$ ไม่งั้น $f(w) = 1$ (f computable เพราะ $\{a^n b^n c^n\}$ decidable) — $\{0\}$ เป็น CFL แต่ $\{a^n b^n c^n\}$ ไม่ใช่
4. F — $HALT_{\text{TM}}$ recognizable อยู่แล้ว ถ้า co-recognizable ด้วยจะ decidable (Thm 4.22) ขัดกับ Thm 5.1
5. T — ไม่ต้องใช้ข้อ $\overline{L}$ เลย: CFL ทุกตัว decidable อยู่แล้ว
6. T — ภาษาจำกัดเขียนเป็น regex (union ของ string) ได้ $\Rightarrow$ regular $\Rightarrow$ decidable
7. T — $SAT \in$ NP และ NP $=$ P $\Rightarrow$ $SAT \in$ P
8. F — NP-hard = “ทุกภาษาใน NP reduce มาได้” ไม่ได้บังคับให้อยู่ใน NP เช่น $HALT_{\text{TM}}$ เป็น NP-hard แต่ undecidable (ไม่อยู่ใน NP)

สอนแทรก

- ข้อ T/F ตอบ “F” ต้องมี**ตัวอย่างค้าน** ตอบ “T” ต้องอ้าง**ทฤษฎีบท** — คะแนนส่วนใหญ่อยู่ที่เหตุผล

ตารางสรุป & จุดพลาดก่อนเข้าห้องสอบ

13.1 ปัญหามาตรฐาน: decidable? recognizable?

โจทย์ 13.1.1 — เติมตาราง

เติม ใช่ / ไม่ ในตาราง

ภาษา	decidable	recognizable	co-recognizable
E_{CFG}			
A_{TM}			
$\overline{A_{\text{TM}}}$			
E_{TM}			
EQ_{TM}			
A_{LBA}			
ALL_{CFG}			

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยทีละขั้น

ภาษา	dec	rec	co-rec	อ้างอิง
E_{CFG}	ใช่	ใช่	ใช่	Thm 4.8
A_{TM}	ไม่	ใช่	ไม่	4.11, 4.23
$\overline{A_{\text{TM}}}$	ไม่	ไม่	ใช่	4.23
E_{TM}	ไม่	ไม่	ใช่	5.2 + dovetail
EQ_{TM}	ไม่	ไม่	ไม่	5.30
A_{LBA}	ใช่	ใช่	ใช่	5.9
ALL_{CFG}	ไม่	ไม่	ใช่	5.13

- กฎตรวจ: ถ้า rec **และ** co-rec ต้อง decidable (4.22) → ห้ามมีแถว “ไม่, ใช่, ใช่”

สอนแทรก

- ใช้ตารางนี้ทวนก่อนสอบ — ถ้าจำแถวไหนไม่ได้ให้ย้อนดูทฤษฎีบทที่อ้างในคอลัมน์ขวา

13.2 Closure ทุกชั้น

โจทย์ 13.2.1 — ปิดหรือไม่

1. CFL ภายใต้ star
2. CFL ภายใต้ complement
3. Recognizable ภายใต้ intersection
4. Recognizable ภายใต้ complement
5. Decidable ภายใต้ complement
6. CFL ∩ regular

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **CFL ปิด star** — CFG ใหม่ $S \to SS_1 \mid \varepsilon$ สร้าง L^*
2. **CFL ไม่ปิด complement** — ถ้าปิด จะได้ $A \cap B = \overline{\overline{A} \cup \overline{B}}$ เป็น CFL เสมอ แต่ $\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$ ไม่ใช่
3. **Recognizable ปิด ∩** — รัน M_1 ถ้า accept รัน M_2 ถ้า accept → accept (ถ้า ตัว ใด loop ก็ ไม่ accept อยู่แล้ว ถูกต้อง)
4. **Recognizable ไม่ปิด complement** — A_{TM} recognizable แต่ $\overline{A_{\text{TM}}}$ ไม่ (Cor 4.23)
5. **Decidable ปิด complement** — สลับ accept/reject ได้เพราะ decider หยุดเสมอ
6. **CFL ∩ regular = CFL เสมอ** — product construction: PDA × DFA ใช้ stack เดียว

สอนแทรก

- "ไม่ปิด" ต้องตอบพร้อม**ตัวอย่างค้าน 1 คู่** "ปิด" ต้องตอบพร้อม**วิธีสร้าง**ที่ใช้ได้ทุกคู่

13.3 Reduction ส่งต่ออะไร

โจทย์ 13.3.1 — สรุปผล

ให้ $A \leq_m B$ สรุปอะไรได้

1. B recognizable
2. A ไม่ recognizable
3. A decidable
4. $\overline{A} \leq_m$?

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. B recognizable $\Rightarrow$ A recognizable (Thm 5.28): recognizer ของ A = "คำนวณ $f(w)$ แล้วรัน recognizer ของ B บน $f(w)$"
2. A ไม่ recognizable $\Rightarrow$ B ไม่ recognizable (Cor 5.29): contrapositive ของข้อ 1
3. A decidable $\Rightarrow$ **สรุปอะไรเกี่ยวกับ B ไม่ได้** — ข้อมูลไหลผิดทิศ (A ง่ายไม่ทำให้ B ง่ายหรือยาก)
4. $\overline{A} \leq_m \overline{B}$ — ใช้ f ตัวเดิม: $w \in A \iff f(w) \in B \Rightarrow w \notin A \iff f(w) \notin B$

สอนแทรก

- ข้อ 4 คือท่าที่ใช้ใน Thm 5.30: แสดง $A_{\text{TM}} \leq_m \overline{EQ_{\text{TM}}}$ = แสดง $\overline{A_{\text{TM}}} \leq_m EQ_{\text{TM}} \rightarrow EQ_{\text{TM}}$ ไม่ recognizable

13.4 ตัวเลขที่ต้องจำ

โจทย์ 13.4.1 — คำนวณ

1. CNF สร้าง string ยาว 7 ใช้กี่ขั้น
2. ลบ $A \to \varepsilon$ จากกฎ $R \to AbAcA$ ต้องเพิ่มกี่กฎ
3. PDA มี 4 state $\to$ CFG มีตัวแปร, กฎแบบ 2, กฎแบบ 3 อย่างละเท่าไร
4. grammar มี $|V| = 4$, ด้านขวายาวสุด 3 $\to$ pumping length ตามบทพิสูจน์
5. decidable concatenation บน $|w| = 5$ ต้องลองกี่จุดตัด

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **CNF ยาว 7**: $A \to BC$ ใช้ $7 - 1 = 6$ ครั้ง + $A \to a$ ใช้ 7 ครั้ง = $2 \cdot 7 - 1 =$ **13 ขั้น**
2. **ลบ $A \to \varepsilon$ จาก $R \to AbAcA$**: A มี 3 ตัว แต่ละตัวอยู่/หาย $\to 2^3 = 8$ แบบ ลบแบบเดิม 1 $\to$ **7 กฎใหม่**
   - $bAcA$, $AbcA$, $AbAc$, bcA, bAc, Abc, bc
3. **PDA 4 state**: ตัวแปร A_{pq} = $4^2 =$ **16** แบบ 2 $A_{pq} \to A_{pr}A_{rq}$ = $4^3 =$ **64** แบบ 3 $A_{pp} \to \varepsilon$ = **4**
4. **pumping length**: $p = b^{|V|+1} = 3^{4+1} =$ **243**
5. **จุดตัด $w = xy$ เมื่อ $|w| = 5$**: ตัดได้ก่อนตัวแรกถึงหลังตัวสุดท้าย = $5 + 1 =$ **6 แบบ**

สอนแทรก

- ตัวเลขพวกนี้มักใช้ใน “อธิบายว่าทำไมหยุดเสมอ” — จำนวนจำกัด = ลองครบได้ = decider

13.5 จุดพลาดบ่อย (ทุกบท)

โจทย์ 13.5.1 — หาจุดผิด 5 ข้อ

แต่ละข้อผิดตรงไหน

1. “เพื่อพิสูจน์ว่า B undecidable เราแสดงว่า $B \leq_{\text{m}} A_{\text{TM}}$”
2. configuration หลัง $\delta(q,1) = (r,0,\text{L})$ จาก $01\,q\,1$ คือ $010\,r$
3. “E_{DFA} decider: ถ้า mark ถึง accept state ให้ accept”
4. “recognizable ปิด union: รัน M_1 จนจบ แล้วรัน M_2”
5. “PDA→CFG: push X จาก $p \to r$ คู่กับ pop X จาก $s \to q$ ได้ $A_{pq} \to aA_{sr}b$”

ลองทำเองก่อนดูแนวคิด/เฉลย

เฉลยละเอียด

1. **ทิศ reduction ผิด**
   - $B \leq_{\text{m}} A_{\text{TM}}$ บอกแค่ว่า B “ไม่ยากกว่า” A_{TM} — ไม่ได้บอกว่า B ยาก
   - ที่ถูก: แสดง $A_{\text{TM}} \leq_{\text{m}} B$ → ถ้า B decidable จะได้ A_{TM} decidable ขัดแย้ง
2. **configuration เลื่อนซ้ายผิด**
   - $01\,q\,1$: $u = 0$, $a = 1$ (ทางซ้ายหัว), $b = 1$ (ใต้หัว)
   - $\delta(q,1) = (r,0,\text{L})$ → เขียน 0 แทน b แล้ว state ไปอยู่หน้า a: $0\,r\,10$
   - คำตอบผิด $010\,r$ คือการเลื่อนขวา
3. **E_{DFA} ตอบกลับด้าน**
   - mark ถึง accept state = มี string ที่ accept = ภาษา**ไม่ว่าง**
   - E_{DFA} คือเซตของ DFA ที่ภาษาว่าง → ต้อง **reject**
4. **recognizable union รันต่อกันผิด**
   - M_1 อาจ loop บน $w \in L_2 \setminus L_1$ → ไม่ได้รัน M_2 → พลาด
   - ที่ถูก: **รันสลับทีละก้าว**
5. **ตัวห้อยไส้สลับ**
   - push $p \to r$, pop $s \to q$ → นอก $= pq$, ใน $=$ ปลาย push กับต้น pop $= rs$
   - ที่ถูก: $A_{pq} \to a\ A_{rs}\ b$

สอนแทรก

- 5 ข้อนี้คือ 5 จุดที่เสียคะแนนบ่อยที่สุดในข้อสอบ — ก่อนส่งให้เช็กทั้ง 5 อย่างนี้ทุกครั้ง